SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA37 – Software Testing

ASSIGNMENT 5 – CO5

Analyze, Compare and Justify Test Measurement, Monitoring, Configuration Management and Review Practices for a Motor Insurance Claims Platform Release

CO5: Analyze software testing activities to monitor quality, track progress and ensure effective control of software testing processes.

Student Name	TANUJA P	**Register Number**	192321052
Department / Section	IT	**Academic Year / Semester**	2025-2026/7th sem
Faculty Name	Dr Gnanajeyaraman	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Analyze Measurements and Milestones	4		
2	Analyze Status Reporting	4		
3	Analyze Test Monitoring and Control	5		
4	Analyze Test Completion Criteria	5		
5	Analyze Configuration Management	4		
6	Analyze Review Types	4		
7	Analyze Review Planning	4		
8	Analyze Review Reporting	6		
9	Integrate Quality Monitoring and Control	6		
10	Re-analyze and Adapt the Quality Control Practice	8		
TOTAL		50		Percentage:

OVERALL REMARKS

TANUJA P		
Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A metric table for weeks 5 and 9, a milestone table, a GQM tree, and a justified final measurement set.	✓	Table 2, Table 3, Table 7; Figure 1
2	Three status reports for week 6, a colour-rule table applied to weeks 3, 6 and 9, and a justified reporting format.	✓	Table 9, Figure 2–Figure 4, Table 10; Table 11
3	A gap calculation, a projection table for the three responses, and a justified control decision.	✓	Table 15, Table 16; Figure 6, Figure 7
4	A table applying the three criteria, a defect-trend chart with remaining defects, a cost comparison, and a justified stopping rule.	✓	Table 19, Table 21; Figure 8, Figure 9
5	Three configuration structures, an incident-replay table, a change-history comparison, and a justified SCM approach.	✓	Figure 10; Table 24, Table 23
6	A review comparison table with defects, time and savings, a list of review problems met, and a justified rule.	P	Table 27, Table 29; repeat with classmates as reviewers
7	Three review plans, an effort calculation against the 10% limit, an entry and exit criteria table, and a justified plan.	✓	Table 32, Table 33, Table 35
8	Review records in three formats, a metrics table, a follow-up traceability comparison, and a justified reporting approach.	✓	Table 37, Table 38, Table 40, Table 41, Table 42
9	A measured comparison of three configurations, an original block diagram and decision table, a transfer table, and a defended final practice.	✓	Table 45, Figure 22, Table 47, Table 48
10	Three adaptation strategies applied and compared with measured and calculated evidence, and a RETAIN / MODIFY / REDESIGN decision.	✓	Table 51, Table 54, Table 55; REDESIGN

QUALITY-CONTROL EVIDENCE CHAIN (Tasks 1–10)

Evidence item	Status	Observation
Metrics computed from the tracking table	✓	Table 2; one script, `sure.py`
Colour rules applied to weeks 1–9	✓	Table 11; Figure 5
Control responses projected with a calibrated model	✓	Table 16; model reproduces week 9 with no change
Remaining defects estimated by a fitted trend	✓	Table 20; Rayleigh $K \approx 271$
Wrong fraud_rules.json replayed in three SCM set-ups	✓	Table 24
Twelve seeded review defects confirmed by executable checks	✓	Table 26
Prototype run for three configurations	✓	Table 45; Figure 18–Figure 20
Sprint prototype with signed evidence records	✓	Table 51, Table 54

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 10 tasks answered	☒ Yes ☐ Partly ☐ No
Metrics, projections, trend fit and costs shown step by step	☒ Yes ☐ Partly ☐ No
Every measured result produced by running the supplied code (Annexure F)	☒ Yes ☐ Partly ☐ No
Assumptions stated and justified	☒ Yes ☐ Partly ☐ No

Requirement	Status
Charts, diagrams and the block diagram included and readable	☒ Yes ☐ Partly ☐ No
Cross-references and table / figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
Reviews repeated with classmates and LLM runs (Tasks 2, 6, 7, 8) recorded by the student	☐ Yes ☒ Partly ☐ No
Viva recommended	☐ Yes ☐ No

SOFTWARE TESTING

ASSIGNMENT 5 – ANSWERS

CO5: Analyze software testing activities to monitor quality, track progress and ensure effective control of software testing processes.

Assignment Title: Analyze, Compare and Justify Test Measurement, Monitoring, Configuration Management and Review Practices for a Motor Insurance Claims Platform Release (Total Marks – 50)

Duration: 1 week

Scenario – Release Quality Control of the "SureClaim" Motor Insurance Claims Platform v2.0

Harbour General Insurance is preparing version 2.0 of SureClaim, adding mobile claim intake with photo upload, automated damage estimate, fraud-flag rules, claim settlement payout and a garage-partner portal. The release is 12 weeks long. At the last release, management could not tell whether testing was on schedule, a test copy of the fraud-threshold configuration file was deployed to production, nobody knew when to stop testing, and design and code reviews were held only when someone had time. The release manager now wants a measurable, controlled and reviewed release.

You are the quality analyst. In every task, three candidate techniques are given. You must apply all three, analyze them, compare them, choose one, and justify your choice with evidence.

Parameter	Value
Release	About 22 KLOC new or changed code; 96 requirements/user stories; 1,400 planned test cases over a 12-week release (testing weeks 1–9, release readiness weeks 10–12)
Team	15 people: 1 release manager, 5 testers, 6 developers, 1 build and configuration engineer, 1 business analyst, 1 QA analyst
Open defects at week 9	20 open: 3 critical, 7 major, 10 minor; defect-density target ≤ 10 defects per KLOC
Coverage at week 9	Requirement coverage 90 % (86 of 96); statement 78 %, branch 64 %
Cost per defect	Review ₹2,500; system test ₹12,000; after release ₹60,000
Configuration items	Git source; fraud_rules.json; premium and claim-limit tables; build scripts; test scripts and data; documents; development, test and staging environments
Review status	No requirement review; informal design/code reviews; 10 reviewers available; reviews ≤ 10 % of project effort

Table 1: Weekly tracking data of the release (given) with derived weekly increments

Week	Planned (cum.)	Executed (cum.)	Passed (cum.)	Found (cum.)	Closed (cum.)	Found / week	Closed / week	Open
1	120	100	88	15	5	15	5	10
2	300	240	200	42	20	27	15	22
3	520	410	340	80	46	38	26	34
4	740	560	450	130	78	50	32	52
5	960	720	590	178	118	48	40	60
6	1150	900	750	214	156	36	38	58
7	1280	1080	920	238	190	24	34	48
8	1360	1230	1075	252	224	14	34	28
9	1400	1340	1210	260	240	8	16	20

ANSWERS – READING GUIDE

Each task is followed by its Ans: block in the order of the evaluation bullets. All measured results come from Python scripts (Annexure F): `sure.py` (metrics, colour rules, stopping criteria, trend fit, control-response model), `exp_mon.py` (Tasks 1–4), `exp_scm.py` (Task 5, real Git repositories), `exp_rev.py` and `settle_seeded.py` (Tasks 6–8), `qc_proto.py` (Task 9) and `qc10.py` (Task 10). Hours, rupee costs of effort and review detection by human reviewers are estimates from stated rules and are marked as such.

- **Assumptions:** cost of effort ₹700 per tester-hour and ₹800 per developer- or reviewer-hour; a critical defect found after release is costed at 1× and 5× ₹60,000 (both shown); 30 productive hours per person per week.
- **Prediction first.** Each task states the prediction made before the analysis and closes with the comparison.

TASK 1 – Measurements and Milestones: Compare Measurement Frameworks [4 Marks]

Management asks for "a few numbers that show whether testing is healthy". Compare three ways of choosing and using measurements for the release.

Technique	Basic idea and key concepts / tools
A. Plan-versus-actual (earned value) measures	Planned versus executed tests, effort spent versus planned, and milestone dates, shown as schedule and cost variance.
B. Defect-based measures	Defect arrival and closure rates, open defects by severity, defect density and defect removal efficiency.
C. Coverage and risk measures	Requirement, statement and branch coverage and the residual risk of the requirements not yet tested.

- Using the tracking table, calculate for week 5 and week 9: percentage executed, pass rate, schedule variance, defect find rate, defect closure rate, open defects and defect density.
- Define five release milestones with dates and measurable completion conditions and mark which were met by the data.
- Analyze the weekly data to determine which measure first signalled trouble, and by how many weeks it preceded the problem visible at week 9.
- Examine the risks of misusing each type of measure and give one example for each approach.
- Evaluate which approach gives management the most useful and least costly set of measures and justify a final set of at most eight metrics.
- Derive a Goal–Question–Metric tree for the goal "release SureClaim with no open critical defect" and judge which metrics can be collected automatically.

Evidence Expected: A metric table for weeks 5 and 9, a milestone table, a GQM tree, and a justified final measurement set.

Ans:

Measuring Whether SureClaim Testing Is Healthy

Prediction. Plan-versus-actual will show trouble first (the plan was never met); defect measures will explain why; coverage will be the most expensive to collect but the only one that shows what is still untested.

1. Metrics for week 5 and week 9

Table 2: Metric table for weeks 5 and 9 (calculated by sure.metrics)

Metric	Week 5 calculation	Week 5	Week 9 calculation	Week 9
Percentage executed (executed ÷ planned to date)	720 ÷ 960	75.0 %	1,340 ÷ 1,400	95.7 %
Executed as % of all 1,400 tests	720 ÷ 1,400	51.4 %	1,340 ÷ 1,400	95.7 %
Pass rate of executed tests	590 ÷ 720	81.9 %	1,210 ÷ 1,340	90.3 %
Schedule variance (executed − planned)	720 − 960	-240 (-25.0 %)	1,340 − 1,400	-60 (-4.3 %)
Defect find rate (new defects that week)	178 − 130	48 / week	260 − 252	8 / week
Defect closure rate (closed that week)	118 − 78	40 / week	240 − 224	16 / week
Open defects (found − closed)	178 − 118	60	260 − 240	20

Metric	Week 5 calculation	Week 5	Week 9 calculation	Week 9
Defect density (found ÷ 22 KLOC)	178 ÷ 22	8.1 / KLOC	260 ÷ 22	11.8 / KLOC (target ≤ 10)
Closure ratio (closed ÷ found)	118 ÷ 178	66.3 %	240 ÷ 260	92.3 %
Defects found per 100 tests executed	178 ÷ 7.2	24.7	260 ÷ 13.4	19.4

Week 5 shows a project 25 % behind plan with one test in five failing and the backlog still growing (find rate 48 > closure rate 40). Week 9 looks better on schedule (−4.3 %) and pass rate (90.3 %), but defect density is **11.8 per KLOC, above the target of 10**, and 3 critical defects are open.

2. Release milestones

Table 3: Five milestones with dates, measurable conditions and the result from the data

ID	Date	Completion condition	Observed	Met?
M1	End of week 2	Test design complete; ≥ 20 % of planned tests executed by end of week 2	17.1 % executed	**No**
M2	End of week 5	50 % of planned tests executed by end of week 5	51.4 % executed	**Yes**
M3	End of week 7	90 % of planned tests executed by end of week 7	77.1 % executed	**No**
M4	End of week 8	Defect density ≤ 10/KLOC and open defects ≤ 30 at end of week 8	density 11.5, open 28	**No**
M5	End of week 9	100 % executed, pass ≥ 95 %, 0 open critical, requirement coverage 100 % by end of week 9	95.7 % executed, pass 90.3 %, 3 critical open, coverage 90 %	**No**

Only M2 was met, and only because it was set on the share of all 1,400 tests (51.4 %) rather than on the week-5 plan (75 %). The milestones that matter for release (M4, M5) were missed.

3. Which measure signalled trouble first?

Table 4: First week each measure crossed its threshold (two weeks running, density one week; measured)

Measure and threshold	Approach	First signal	Weeks before week 9
Schedule variance > 15 % behind	A	Week 1	8
Open defects rising week on week	B	Week 2	7
Closure ratio < 80 %	B	Week 1	8
Pass rate < 85 %	B	Week 2	7
Defect density > 10/KLOC	B	Week 7	2
Weekly execution below weekly plan	A	Week 1	8
Requirement / branch coverage	C	Not measurable (week-9 data only)	–

The **schedule variance** (A) and the **closure ratio** (B) both signalled in weeks 1–2 and never recovered until week 7–8: they preceded the week-9 problem (tests still unexecuted, 3 critical defects open) by **7–8 weeks**. The **open-defect trend** signalled from week 2 (7 weeks ahead) and is the better early warning of the quality problem: work was arriving faster than it was being closed. **Defect density** crossed the target only in week 7, 2 weeks before. Coverage measures (C) exist only at week 9 in the data, so they could not warn at all: the data were not collected weekly.

4. Risks of misusing each type of measure

Table 5: Misuse risks with one example for each approach

Approach	Misuse risk	Example on SureClaim	Safeguard
A Plan vs actual	Rewarding speed of execution	Testers run easy UI tests first to close the gap; the fraud-rule and payout tests (the risky ones) are left to week 9	Report progress by risk class, not one total
B Defect-based	Rewarding defect counts or punishing developers for them	Testers raise one defect per wrong label to hit a target; developers argue severities down so that 'critical open' looks like 0	Use defect data for the product, never for individual appraisal; severity set by a triage group
C Coverage and risk	Treating coverage as quality	Branch coverage raised from 64 % to 80 % by tests with no assertions on the payout amount	Require an assertion per covered decision; review a sample of tests

5. Evaluation and final measurement set

Table 6: Comparison of the three measurement approaches

Criterion	A Plan vs actual	B Defect-based	C Coverage and risk
Early warning in this data	Week 1	Week 1–2 (closure) / week 2 (open)	None (only week 9 data)
Answers 'can we release?'	No	Partly (open critical, trend)	Partly (what is untested)
Cost to collect	Very low (TM tool)	Low (JIRA)	Medium (CI coverage per build, risk mapping)
Easy to misread	Yes – looks fine at 95.7 %	Moderate	Yes – high coverage ≠ good tests

Table 7: Final measurement set (eight metrics) and why each is kept

#	Metric	Approach	Why it is kept	Collected
1	Schedule variance (% of planned executed)	A	Earliest warning (week 1)	Auto – TM tool
2	Pass rate of executed tests	A/B	Product stability; read with #3	Auto – TM tool
3	Open defects by severity (and age of oldest critical)	B	Directly answers the release goal	Auto – JIRA
4	Weekly find and closure rates (backlog trend)	B	Shows whether fixing keeps up	Auto – JIRA
5	Defect density per KLOC	B	Target ≤ 10 is a stated release criterion	Auto – JIRA + KLOC from Git
6	Requirement coverage, high-risk stories first	C	Shows what is untested	Auto – TM tool links
7	Branch coverage of claim, fraud and payout code	C	Structural gaps in the risky code	Auto – CI
8	Estimated remaining defects (trend fit)	B	Basis of the stopping rule (Task 4)	Semi – script on JIRA export

Decision: approach B as the core, with one metric each from A and C. Defect measures gave the most useful warning and are free to collect from JIRA; schedule variance is kept because it warned earliest; coverage of the risky code and requirements is kept because neither A nor B shows untested areas. Seven of the eight metrics can be collected automatically.

6. GQM tree for "release SureClaim with no open critical defect"

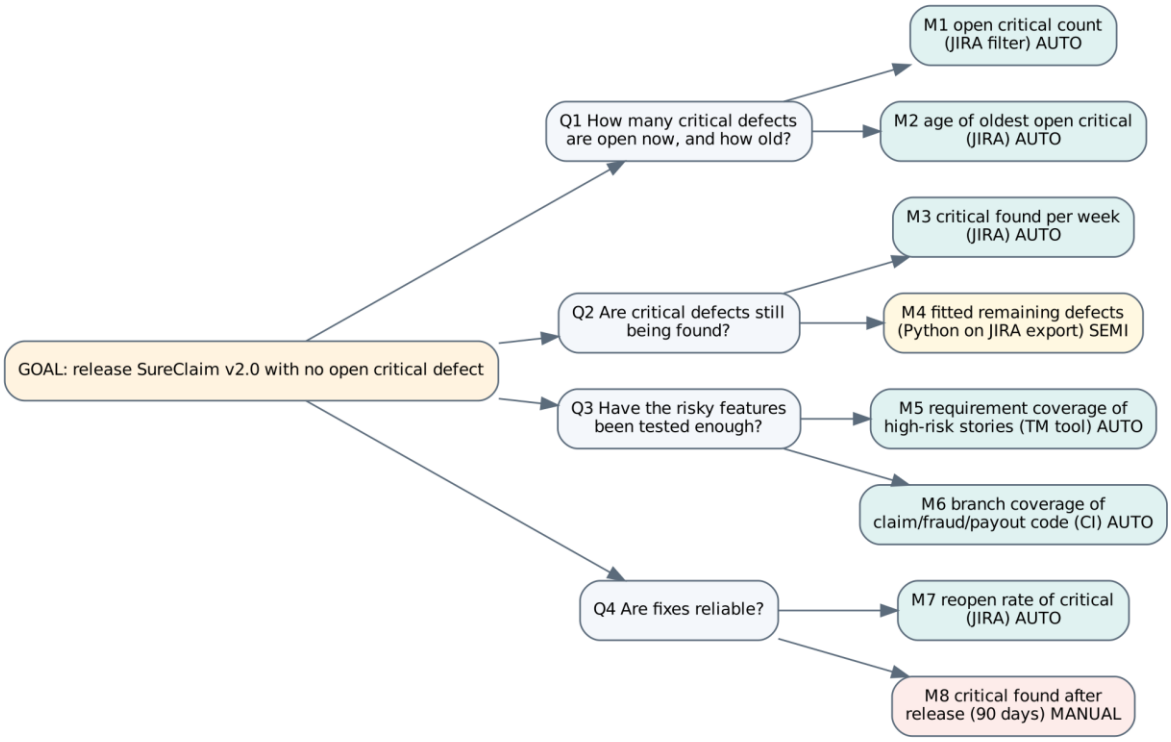


Figure 1: Goal–Question–Metric tree; green = automatic, yellow = semi-automatic, red = manual

Table 8: GQM metrics and automatic collection with the available tools

Question	Metric	Tool	Automatic?
Q1 How many critical defects are open, and how old?	M1 open critical; M2 age of oldest	JIRA filter / dashboard gadget	Yes
Q2 Are critical defects still being found?	M3 critical found per week; M4 fitted remaining	JIRA; Python fit on export	M3 yes; M4 scripted
Q3 Have the risky features been tested enough?	M5 requirement coverage of high-risk stories; M6 branch coverage of claim/fraud/payout	TM tool; coverage.py in CI	Yes
Q4 Are fixes reliable?	M7 reopen rate of critical; M8 critical found after release	JIRA; production incident log	M7 yes; M8 manual (after release)

7. Supporting evidence for Task 1

- `sure.py` (metrics) and `exp_mon.py` (milestones, signals) – Annexure F; results in `results_mon.json`.

Threats to validity

Thresholds (15 %, 80 %, 85 %) are chosen by the analyst; with other thresholds the signal weeks shift by about a week but the order (schedule and backlog first, density last) stays.

Marking notes and common errors

Common errors: computing schedule variance against 1,400 instead of the plan to date; reporting density without the target; a GQM tree whose metrics cannot be collected.

TASK 2 – Status Reports: Compare Status Reporting Formats [4 Marks]

The release manager receives a long weekly e-mail that nobody reads. Compare three formats for reporting test status.

Technique	Basic idea and key concepts / tools
A. Red–amber–green summary	A one-page report with a colour status for scope, schedule, quality and risk, each with a one-line comment.
B. Metric-chart pack	Charts of cumulative planned, executed and passed tests, defects found and closed, and open defects by severity.
C. Daily tool-generated status	A live dashboard in JIRA or the test-management tool, with a five-minute daily call and no written report.

- Prepare the status report for week 6 in each of the three formats, with at least three charts for format B.
- Define the colour rules for format A and apply them to weeks 3, 6 and 9.
- Analyze what each format reveals and hides about the trend: which format would have shown the widening gap earliest?
- Analyze the effort of producing each format, the audience that can understand it, and its risk of giving false comfort.
- Evaluate the format the release manager should receive each week and the one the testing team should use daily.
- Critique a status report written by an LLM from the tracking table.

Evidence Expected: Three status reports for week 6, a colour-rule table applied to weeks 3, 6 and 9, and a justified reporting format.

Ans:

Reporting SureClaim Test Status So That It Is Read

Prediction. The chart pack (B) will show the widening gap earliest; the RAG page (A) will be read but can hide trends; the live dashboard (C) is best for the team but leaves no record.

1. Week-6 status in three formats

Table 9: Format A – one-page red–amber–green status, week 6

Area	Status	One-line comment
Schedule	RED	900 of 1,150 planned tests executed (78.3 %); gap 250 and widening since week 1
Quality	RED	Pass rate 83.3 %; 58 open defects; oldest critical open 8 days
Scope	AMBER	About 59 of 96 stories have an executed test (week-6 target 79)
Risk	AMBER	Closure ratio 72.9 %; backlog fell for the first time (60 → 58)
Decision needed	–	Approve the cause-removal actions of Task 3 (environment, fix turnaround)

Format B – metric-chart pack, week 6 (three charts):

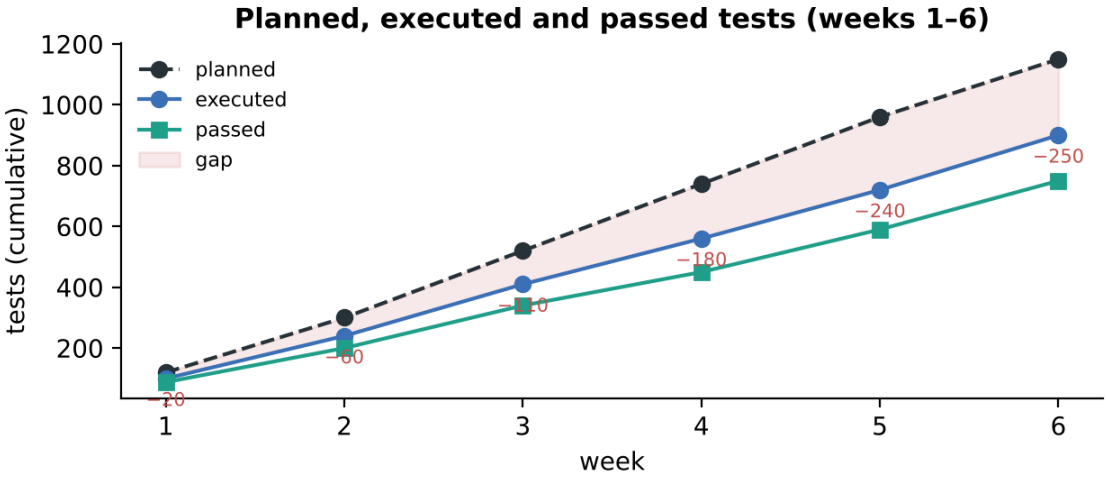


Figure 2: Chart B1 – cumulative planned, executed and passed tests to week 6, with the gap

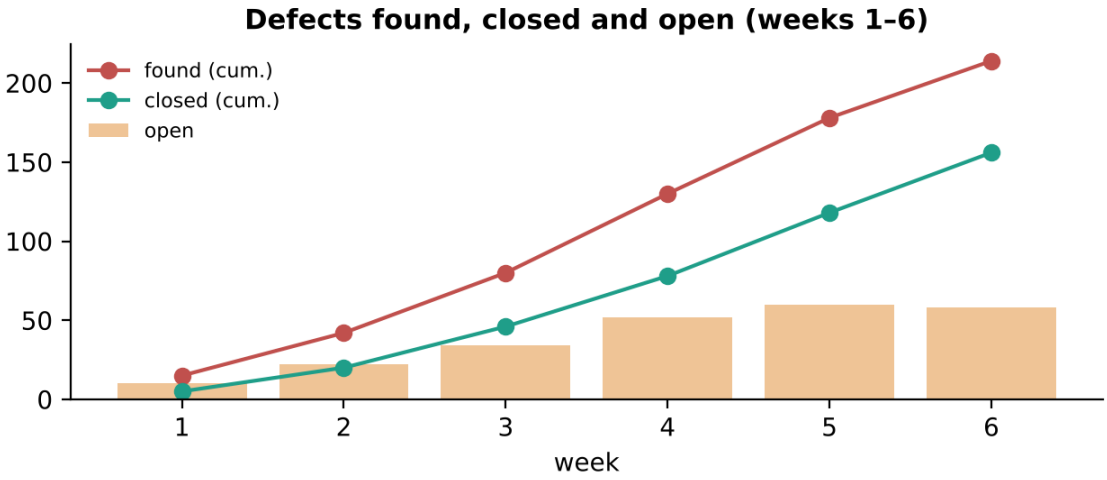


Figure 3: Chart B2 – defects found, closed and open to week 6

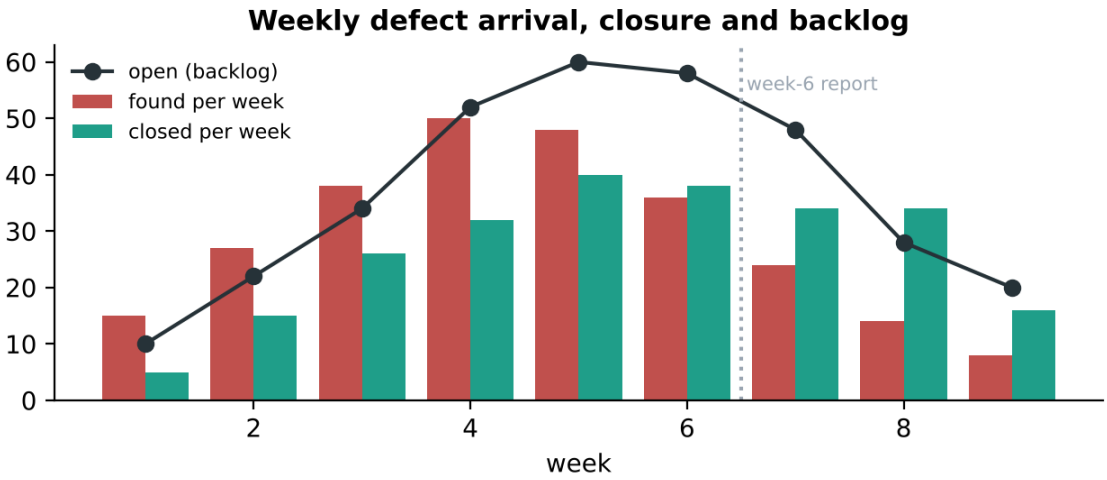


Figure 4: Chart B3 – weekly arrival and closure with the backlog (the week-6 report sees weeks 1–6)

Table 10: Format C – live dashboard gadgets shown in the 5-minute daily call (values at the end of week 6)

Gadget	Value	Source
Tests executed today / planned today	36 / 38 (average of week 6: 180 / 190)	TM tool
Blocked tests (reason)	e.g. 7 – staging down 2 h	TM tool
Open defects by severity	58 open (severity split from JIRA)	JIRA filter
Defects created vs resolved (7 days)	36 vs 38	JIRA chart
Oldest critical	8 days	JIRA
Builds deployed to test today	1 (tag v2.0.0-rc6)	CI

2. Colour rules applied to weeks 3, 6 and 9

Table 11: Colour rules (format A) and their application

Area	Rule	Week 3	Week 6	Week 9
Schedule	RED if > 15 % of planned tests not executed; AMBER if 5–15 %; else GREEN	RED	RED	GREEN
Quality	RED if any critical defect open > 7 days or open defects rising for 2 weeks; AMBER if pass rate < 85 %; else GREEN	RED	RED	RED
Scope	RED if requirement coverage < 70 % of the week's target; AMBER if < 90 %; else GREEN	AMBER	AMBER	AMBER
Risk	RED if find rate rising for 2 weeks after week 4; AMBER if closure ratio < 80 %; else GREEN	AMBER	AMBER	GREEN

Inputs: schedule -21.2 %, -21.7 %, -4.3 %; oldest critical open 9, 8 and 12 days (assumed from the tracker); open defects 34, 58, 20; closure ratio 57.5 %, 72.9 %, 92.3 %. Week 9 shows schedule GREEN but quality RED: the 3 critical defects open for 12 days must stop a 'nearly done' message.

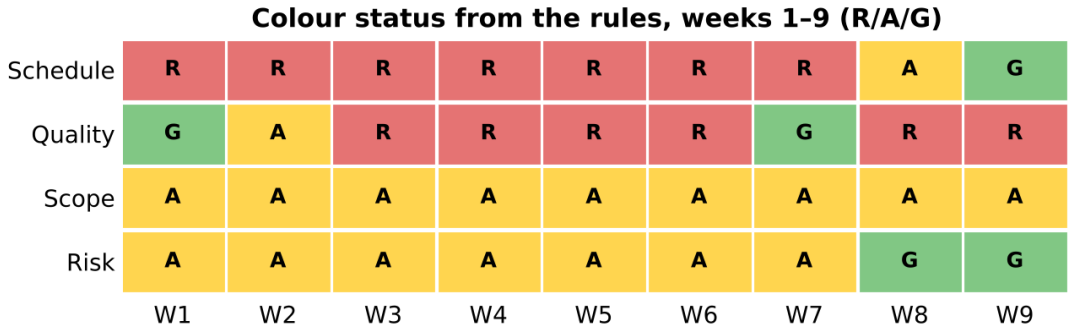


Figure 5: Colour status for all nine weeks under the rules

3. What each format reveals and hides

Table 12: Trend visibility of the three formats

Format	Reveals	Hides	Gap visible from
A RAG	Current state, decisions needed	Size and direction of trends (RED in week 1 and week 6 look the same)	Week 1 (RED), but not that it was widening
B Charts	Trends: widening gap weeks 1–6, backlog peak at week 5	Daily blockers; needs reading time	Week 2–3 (gap 60 → 110 visibly widening)
C Dashboard	Today's blockers and counts	Weekly trend unless someone opens the chart; no history in the call	Day by day, if anyone looks at the cumulative gadget

The **chart pack (B)** would have shown the widening gap earliest as a trend: the shaded gap in Chart B1 grows every week from 20 to 250. The RAG page showed RED from week 1 but the same RED for six weeks, which people learn to ignore.

4. Effort, audience and false comfort

Table 13: Effort, audience and risk of false comfort (effort estimated)

Format	Effort per week	Audience	Risk of false comfort
A RAG	0.5 h (rules computed by script)	Release manager, sponsors	Medium – GREEN schedule at week 9 can hide RED quality if read alone
B Charts	1 h (script draws charts; 0.5 h commentary)	Release manager, test lead, QA	Low – trends visible; cumulative charts flatten bad weeks
C Dashboard	0 h report + 5 min × 15 people × 5 days = 6.25 h of call time	Testers, developers	High for management – no record, no trend, depends on who attends

5. Evaluation

Decision: the release manager receives A + B each week – the RAG page on top, the three charts behind it – and the team uses C daily. A is read because it is one page; B prevents a RED from becoming background noise by showing whether it is getting better or worse; both are generated by the same script (Task 9), so the effort is about 1.5 h per week. C suits the team because blockers must be cleared within a day, but it leaves no record for the release decision or for the auditors.

6. Critique of an LLM-written status report

The tracking table for weeks 1–6 is given to an LLM with the request "write a weekly test status report for week 6". Each sentence is then checked against the fact sheet below, generated from the data by `sure.metrics`, and classified as correct, wrong, unsupported or invented.

Table 14: Fact sheet for checking the LLM status report (student fills the last column)

Topic	Fact (week 6)	LLM statement and verdict
Progress	900 of 1,150 planned (78.3 %); 64.3 % of all 1,400	
Gap trend	Gap 250, up from 240 in week 5 (widening)	
Pass rate	750 ÷ 900 = 83.3 %	
Open defects	214 − 156 = 58; first fall (from 60)	
Severity of open defects	Not in the week-6 data	
Forecast of completion	At 180 tests/week, 250 tests behind → about 1.4 weeks late	

Typical checks: an LLM may compute pass rate against planned rather than executed tests, describe the backlog as 'under control' without a trend, or state severities that the table does not contain (unsupported). An LLM report is acceptable only as a draft whose numbers are regenerated from the data.

7. Supporting evidence for Task 2

- `sure.rag()` applies the colour rules; `figs5.py` draws the charts (Annexure F).

Threats to validity

The age of the oldest critical defect and weekly requirement coverage are not in the given data; they are assumptions listed in `sure.py` and would come from JIRA and the TM tool.

Marking notes and common errors

Common errors: colour rules that cannot be computed; a chart pack with only cumulative charts; claiming the dashboard replaces the record needed for the release decision.

TASK 3 – Test Monitoring and Control: Compare Control Responses to a Slipping Test Schedule [5 Marks]

By week 5 the executed tests are 25% behind plan (720 executed against 960 planned) and the gap keeps growing. Compare three control responses.

Technique	Basic idea and key concepts / tools
A. Add effort	Overtime and two additional temporary testers for four weeks.
B. Re-prioritise and descope	Rank the remaining tests by risk and drop or defer the lowest-risk 20%.
C. Remove the cause of the delay	Find the causes of blocked and failed tests (environment downtime, slow defect fixing, unstable builds) and correct them.

- Calculate for weeks 1 to 9 the gap, the average weekly execution rate, and the week in which the plan would have been completed if the week-5 rate had continued.
- Simulate each response from week 5 onwards with stated assumptions and project the executed tests, defects found and open defects at week 9.
- Analyze the trade-offs of each response in cost, quality risk, team morale and the likelihood of fixing the real cause.
- Examine the defect closure data and what the backlog pattern suggests about the control actions needed and when they were taken.
- Evaluate the response or combination the release manager should choose at week 5.
- Judge the statement "adding people to a late test effort makes it later" using your simulation.

Evidence Expected: A gap calculation, a projection table for the three responses, and a justified control decision.

Ans:

Controlling a Slipping Test Schedule

Prediction. Adding testers (A) will be slower in the first week and expensive; descoping (B) will meet the date by testing less; removing the cause (C) will give the best result if the causes are real.

1. Gap, execution rate and projected completion

Table 15: Gap between planned and executed tests, weekly and average execution rate (calculated)

Week	Planned	Executed	Gap	Executed this week	Average rate to date
1	120	100	20	100	100
2	300	240	60	140	120
3	520	410	110	170	137
4	740	560	180	150	140
5	960	720	240	160	144
6	1150	900	250	180	150
7	1280	1080	200	180	154
8	1360	1230	130	150	154
9	1400	1340	60	110	149

At week 5, 680 tests remained. At the week-5 rate of 160 tests/week the plan needs 4.25 more weeks, finishing in **week 10** (5 + 4.25); at the average rate of weeks 1–5 (144/week) it needs 4.72 weeks (week 10, end). The plan assumed 220 tests/week in week 5; the team never exceeded 180.

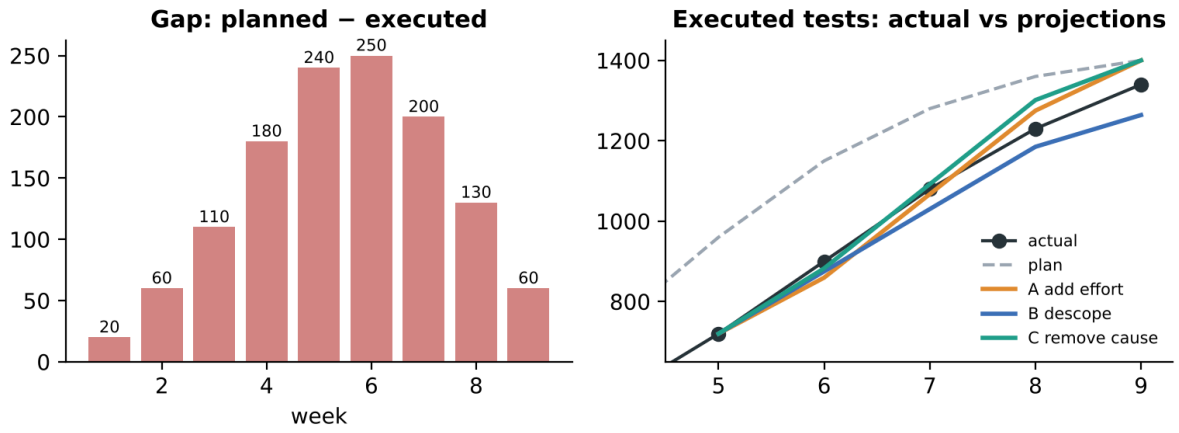


Figure 6: Gap by week, and executed tests: actual against the three projected responses

2. Simulation of the three responses from week 5

Model (sure.project). The team executes a base 155 tests per week (average of weeks 2–5). Defects found per test start at 0.30 (week 5: 48 defects ÷ 160 tests) and fall by 30 % each week as the product stabilises. Closing capacity is 85 % of the average closures of weeks 4–5. The two decay and capacity constants were **calibrated** so that the model with no change reproduces the actual week 9 (1,340 executed, 260 found, 20 open); the model then predicts the three responses.

- **A Add effort:** 10 % overtime from week 6; two temporary testers who reach 60 % of a tester's productivity after one week of training, during which they take half a senior tester's time each; temporary testers ₹900/h and overtime ₹600/h (assumed).
- **B Descope:** the lowest-risk 20 % of the remaining 680 tests are deferred (136 tests). Dropped tests are low-risk but not defect-free (defect yield of the rest 15 % lower).
- **C Remove the cause:** fixing environment downtime and build stability raises the execution rate by 35 % from week 7 (5 % in week 6 while fixing); faster defect turnaround raises closures by 40 %.

Table 16: Projection: executed / found / open at weeks 6–9 for each response (model)

Week	Actual (no change)	A Add effort	B Descope	C Remove cause
6	900 / 214 / 58	860 / 207 / 56	875 / 206 / 57	883 / 212 / 51
7	1080 / 238 / 48	1067 / 238 / 53	1030 / 225 / 46	1092 / 243 / 39
8	1230 / 252 / 28	1275 / 259 / 40	1185 / 239 / 29	1301 / 264 / 18
9	1340 / 260 / 20	1400 / 268 / 16	1264 / 243 / 3	1400 / 272 / 0
Cost	–	₹2,88,000	₹0 (136 tests deferred)	Effort of the team (≈ 40 h of build/env work)

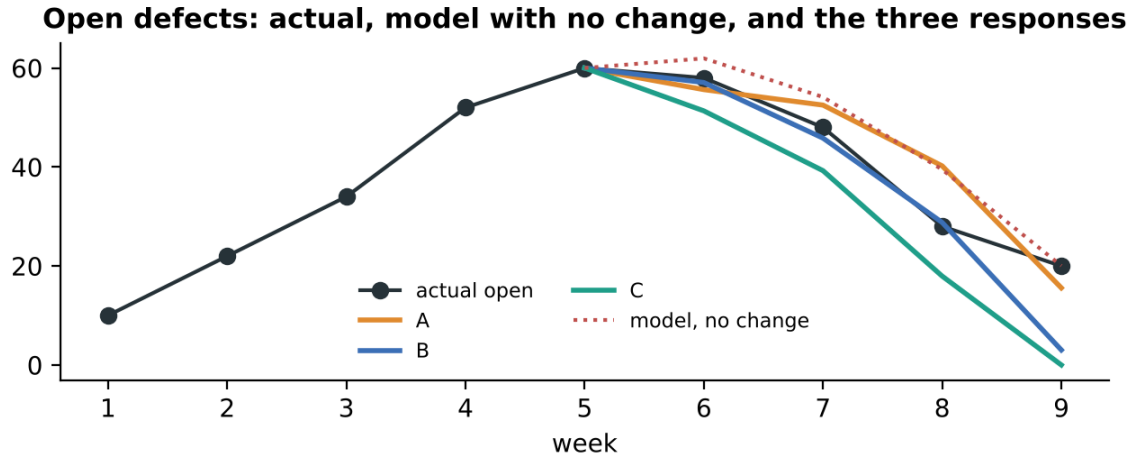


Figure 7: Open defects: actual, model without change and the three responses

Table 17: Sensitivity of week-9 results to the assumptions (executed / found / open)

Variant	Week 9 result
A ramp 0.3	1400 / 266 / 14
A ramp 0.8	1400 / 270 / 17
C fix gain 0.15	1400 / 268 / 0
C fix gain 0.0	1348 / 262 / 0
B descope 10 %	1332 / 248 / 7
C no gains (0, 1.0)	1348 / 262 / 22
C half gains (0.15, 1.2)	1400 / 268 / 3

3. Trade-offs

Table 18: Trade-offs of the three responses

Criterion	A Add effort	B Descope	C Remove cause
Cost	₹2,88,000 (model)	None in money	Team time; small tooling cost
Week-9 executed	1,400 (all)	1,264 (136 deferred)	1,400 if causes are real
Quality risk	New testers miss domain defects; 16 open	Deferred tests may hide defects; release with untested low-risk areas	Lowest – also fixes closure speed; 0 open in model, 22 if gains do not appear
Morale	Overtime for 4 weeks; training load	Testers dislike dropping planned work	Positive – removes daily frustrations
Fixes the real cause?	No	No	Yes, by definition – if diagnosed correctly

4. What the backlog pattern says

Open defects rose from 10 (week 1) to 60 (week 5) because defects arrived faster than they were closed every week from 1 to 5 (15 vs 5, 27 vs 15, 38 vs 26, 50 vs 32, 48 vs 40). The backlog fell only from week 6, when closures (38, 34, 34) finally exceeded arrivals (36, 24, 14). The pattern says the **bottleneck was defect fixing, not test execution alone**. Blocked and failed tests waiting for fixes slowed execution, so the schedule gap and the backlog are the same problem. The control action that worked (from week 6) came about 3–4 weeks late: the closure ratio had been below 80 % from week 1.

5. Decision at week 5

Decision: C first, with a small B as a safety valve. Remove the causes (dedicated staging slot, daily stable build, 2-day fix target for critical and major defects) from week 5, and prepare a risk-ranked list of the lowest 10 % of tests to defer only if week 7 is still more than 10 % behind. The model gives C 1,400 executed and the backlog closed by week 9; even with half the assumed gains it gives 1,400 executed and 3 open (sensitivity table). A costs ₹2,88,000 and still leaves 16 open defects, because extra testers find more defects without fixing them faster.

6. "Adding people to a late test effort makes it later"

The simulation supports the statement **for the first week only**: with two trainees taking senior time, A executed 860 tests by week 6 against 875 without change. From week 7 A is ahead, and it reaches all 1,400 tests by week 9. Brooks's law holds where the work needs scarce knowledge (fraud rules, payout flows) and where the bottleneck is elsewhere: in our data the bottleneck was fixing, so more testers raised the defect inflow and kept the backlog at 16, but did not make the release later. Adding people is useful only after the cause is removed and when the work can be split.

7. Supporting evidence for Task 3

- `sure.project()` and `exp_mon.py` (Annexure F); calibration shown in the 'Actual' column against the model with no change.

Threats to validity

The productivity, decay and closure assumptions drive the result; the sensitivity table shows the decision is unchanged unless C produces no gain at all, in which case C equals doing nothing and B becomes the fallback.

Marking notes and common errors

Common errors: projecting with the planned rate instead of the actual; ignoring the training effect; analysing the schedule without the defect backlog.

TASK 4 – Criteria for Test Completion: Compare Stopping Criteria [5 Marks]

Nobody knows when to stop testing SureClaim. Compare three ways of deciding that testing is complete.

Technique	Basic idea and key concepts / tools
A. Test-case completion	Stop when 100% of planned tests are executed and at least 95% of executed tests pass.
B. Defect-trend stopping	Stop when the weekly defect discovery rate falls below a stated threshold for two weeks and no critical defect is open.
C. Risk and cost-based stopping	Stop when the expected cost of the remaining defects after release is less than the cost of further testing.

- Apply each criterion to the tracking data and open-defect information: state for each whether testing could stop at week 9, and if not, when it could.
- Fit a simple trend to the weekly defect discovery and estimate the number of defects still undetected.
- Using the cost data, calculate the expected cost of releasing at week 9 against the cost of two more weeks of testing.
- Analyze the weakness of each criterion.
- Evaluate which criterion or combination should be written into the release policy, and justify the thresholds.
- Judge the statement "testing is never complete; it only stops".

Evidence Expected: A table applying the three criteria to the data, a defect-trend chart with an estimate of remaining defects, a cost comparison, and a justified stopping rule.

Ans:

Deciding When SureClaim Testing Can Stop

Prediction. None of the three criteria will allow stopping at week 9; the trend will show about 10–20 defects still undetected.

1. The three criteria applied to the data

Table 19: Stopping criteria applied at week 9 (measured from the data)

Criterion	Values at week 9	Stop at week 9?	When it could stop
A 100 % executed and ≥ 95 % pass	1,340 of 1,400 (95.7 %); pass 1,210 ÷ 1,340 = 90.3 %	No	After the 60 remaining tests and a re-run of the 130 failed tests (90 % expected to pass): 98.9 % → end of week 10–11
B discovery < 10/week for 2 weeks and 0 open critical	Last two weeks 14, 8; 3 critical open	No	Forecast week 10: 5.1 new defects → two weeks below 10 at week 10 if the 3 criticals are closed
C expected cost of release < cost of 2 more weeks	Release now ₹8.65 lakh (critical ×1) / ₹15.85 lakh (×5) vs ₹7.74 lakh	No	After 2 more weeks the remaining ≈ 2 defects cost less than further testing

2. Trend fit and undetected defects

Table 20: Trend models fitted to cumulative defects (scipy curve_fit; measured)

Model	Formula	Parameters	Sum of squared errors	Total defects K	Still undetected
Rayleigh (rise and fall)	K(1 − e^(−t²/2σ²))	σ = 3.46 weeks	67	271.4 ± 6.0	11.4 (95 % range 5–17)
Exponential (Goel–Okumoto)	a(1 − e^(−bt))	b = 0.021	2708	1652	Not credible – model does not fit an S-shaped curve
Rayleigh fitted on weeks 1–6 only	same	σ = 3.67	34	291 ± 26	At week 6 the model already said ≈ 290 total

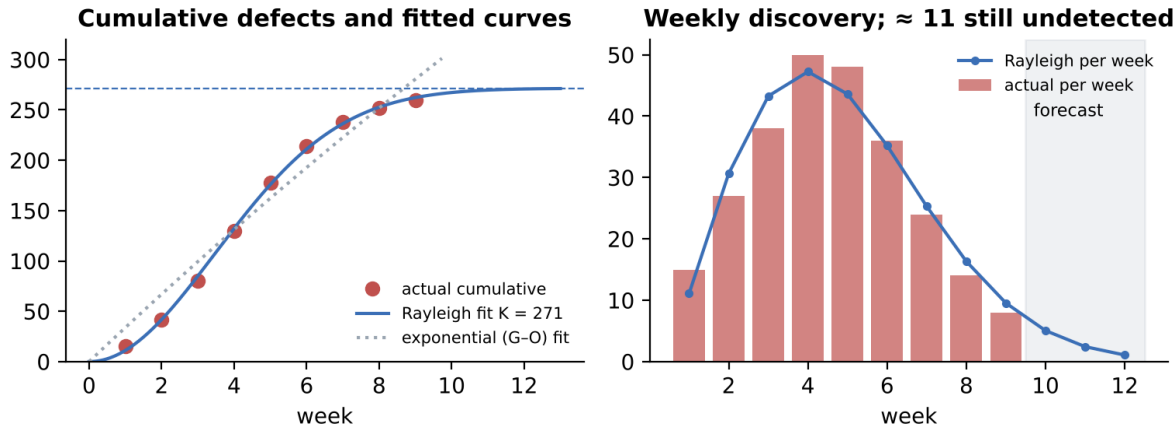


Figure 8: Cumulative defects with fitted curves, and weekly discovery with the Rayleigh forecast

The weekly discovery rose to a peak in week 4 and fell, the shape of a Rayleigh curve, which fits with a sum of squared errors of 67 (the exponential model fits 40 times worse). It estimates **K ≈ 271 defects in total, so about 11 remain undetected** (between 5 and 17). It forecasts 5.1, 2.4 and 1.1 new defects in weeks 10–12; two more weeks of testing would find about 81 % of the remainder.

3. Cost of releasing at week 9 against two more weeks

Table 21: Expected cost comparison (calculated)

Item	Release at week 9	Test two more weeks
Undetected defects	11.4 × ₹60,000 = ₹6,85,265	2.2 left × ₹60,000 = ₹1,29,035; 9.3 found × ₹12,000 = ₹1,11,246
3 known critical defects	3 × ₹60,000 = ₹1,80,000 (or 3 × ₹3,00,000 = ₹9,00,000 if a critical costs 5×)	3 × ₹12,000 = ₹36,000 (fixed before release)
Cost of the two weeks	–	5 testers × 60 h × ₹700 + 6 developers × 60 h × ₹800 = ₹4,98,000
Total	**₹8.65 lakh** (×1) / **₹15.85 lakh** (×5)	**₹7.74 lakh**

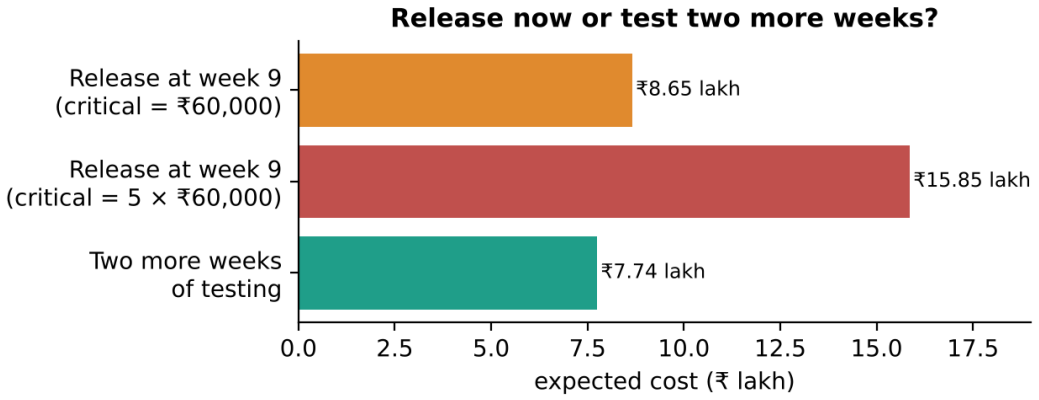


Figure 9: Expected cost of releasing now against two more weeks of testing

Even when a critical defect costs only the standard ₹60,000 after release, two more weeks are cheaper (₹7.74 lakh against ₹8.65 lakh). If a critical defect in a payout or fraud rule costs five times more (refunds, regulator, reputation), releasing now costs twice as much as testing on.

4. Weaknesses of each criterion

Table 22: Weakness of each criterion

Criterion	Weakness	Seen in this data?
A Completion	Measures effort, not quality; a planned test that is weak counts the same as a strong one; 95 % pass can include a critical failure	Yes – 95.7 % executed looked close to done while 3 critical were open
B Defect trend	Discovery can fall because testers are exhausted, tests are repeated, or the remaining tests are easy, not because quality is high	Possible – the fall from 24 to 8 coincides with the end of new test design
C Risk and cost	Depends on estimates (remaining defects, field cost) that are uncertain; can be tuned to justify a wanted date	Range 5–17 remaining defects; the decision holds across it

5. The stopping rule for the release policy

Decision: a combination – B as the trigger, A and C as conditions. Testing may stop when (1) fewer than **10 new defects per week for two consecutive weeks** (about 4 % of the total found, and where the Rayleigh curve has passed 95 % of K); (2) **0 open critical and ≤ 5 open major** with workarounds; (3) **100 % of high-risk tests executed and passed**, with any low-risk test not run listed and accepted; and (4) the **cost check** shows that releasing is cheaper than two more weeks. With the data, this gives a stop at the end of week 10 or 11, inside the readiness window of weeks 10–12.

6. "Testing is never complete; it only stops"

The statement is true: the model still expects about 2 to 11 defects in the product at release, whatever criterion is used. The consequence is that the release decision must be explained to management as a **risk decision with stated residual risk**, not as 'testing finished': "we stop because new defects have fallen below 10 a week, no critical defect is open, all high-risk tests pass, and we expect about 2 defects to remain, at an expected cost of about ₹1.3 lakh, which is less than the cost of testing longer."

7. Supporting evidence for Task 4

- `sure.fit_rayleigh()`, `sure.fit_go()`, `sure.crit_A/B/C()` and `exp_mon.py` (Annexure F).

Threats to validity

The Rayleigh model assumes a constant test effort; the effort fell in week 9 (110 tests). The field costs of critical defects are assumptions; the decision is the same at 1× and 5×.

Marking notes and common errors

Common errors: stopping on test completion alone; fitting an exponential model to S-shaped data; leaving the cost of the extra testing out of the comparison.

TASK 5 – Software Configuration Management: Compare Configuration Control Approaches [4 Marks]

The test copy of fraud_rules.json reached production. Compare three configuration management approaches that could have prevented it.

Technique	Basic idea and key concepts / tools
A. Shared-folder versioning by file name	Files are copied to a shared folder with names such as fraud_rules_final_v3.json.
B. Git-based version control	All configuration items are in Git with branches, tags and pull requests; deployments are made from tags.
C. Full SCM with baselines and change control	Git, plus formal baselines, a change control board for configuration files, and an automated audit that compares the deployed files with the baseline.

- Set up a small repository (or folder structure) for each approach with the code, fraud_rules.json, test scripts and a release note; carry out three changes in each.
- Replay the incident: deploy the wrong version of fraud_rules.json in each approach, and record whether and how it is detected and how long recovery takes.
- Analyze each approach on traceability, reproducing a past release, prevention of unauthorised change, and effort.
- Examine the relation between configuration management and testing.
- Evaluate which approach the company should adopt, and which items must be placed under change control.
- Critique the proposal to let an AI tool change configuration files automatically to tune the fraud threshold.

Evidence Expected: Three configuration structures, an incident-replay table, a change-history comparison, and a justified SCM approach.

Ans:

Keeping the Fraud Rules Under Control

Prediction. Only C will detect the wrong file without a human noticing; B will make recovery fast but will not detect the error; A will neither detect it nor allow a clean recovery.

1. Three configuration structures and three changes in each

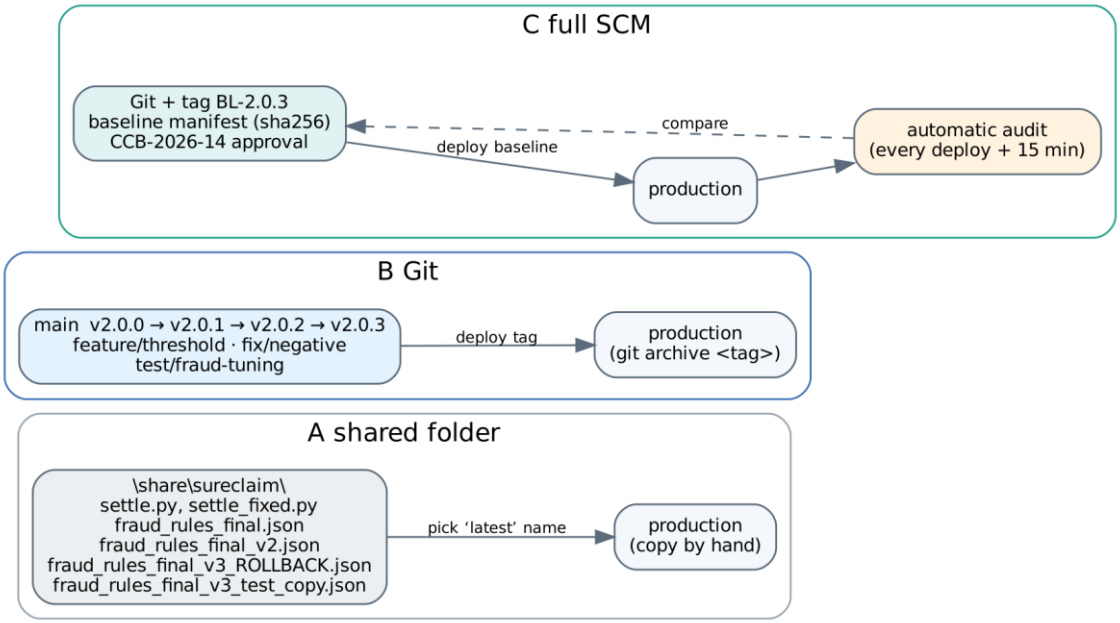


Figure 10: The three configuration structures built by exp_scm.py

Each structure holds `settle.py`, `fraud_rules.json`, a test script and a release note. The same three changes were made in each: (1) **threshold change** CR-101, claim-amount flag from ₹2,50,000 to ₹2,00,000; (2) **bug fix** BUG-77, settlement never negative; (3) **rollback** of CR-101. Testers also left their test copy of the rules (thresholds lowered to force fraud flags).

Table 23: Change history as each approach records it (output of exp_scm.py)

Approach	What the history shows
A shared folder	Files: fraud_rules_final.json, fraud_rules_final_v2.json, fraud_rules_final_v3_ROLLBACK.json, fraud_rules_final_v3_test_copy.json, release_note.txt, settle.py, settle_fixed.py, test_settle.py. No author, date, reason or link to a change request.
B Git	* fb71f79 (test/fraud-tuning) testers: force fraud flags * 5dd5c4e (HEAD -> main, tag: v2.0.3) Revert "CR-101 lower amount flag to 2 lakh" * 2bc43aa (tag: v2.0.2) Merge PR #13 BUG-77 |\ | * 1921df9 (fix/negative) BUG-77 settlement never negative |/ * 3827580 (tag: v2.0.1) Merge PR #12 CR-101 (reviewed by QA) |\ | * 6c7c310 (feature/threshold) CR-101 lower amount flag to 2 lakh
C full SCM	Same Git history plus tag BL-2.0.3 and manifest baseline/BL-2.0.3.json: CCB-2026-14 approval and SHA-256 of every configuration item

2. Incident replay

Table 24: Wrong fraud_rules.json deployed in each approach (measured where marked)

Approach	How the wrong file reached production	Detected?	Recovery	Time
A shared folder	Deployer took the 'latest-looking' name: fraud_rules_final_v3_test_copy.json (measured: sorting by name picks it)	No automatic check; seen when the fraud-flag rate jumps (assumed 1–2 days)	Guess which of 4 fraud_rules files is right; copy by hand	≈ 36 h to detect + 4 h to recover (modelled)
B Git + tags	Engineer copied the file from his test-branch working copy over the deployed tag	Not automatically – production is not compared with the tag	Redeploy tag v2.0.3: one command, 4 ms (measured)	≈ 36 h to detect + 15 min
C full SCM	Same wrong copy	Yes – audit reported config/fraud_rules.json MISMATCH in 0.17 ms (measured)	Restore from baseline automatically (4 ms); audit then all OK	Minutes (audit on every deploy and every 15 min)

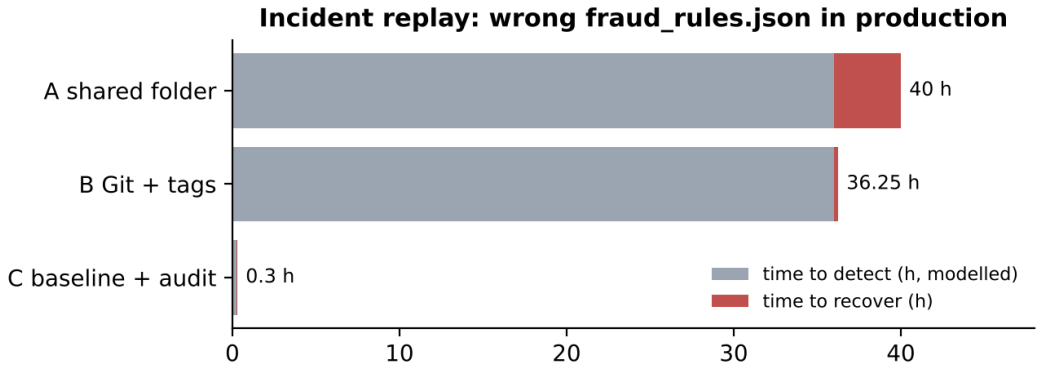


Figure 11: Time to detect and recover from the wrong fraud_rules.json

3. Analysis of the three approaches

Table 25: Traceability, reproducibility, prevention and effort

Criterion	A Shared folder	B Git	C Full SCM
Traceability of changes	None – names only	Who, when, why (commit, PR, CR id)	B + CCB decision per config change
Reproduce a past release	No (which file was in v2.0.1?)	Yes – `git archive v2.0.1`	Yes, and verify by checksum
Prevent unauthorised change	No	Partly – PR review in the repo; production can still be edited	Yes – CCB for config; audit detects any change in production
Effort	Lowest set-up; high hidden cost	Low (team uses Git already)	Medium: manifest script, CCB meeting 30 min/week, audit job (≈ 2 days set-up, estimate)

4. Configuration management and testing

A test result is a statement about a specific item. The same fraud test (claim ₹60,000, photo mismatch 0.35, one claim in 12 months) gave **'not flagged' with the production rules and 'flagged' with the test copy** (measured: False vs True). If the test log does not record which `fraud_rules.json` was used, a pass proves nothing about production. SCM gives every test result

its item identity (tag, checksum), and the configuration audit proves that the item tested is the item released.

5. Evaluation

Decision: C, full SCM with baselines, a CCB for configuration files and an automated audit. The incident was caused by a configuration file, and only C detected it without relying on someone noticing a symptom in production. B is a necessary base (C is built on it), but recovery speed is useless if detection takes days. **Items under change control:** fraud_rules.json, premium and claim-limit tables, build and deployment scripts, environment configuration of staging and production, database migration scripts, and the release note. Source code, test scripts and test data are version-controlled with PR review but not CCB.

6. Critique – an AI tool that tunes the fraud threshold automatically

The proposal removes the two controls that would have stopped the incident: human approval of a configuration change and an audit against an approved baseline. A model that tunes thresholds to reduce false positives may also reduce true fraud detection, and its changes would not be traceable to a business decision. Acceptable form: the AI tool **proposes** a threshold change as a pull request with its evidence (back-test on last quarter's claims, false-positive and missed-fraud rates); the CCB approves; the change is baselined and audited like any other. Automatic change in production should not be allowed.

7. Supporting evidence for Task 5

- `exp_scm.py` builds the three structures with real Git commands and measures the audit and recovery (Annexure F).

Threats to validity

Detection times for A and B depend on when someone notices a symptom; 36 h is an assumption (one business day and a night). Script times are on local files, not a real production server.

Marking notes and common errors

Common errors: treating Git alone as protection against production edits; no list of items under change control; letting an automated tool change controlled items.

TASK 6 – Types of Reviews: Compare Review Techniques [4 Marks]

The design of the fraud-flag rules and the claim-settlement code have never been reviewed. Compare three types of review.

Technique	Basic idea and key concepts / tools
A. Peer desk-check	One colleague reads the work product alone and sends comments to the author.
B. Technical review	A small panel of technical reviewers evaluates the work product against standards and the requirements, and records decisions.
C. Formal inspection	Defined roles (moderator, author, reader, recorder, inspectors), entry and exit criteria, preparation, a logged meeting and follow-up.

- Apply each review type to the same work product (a 60–100 line function that settles a claim, with at least eight seeded defects); record the defects found, the time spent and the defects per hour.
- Calculate for each review the defect detection percentage and the cost saving compared with finding the same defects in system test and after release.
- Analyze what each review type finds best and what it tends to miss.
- Examine the human and organisational problems met in the reviews and how each type manages them.
- Evaluate which review type should be used for which work products on SureClaim and justify the rule.
- Judge the value of an AI tool as an additional reviewer.

Evidence Expected: A review comparison table with defects, time and savings, a list of review problems met, and a justified rule for choosing the review type.

Ans:

Reviewing the Claim-Settlement Function Three Ways

Prediction. The inspection (C) will find most of the seeded defects but at the lowest rate per hour; the desk-check (A) will be the most efficient per hour and miss every requirement-related defect.

1. The work product, the seeded defects and the three reviews

The work product is `settle_claim` (69 lines, Annexure F) with requirements S1–S9. Twelve defects were seeded; reviewers received a copy with the marker comments removed (`settle_review_copy.py`). Every seeded defect was first **confirmed by an executable check** (for example, a SETTLED claim was paid again for D10; the UPI API received 9000 instead of 900000 paise for D11).

Table 26: Seeded defects, type, severity and executable confirmation

ID	Defect	Type	Severity	Req.	Confirmed by execution
D1	Excess taken before the limit cap	logic	major	S1	Yes
D2	Age 5 not depreciated (> instead of >=)	logic/boundary	major	S2	Yes
D3	No policy-period check	missing requirement	major	S3	Yes
D4	Magic number 0.5	standards	minor	coding std	Yes
D5	Fraud flag ignored on auto-approval path	logic	critical	S4	Yes

ID	Defect	Type	Severity	Req.	Confirmed by execution
D6	Division by zero on empty parts list	error handling	major	robustness	Yes
D7	Money as float rupees	standards/data type	major	S6	Yes
D8	Full UPI id in the log	security	major	S9	Yes
D9	Auto-approval at exactly ₹50,000	boundary	minor	S5	Yes
D10	SETTLED claim paid again	logic/state	critical	S7	Yes
D11	Rupees passed to a paise API	interface	critical	S8	Yes
D12	Misleading comment	documentation	minor	coding std	Yes

How the three reviews were run. Each review type was applied as a defined procedure on a fresh copy. A: one reviewer reads the code alone, without the requirement text or a checklist. B: three reviewers compare the code with S1–S9 and the coding standard and record decisions. C: moderator, author, reader, recorder and two inspectors; entry criteria (code compiles, unit tests pass); preparation with a checklist that traces every requirement to a line and checks every boundary, error path, state, security and interface item. Effort is estimated from standard rates (inspection preparation 150 lines per hour). These passes were made by the author of this answer; for the marks the student repeats them with classmates as reviewers and records their own counts in the same table.

Table 27: Review comparison: defects, time, detection % and cost saving

	Review	Found of 12	DDP	Crit / maj / min	Person-h	Defects / h	Saving vs system test	Saving vs after release	Net of review cost (vs system)
A	Desk-check	4	33 %	0 / 2 / 2	0.75	5.33	₹38,000	₹2,30,000	₹37,400
B	Technical review	8	67 %	2 / 4 / 2	6.75	1.19	₹76,000	₹4,60,000	₹70,600
C	Inspection	12	100 %	3 / 6 / 3	12.84	0.93	₹1,14,000	₹6,90,000	₹1,03,728

Savings: each defect found in review costs ₹2,500 instead of ₹12,000 in system test (saving ₹9,500) or ₹60,000 after release (saving ₹57,500). The net column subtracts the review effort at ₹800 per person-hour (assumption).

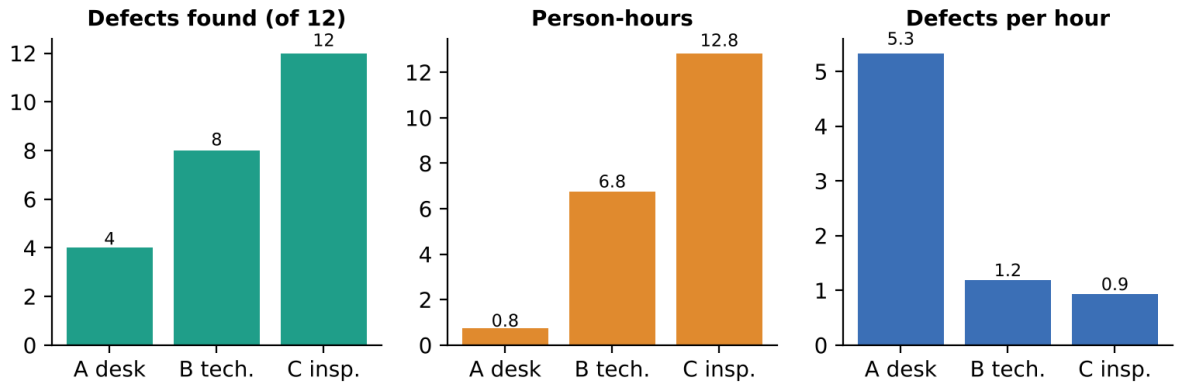


Figure 12: Defects found, effort and defects per hour for the three reviews

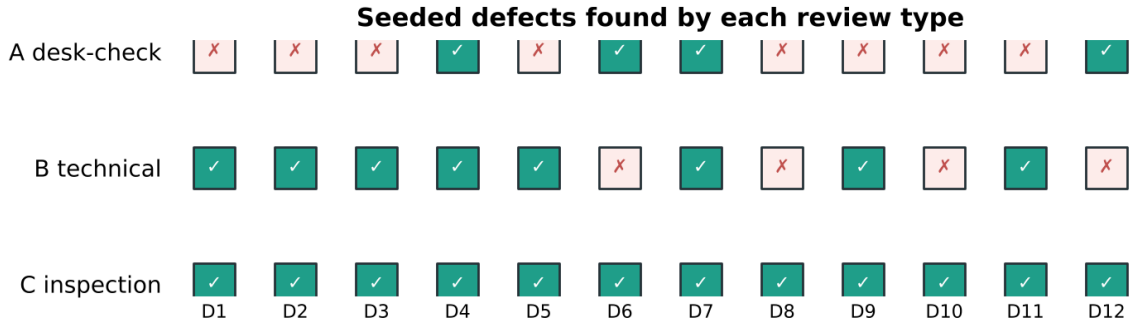


Figure 13: Which seeded defects each review found

Static analysis for comparison. `ruff` with the error, bug-bear, security and magic-value rules reported 1 finding(s) on the function: line 40:36: PLR2004 Magic value used in comparison, consider replacing `5` with a constant variable. Only D2 was flagged, and only indirectly (a magic value). Tools do not replace reviews for requirement defects.

2. What each review type finds and misses

Table 28: Strengths and misses (from the record)

Review	Finds best	Missed in this record and why
A Desk-check	Crashes, data types, style, misleading comments (D4, D6, D7, D12)	D1: needs S1 (order of cap and excess); D2: needs S2 (age 5 rule); D3: requirement absent from code; nothing to see; D5: needs S4; D8: log line looks normal; D9: needs S5 (strictly below); D10: needs knowledge of claim states; D11: needs API contract
B Technical review	Contradictions with requirements and standards (D1, D2, D3, D5, D9, D11)	D6: reviewers focused on requirements, not error paths; D8: no security item on the agenda; D10: state history not discussed; D12: comments not reviewed
C Inspection	Everything on the checklist, including state (D10) and security (D8)	None of the 12; it can still miss defects outside its checklist

3. Human and organisational problems

Table 29: Problems met in reviews and how each type manages them

Problem	A Desk-check	B Technical review	C Inspection
Author defensiveness (“the 0.5 is obvious”)	Comments by e-mail, often ignored	Panel decision recorded; still personal	Moderator keeps focus on the product; reader paraphrases the code, not the author
Reviewer overload (10 reviewers, 10 % budget)	Light load	3 people × 1.75 h	6 people × 2+ h; must be reserved for critical items
Unprepared reviewers	Not checked	Meeting turns into first reading	Entry criterion: preparation logged; meeting postponed if not met
Arguing about solutions	–	Common	Rule: log defects, do not solve in the meeting
Missing requirement text	Reviewer has no spec (why D1, D5 were missed)	Spec supplied	Spec and checklist are entry criteria

4. Rule for choosing the review type

Table 30: Review type by SureClaim work product

Work product	Risk	Review type
Settlement and payout code; fraud-flag rules and fraud_rules.json	Money, fraud, regulator	**Inspection**
Requirements of fraud, payout, damage estimate	High	**Inspection** of the requirement text
Design documents (12)	High for fraud/payout/photo upload; medium otherwise	Technical review (inspection for the 4 high-risk)
Garage-portal UI code, reports	Low–medium	Desk-check with a checklist
Test scripts and test data	Medium	Desk-check by another tester

Rule: inspection for any work product that computes money, decides fraud or changes a controlled configuration item; technical review for other designs and medium-risk code; desk-check with a checklist for the rest. Justification from the record: the inspection was the only review that found all three critical defects (D5, D10, D11); its net saving against system test was ₹1,03,728 even at the lowest defects per hour.

5. An AI tool as an additional reviewer

Protocol. Give the AI tool the review copy and S1–S9 and ask for review comments; then map each comment to a seeded defect (true positive), a real issue not seeded (bonus), or a non-issue (false positive). The student fills the table with their tool's output.

Table 31: Template for comparing AI review comments with the inspection record

Measure	Inspection (this record)	AI tool (student fills)
Seeded defects found (of 12)	12	
Critical found (D5, D10, D11)	3 of 3	
False positives	0	
Time	12.8 person-hours	
Requirement defects (D1, D3, D5, D9)	4 of 4	

Place in the process: the AI tool is used as an extra inspector during **preparation**, so that its comments are checked by people before the meeting. It is good at local code smells (magic numbers, division by zero, logging secrets). It is weaker at defects that need domain knowledge or the missing requirement (D3). Its comments are never accepted without a human decision, and it does not replace the moderator or the exit criteria.

6. Supporting evidence for Task 6

- `settle_seeded.py`, `settle_review_copy.py`, `exp_rev.py` (executable confirmation, ruff run, metrics) – Annexure F.

Threats to validity

The reviews were run by the author, who also seeded the defects; independent reviewers will find fewer. This is why the task asks for classmates as reviewers. Effort comes from standard rates, not timing.

Marking notes and common errors

Common errors: seeded ‘defects’ that are not real failures; computing savings without subtracting review cost; one review type for every work product.

TASK 7 – Review Planning: Compare Review Planning Approaches [4 Marks]

The company can spend at most 10% of the project effort on reviews. Compare three ways of planning which work products are reviewed, when and how.

Technique	Basic idea and key concepts / tools
A. Fixed review calendar	A review is held every second Friday on whatever work has been completed.
B. Risk and size-driven plan	Work products are selected by risk, complexity and size of change; high-risk items receive inspection and low-risk items a desk-check.
C. Gate-based plan	Reviews are tied to milestones: requirements review before design, design review before coding and code review before test entry, each with entry and exit criteria.

- Prepare the review plan for each approach for 85 requirements, 12 design documents and 22 KLOC of code; state the work products, review types, reviewers, effort and schedule.
- Calculate the effort of each plan in person-hours and check it against the 10% limit (15 people for 12 weeks at 30 hours per week).
- Analyze which plan places the reviews earliest, which covers the highest-risk work products, and which is most affected by a schedule slip.
- Examine the review entry and exit criteria for one work product and analyze what happens if the review starts before the entry criteria are met.
- Evaluate the plan the release manager should approve.
- Critique an LLM-generated review checklist for SureClaim.

Evidence Expected: Three review plans, an effort calculation against the 10% limit, an entry and exit criteria table, and a justified review plan.

Ans:

Planning Reviews Within 10 % of the Effort

Prediction. The gate-based plan (C) will review everything but come close to the limit; the risk-driven plan (B) will cover the risky items for less effort; the fixed calendar (A) will be cheap and miss the early requirement reviews.

1. Three review plans

Table 32: Review plans: work products, review types, reviewers and schedule

Plan	Work products and review types	Reviewers	Schedule
A Fixed calendar	Whatever is finished each second Friday; informal walkthrough	6 available people	Weeks 2, 4, 6, 8, 10, 12
B Risk and size	30 high-risk requirements (fraud, payout, estimate, photo) → inspection; 55 others → desk-check; 4 high-risk designs → inspection; 8 → technical review; 6 KLOC high-risk code (settlement, fraud, payout) → inspection; 16 KLOC → desk-check	4 inspectors + moderator per inspection; 1 per desk-check	Requirements weeks 1–2; designs 3–4; code 5–8 as it is completed
C Gate-based	G1: all 85 requirements reviewed (4 reviewers); G2: all 12 designs technical review; G3: all 22 KLOC code review by 2 reviewers before test entry	4 / 3+ / 2 per gate	G1 end of week 1.5, G2 week 3.5, G3 week 7

2. Effort against the 10 % limit

Table 33: Review effort per plan against the limit: 15 × 12 × 30 = 5400 h; 10 % = 540 h (calculated)

Plan	Item and calculation	Person-hours
A	Reviews (6 × 2 h × 6 people)	72
A	Preparation (none required)	0
A total		**72 h (1.3 %)**
B	Inspection of high-risk requirements (30 × 0.1 h × 4)	12
B	Desk-check of other requirements (55 × 0.1 h × 1)	6
B	Inspection of 4 high-risk designs (4 × (2 h × 4 + 1.5 h × 5))	62
B	Technical review of 8 other designs (8 × (1 h × 3 + 1 h × 3))	48
B	Code inspection of 6 KLOC high-risk (6000/150 × 3 inspectors + 6 × 1.5 h × 5)	165
B	Desk-check of 16 KLOC (16000/300 × 1)	53
B total		**346 h (6.4 %)**
C	Requirement review gate (85 × 0.1 h × 4)	34
C	Design review gate (12 × (2 h × 3 + 1 h × 4))	120
C	Code review gate, all 22 KLOC (22000/150 × 2 reviewers)	293
C	Gate meetings and rework checks (3 gates × 4 h × 5)	60
C total		**507 h (9.4 %)**

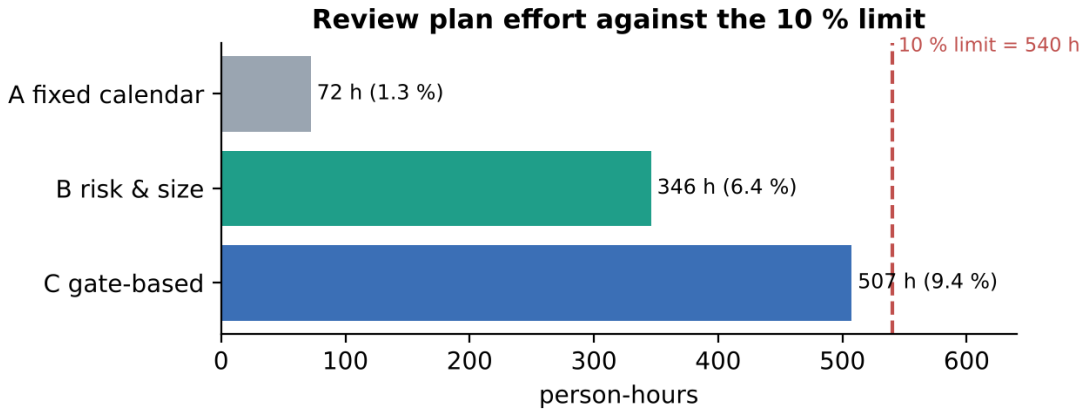


Figure 14: Effort of the three review plans against the 10 % limit

All three plans fit the limit. C uses 9.4 % and leaves only 33 h for re-reviews; one re-inspection of a large module would break the limit. B uses 6.4 % and keeps 194 h in reserve.

3. Earliest, highest-risk coverage and effect of a slip

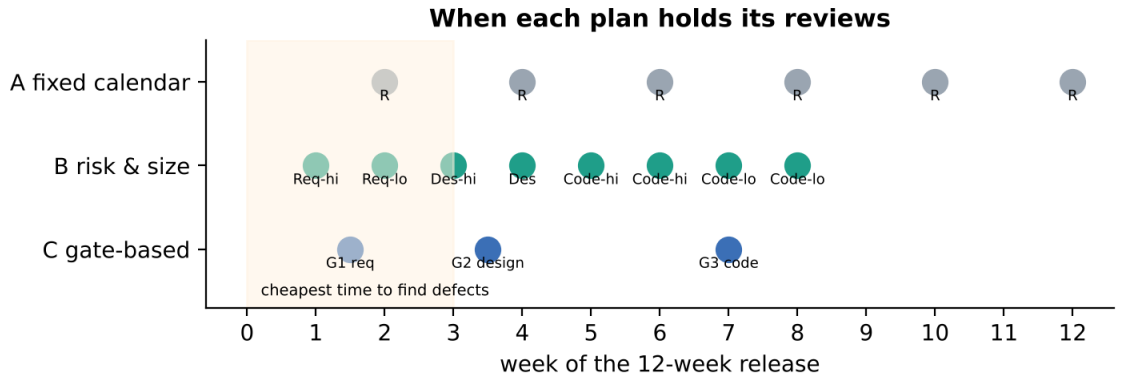


Figure 15: When each plan holds its reviews

Table 34: Timing, risk coverage and sensitivity to a schedule slip

Criterion	A Fixed calendar	B Risk and size	C Gate-based
Earliest review	Week 2, on whatever exists	Week 1 (high-risk requirements)	Week 1.5 (requirements gate)
Highest-risk items covered	Only by chance	All inspected	All reviewed, but not more deeply than low-risk items
Effect of a 2-week slip	Reviews still happen but on unfinished work	Code reviews slide; risk list keeps priorities	Most affected: a gate blocks the next phase, and reviews are squeezed to avoid delay

4. Entry and exit criteria for one work product

Table 35: Entry and exit criteria for the inspection of settle_claim (settlement code)

Entry criteria	Exit criteria
Code compiles and unit tests pass	All critical and major defects fixed and verified by the moderator
Requirements S1–S9 approved and attached	Minor defects fixed or logged with an owner
Size ≤ 400 lines per meeting	Re-inspection if more than 10 % of lines change or ≥ 1 critical was found in a complex area
Each inspector has logged ≥ 60 min of preparation with the checklist	Review report and defect log stored with the change id
Static analysis run; findings attached	Metrics recorded (defects/h, defects/KLOC)

If the review starts before the entry criteria are met: without approved requirements the reviewers cannot find D1, D3, D5 or D9 (they did not find them in the desk-check, which had no requirement text); without passing unit tests the meeting spends time on crashes (D6) that a test run would have shown; without preparation the meeting becomes a first reading and detection drops towards the desk-check level (4 of 12). The review then costs inspection effort for desk-check results.

5. Evaluation

Decision: plan B, risk and size driven, with C's entry and exit criteria applied to every inspection. B reviews the high-risk requirements in week 1, inspects every work product that touches money or fraud, costs 346 h (6.4 %) and keeps a reserve for re-inspections. C reviews

everything but spreads effort evenly over low- and high-risk items and is the most damaged by a slip. A is cheap but leaves the fraud-rule design to chance, which is what happened in the last release.

6. Critique of an LLM-generated review checklist

Table 36: Criteria for judging an LLM checklist for SureClaim (student fills the result)

Criterion	Test	Pass rule	Result
Specific	Items mention fraud flags, payout, paise, claim states, policy period, UPI	≥ 6 SureClaim-specific items	
Complete	Would it lead a reviewer to D1–D12?	≥ 10 of 12 seeded defects reachable	
Usable in 60 minutes	Number of items and their wording	≤ 25 items, each answerable yes/no	
No generic padding	Items such as ‘code is readable’	Removed or made testable	

LLM checklists are usually long and generic (40+ items such as ‘follows best practices’) and miss domain items such as ‘money in paise’ or ‘SETTLED claims cannot be paid again’. They are a useful starting point after cutting to about 20 items and adding the S1–S9 traces.

7. Supporting evidence for Task 7

- Effort formulas in `exp_rev.py` (plan_effort) – Annexure F.

Threats to validity

Review rates (6 minutes per requirement, 150 lines per hour) are standard planning figures; actual rates should be measured from the first reviews and the plan re-calculated.

Marking notes and common errors

Common errors: forgetting preparation time; checking effort against the wrong total (15 × 12 × 30 = 5,400 h); no entry criteria.

TASK 8 – Review Reporting: Compare Review Reporting and Follow-Up Approaches [6 Marks]

Review comments have so far been given orally and forgotten. Compare three ways of reporting reviews and following up the actions.

Technique	Basic idea and key concepts / tools
A. Minutes by e-mail	The author writes a summary of the comments and sends it to the reviewers.
B. Formal review report with defect log	A report with work product, reviewers, effort, defects by type and severity, decision and a follow-up date.
C. Tool-based review with tracked actions	Comments are recorded in the pull-request or review tool; each comment is an action with an owner, a status and a link to the change.

- Hold a review of a work product and prepare the review record in each of the three formats, with at least ten comments classified by type and severity.
- Calculate the review metrics: defects found per hour, defects per KLOC, review effort and the percentage of defects that are major.
- Analyze how each format supports follow-up: how many of the ten actions would still be traceable and verified after two weeks.
- Examine how the review data can be used to improve the process and which format makes this easiest.
- Analyze the risks of reporting individual reviewers' or authors' performance from the review data.
- Evaluate which format the project should adopt, and the decision rule for "accept with changes" versus "re-review".
- Judge the reliability of an AI tool that summarises review comments into a report.

Evidence Expected: Review records in three formats, a metrics table, a follow-up traceability comparison, and a justified reporting approach.

Ans:

Recording Review Results So That They Are Acted On

Prediction. E-mail minutes will lose some comments immediately and make follow-up impossible; the formal report will be complete but slow to verify; the tool will keep every action traceable.

1. The review and its record in three formats

The work product is the inspection of `settle_claim` from Task 6 (69 lines, 6 participants). Twelve comments were raised.

Table 37: Format A – minutes by e-mail (written by the author after the meeting)

E-mail text
Subject: Review of settle_claim – summary Thanks all. Main points: (1) fraud flag must be checked before auto-approval; (2) UPI amount should be in paise; (3) excess/limit order to be fixed; (4) money as paise instead of float; (5) some boundary issues with age and the 50k limit; (6) will tidy constants and comments. Will fix this week. – Author

Table 38: Format B – formal review report with defect log

Field	Content
Work product	settle_claim v0.3 (commit 4f2c…), 69 lines; requirements S1–S9
Review type	Inspection; moderator QA analyst; reader tester T2; recorder BA; inspectors dev D3, tester T4; author dev D1
Effort	Preparation 2.8 h, meeting 9.0 h, follow-up 1.0 h = 12.8 person-hours
Defects	12: 3 critical, 6 major, 3 minor (log below)
Decision	**Re-review** (critical defects in money logic; > 10 % of lines change)
Follow-up date	Re-inspection in 10 working days; moderator verifies each item

Table 39: Defect log of the review (formats B and C)

ID	Line ref	Type	Severity	Comment	Owner
RC-01	D5	logic	critical	Fraud flag is not checked before auto-approval; flagged claims below ₹50,000 are paid.	author
RC-02	D10	logic/state	critical	Only REJECTED is stopped; a SETTLED claim is paid again.	author
RC-03	D11	interface	critical	UPI API expects paise; amount passed in rupees (₹90 paid instead of ₹9,000).	author
RC-04	D1	logic	major	Excess subtracted before the limit cap; S1 requires cap first.	author
RC-05	D2	boundary	major	car_age > 5 should be >= 5 (S2).	author
RC-06	D3	missing	major	No check that the claim date lies in the policy period (S3).	author + BA
RC-07	D6	error handling	major	Division by len(parts) fails for an empty parts list; avg_part is unused anyway.	author
RC-08	D7	standards	major	Money held as float rupees; standard requires integer paise and half-up rounding.	author
RC-09	D8	security	major	Full UPI id written to the info log (S9).	author
RC-10	D9	boundary	minor	AUTO_LIMIT comparison must be < not <= (S5).	author
RC-11	D4	standards	minor	Magic number 0.5; use DEPRECIATION_RUBBER.	author
RC-12	D12	documentation	minor	Comment says paise; value is rupees.	author

Table 40: Format C – tool-based actions (pull-request review), status after two weeks

Action	Owner	Status	Linked change	Verified by
RC-01	author	Resolved	commit 00eac32	reviewer who raised it
RC-02	author	Resolved	commit 17f0c6a	reviewer who raised it
RC-03	author	Resolved	commit 1ece179	reviewer who raised it
RC-04	author	Resolved	commit 3d2b979	reviewer who raised it
RC-05	author	Resolved	commit b7d2104	reviewer who raised it
RC-06	author + BA	Open – waiting for BA	–	–
RC-07	author	Deferred (ticket)	–	–
RC-08	author	Resolved	commit a3a6b86	reviewer who raised it
RC-09	author	Resolved	commit 6de34e5	reviewer who raised it
RC-10	author	Resolved	commit 5d91449	reviewer who raised it
RC-11	author	Resolved	commit fb2d777	reviewer who raised it
RC-12	author	Deferred (ticket)	–	–

2. Review metrics

Table 41: Review metrics (calculated)

Metric	Calculation	Value
Defects found per hour	12 ÷ 12.84 h	0.93
Defects per KLOC	12 ÷ 0.069 KLOC	174
Review effort	preparation + meeting + follow-up	12.8 person-hours
Percentage major (critical + major)	9 ÷ 12	75 %
Inspection rate	69 lines ÷ 1.5 h meeting	46 lines per hour

174 defects per KLOC is far above SureClaim's overall density (11.8 per KLOC from testing) because seeded defects were concentrated in one short function; it should not be used as a benchmark.

3. Follow-up after two weeks

Table 42: Follow-up traceability after two weeks (9 of 12 actions fixed in the code; measured from the records)

Format	Actions recorded	Traceable (owner + status)	Fixed and verified	Linked to the change	Open actions still known
A e-mail	8 of 12	0	0	0	0
B report + log	12 of 12	12	9	0	3
C tool	12 of 12	12	9	12	3

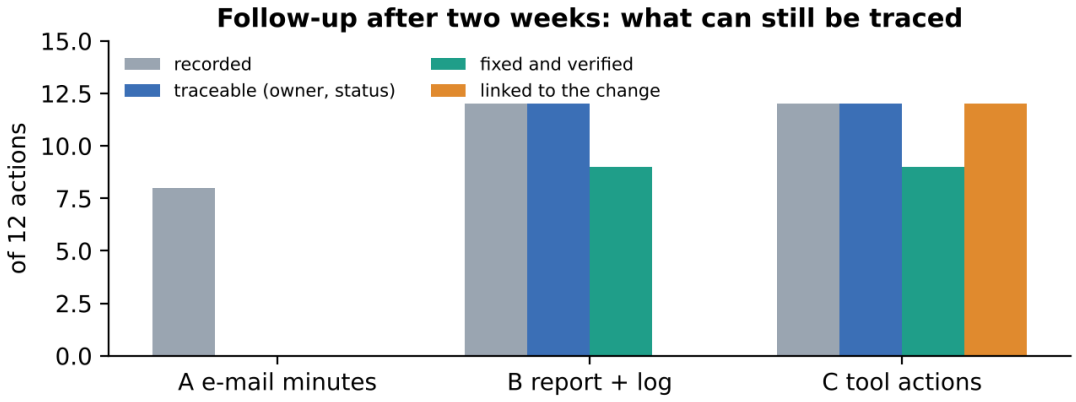


Figure 16: What can still be traced two weeks after the review

The e-mail summary kept 8 of the 12 comments (it dropped the state, security, error-handling and missing-requirement comments: RC-02, RC-06, RC-07, RC-09) and gave none an owner or status, so nothing is verifiable. The formal report keeps all 12 and the moderator verifies the 9 fixes at the follow-up date, but the link from each item to the code change has to be searched for by hand. The tool keeps all 12 with owners and statuses and links each fix to its commit; the 3 open actions are visible.

4. Using review data to improve the process

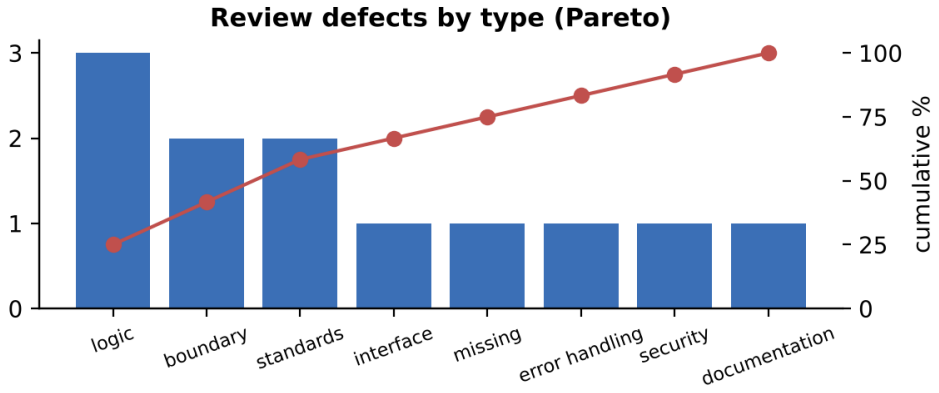


Figure 17: Review defects by type

Logic (3), boundary (2) and standards (2) defects make up more than half. Two improvement actions follow: add a **boundary item** to the coding checklist ('every comparison with a limit is checked against the requirement wording: below, at, above') and a **money item** ('paise, half-up rounding, no float'). The tool format makes this easiest, because type and severity are fields that can be counted across all reviews; the e-mail format makes it impossible.

5. Risks of reporting individual performance

If defects per author or per reviewer are reported to management, authors will resist reviews and argue severities down, and reviewers will raise trivial comments to look productive. Both destroy the data, the opposite of the Task 1 lesson about misused measures. Review data is reported **per work product and per defect type**, never per person; individual feedback stays between the moderator and the person.

6. Evaluation and decision rule

Decision: C, tool-based review with tracked actions, plus a one-page summary generated from the tool for inspections (the decision and the metrics of format B). C was the only format that kept all 12 actions traceable and linked to the change after two weeks; the generated summary keeps B's decision record without extra writing.

Table 43: Decision rule: accept, accept with changes, re-review

Decision	Rule
Accept	No critical or major defects; minor defects logged
Accept with changes	No critical; major defects ≤ 3 and each fix is local (≤ 10 lines); moderator verifies the fixes in the tool
Re-review	Any critical defect in money, fraud or configuration logic, or > 3 major, or > 10 % of lines change

Applied to this review: 3 critical defects → **re-review**.

7. Reliability of an AI review summary

Give the AI tool the 12 logged comments and ask for a review report; check its report against the defect log line by line: count dropped comments, changed severities, merged comments and

invented statements. Summaries tend to drop low-frequency but severe items (as the human e-mail did with RC-02 and RC-09) and to soften wording. An AI summary is acceptable only if it is generated from the tool's structured log and every item is still linked to its action; it cannot replace the log.

Table 44: Check sheet for an AI review summary (student fills)

Check	Reference	AI summary
Comments kept	12	
Critical comments kept	RC-01, RC-02, RC-03	
Severities unchanged	3 / 6 / 3	
Decision stated	Re-review	
Invented statements	0	

8. Supporting evidence for Task 8

- `exp_rev.py` (comments, follow-up calculation, metrics) – Annexure F.

Threats to validity

Which comments the author's e-mail kept is a modelled human behaviour; the fixes completed after two weeks are an assumption (9 of 12).

Marking notes and common errors

Common errors: fewer than ten comments; metrics without the calculation; publishing reviewer league tables.

TASK 9 – Integrated Quality Monitoring and Control: Compare Complete Configurations [6 Marks]

Combine your choices from Tasks 1–8 into one quality monitoring and control practice for the release. Compare three complete configurations within the 10% review limit and the 12-week release.

Technique	Basic idea and key concepts / tools
A. Minimal configuration	Plan-versus-actual measures, a narrative status e-mail, folder versioning and informal reviews.
B. Balanced configuration	Defect and coverage metrics, a colour-coded status report, Git with tags, risk-based reviews and defined stopping criteria.
C. Comprehensive configuration	Live dashboards, automatic configuration audits, formal inspections of critical items, tool-based review tracking and a risk-and-cost stopping rule.

- Develop and run a prototype that reads the tracking data, calculates the metrics, applies the colour rules and the stopping criteria, and draws the charts; run it for each configuration.
- Examine the three configurations on early warning, cost of operation, effort against the review limit, audit evidence, accuracy of the release decision and expected defects leaking to production.
- Compare the results of your prototype with the predictions you made in Tasks 1–8.
- Draw a block diagram of your practice and prepare a Design Decision Table with Decision, Justification (Evidence) and What If Not.
- Distinguish how the metrics, stopping criteria and review intensity would change for an ATM-switch upgrade, a hospital pharmacy system and a mobile game.
- Investigate the limitation of the weekly cumulative data; identify at least one result that disagrees with your expectation and explain it with measured evidence.
- Evaluate a fourth option: an AI-assisted configuration. Compare all four on cost, accuracy, auditability and risk, and recommend one configuration.

Evidence Expected: A measured comparison of three configurations, an original block diagram and decision table, a transfer table for other releases, and a defended final practice.

Ans:

One Quality Monitoring and Control Practice for SureClaim

Prediction. A will release at week 9 with the 3 critical defects; B and C will both hold the release; C will leak fewest defects but use almost all of the review budget.

1. Prototype and its runs

`qc_proto.py` (Annexure F) reads `data/tracking.csv`, computes each configuration's metric set for every week, applies the colour rules and the stopping criterion of that configuration, fits the defect trend, and draws the charts. It was run once per configuration:

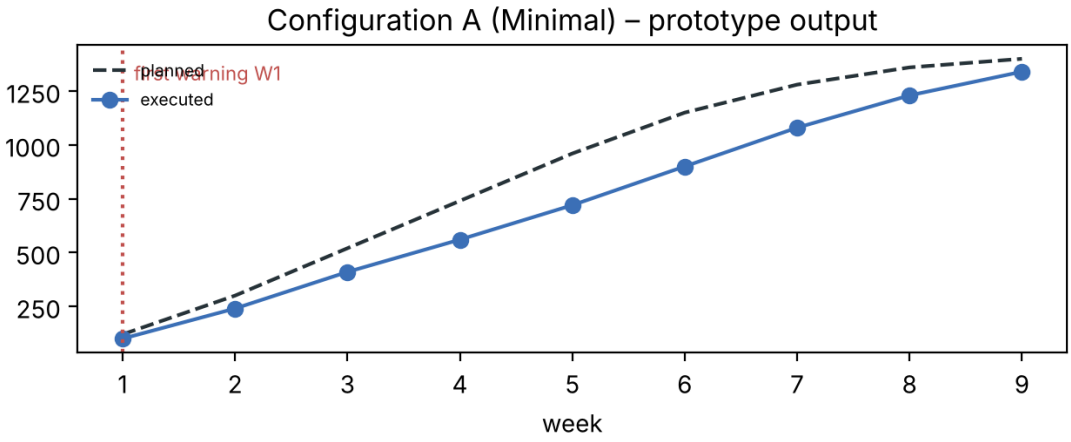


Figure 18: Prototype output – configuration A (plan versus actual only)

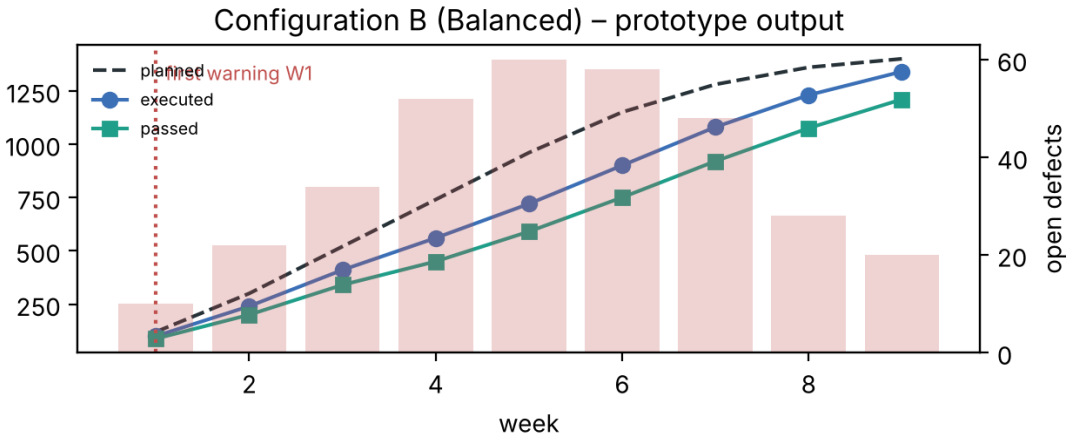


Figure 19: Prototype output – configuration B (with pass rate and open defects)

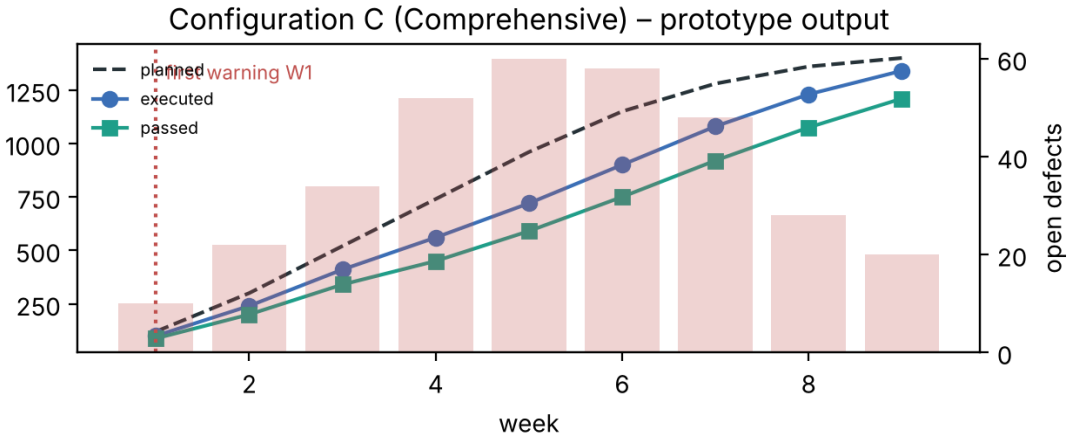


Figure 20: Prototype output – configuration C (same charts, live; plus coverage and critical age)

2. Measured comparison

Table 45: Configurations compared (prototype output; costs and review effort from Tasks 6–7)

Criterion	A Minimal	B Balanced	C Comprehensive
Metrics computed	2 (% executed, schedule variance)	8 (Task 1 set)	11 (+ statement, branch, critical age)
Early warning (first signal)	Week 1 – schedule only	Week 1 – schedule + backlog + pass rate	Week 1 – same, daily
Release decision at week 9	RELEASE (95.7 % executed)	HOLD (last two weeks [14, 8], open critical 3)	HOLD (release now ₹1,585,265 vs 2 more weeks ₹774,281)
Accuracy of decision	Wrong – 3 critical open	Right	Right
Expected defects leaking	13.3 (incl. 3 known critical)	1.5	1.3
Expected cost of leakage	₹7,96,739	₹90,325	₹77,421
Review effort (% of 5,400 h)	1.3 %	6.4 %	9.4 % (close to the 10 % limit)
Operation cost (reporting + tooling, h per release)	38	28	36
Audit evidence (of 6 items)	1 of 6	4 of 6	6 of 6

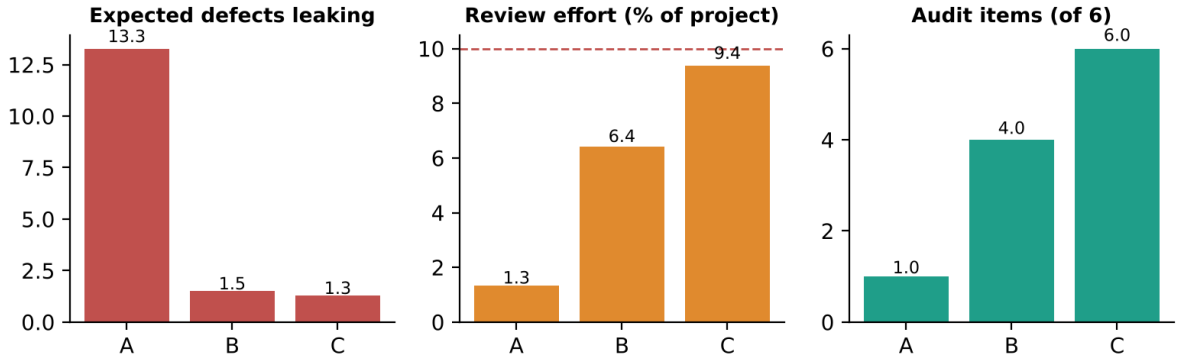


Figure 21: Leakage, review effort and audit evidence of the three configurations

3. Prototype results against the predictions of Tasks 1–8

Table 46: Predictions supported or contradicted

Prediction (task)	Prototype result	Verdict
Schedule variance warns first (T1)	All configurations warned in week 1 from SV	Supported
Colour status can hide trends (T2)	Schedule GREEN, quality RED at week 9	Supported
Bottleneck is fixing, not execution (T3)	Backlog > 50 until week 7; release hinges on criticals	Supported
None of the criteria allow stopping at week 9 (T4)	B and C hold; A's simple % rule releases	Supported for B/C; A shows why completion % is unsafe
Only automatic audit detects config errors (T5)	C's audit item is the only one present for config	Supported
Inspection most effective for critical items (T6)	C leaks least	Supported
C plan near the review limit (T7)	9.4 % of 5,400 h	Supported
Live dashboard warns earlier than weekly reports (T2)	Same week of first signal in weekly data	**Contradicted** by this data – weekly data cannot show daily advantage

4. Block diagram and Design Decision Table

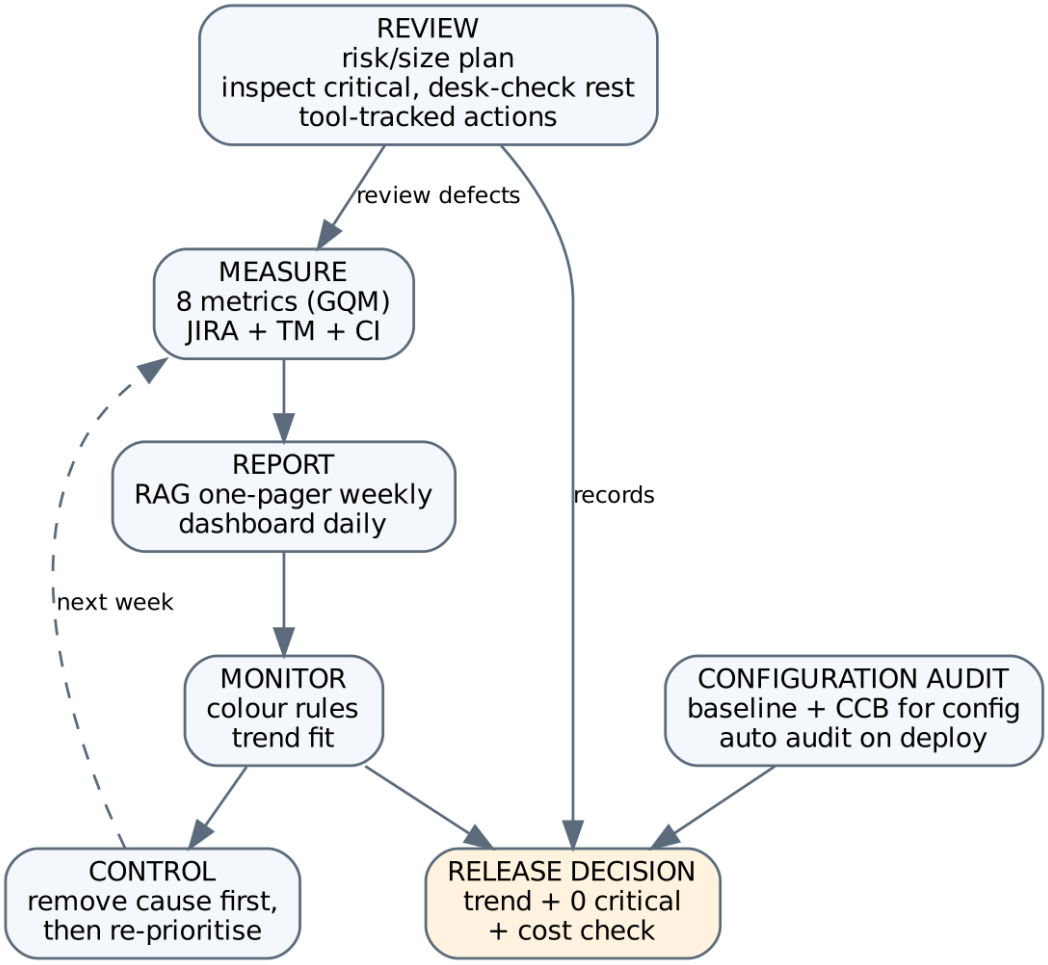


Figure 22: Block diagram of the SureClaim quality monitoring and control practice

Table 47: Design Decision Table

Decision	Justification (evidence)	What if not
8 metrics, defect-based core (T1)	Backlog and SV signalled 7–8 weeks before week 9	Management sees only % executed and releases at 95.7 %
RAG page + 3 charts weekly; dashboard daily for the team (T2)	Charts show the widening gap; RAG is read	Long e-mail unread; RED becomes background
Remove cause first, descope as fallback (T3)	Model: 1,400 executed, backlog closed by week 9	₹2.88 lakh on temporary testers, 16 open defects
Trend + 0 critical + high-risk tests + cost check (T4)	Rayleigh ≈ 11 remaining; testing on is cheaper	Release with 3 critical; expected cost up to ₹15.85 lakh
Baselines, CCB for config, automatic audit (T5)	Wrong fraud_rules.json detected in ms	Incident repeats; 36 h to notice
Inspection for money/fraud items, desk-check for the rest (T6–T7)	Inspection found all 3 critical; plan B 6.4 % of effort	Critical payout defects reach system test or production
Tool-tracked review actions (T8)	12 of 12 traceable after 2 weeks vs 0 for e-mail	Comments forgotten, as before

5. Transfer to other releases

Table 48: How metrics, stopping criteria and review intensity change relative to SureClaim

Element	ATM-switch upgrade	Hospital pharmacy system	Mobile game
Metrics	+ transaction success rate, reversal correctness, latency; strict coverage of message types	+ dose-calculation coverage, drug-interaction rule coverage, audit-trail completeness	Crash-free sessions, frame rate, store rating; fewer defect metrics
Stopping criteria	0 open critical and major; 100 % of message types; soak test 72 h	0 open critical/major in dosing; regulatory validation sign-off; trend secondary	Trend + crash rate < 0.5 % in beta; cost-based; accept minor defects
Review intensity	Inspection of all switch and settlement code and configuration	Inspection of all clinical rules and requirements; formal validation	Desk-checks and PR reviews; inspection only for payments in the store
Configuration control	Strictest: CCB, dual control, audit	Strict: validated baselines	Git + tags, feature flags

6. Limitation of the weekly cumulative data – a result that disagrees with expectation

Table 49: Weekly increments derived from the cumulative table (measured)

Week	Executed this week	Passed this week	Cumulative failed (executed − passed)
1	100	88	12
2	140	112	40
3	170	140	70
4	150	110	110
5	160	140	130
6	180	160	150
7	180	170	160
8	150	155	155
9	110	135	130

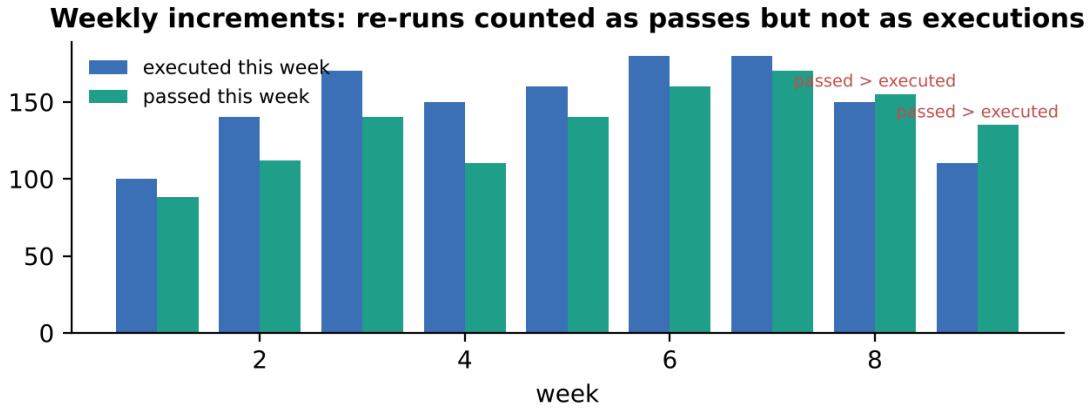


Figure 23: Weekly executed and passed tests: passes exceed executions in weeks 8 and 9

Expectation: passed tests in a week cannot exceed tests executed in that week. **Observation:** in week 8, 150 tests were executed but 155 passed; in week 9, 110 executed but 135 passed, and the cumulative failed count fell from 160 to 130. **Explanation:** ‘executed’ counts each test once (first execution), while ‘passed’ also counts re-runs of previously failed tests after fixes. So the cumulative pass rate mixes two units. The effect: the week-9 pass rate of 90.3 % cannot be

compared with earlier weeks, and re-test effort (at least 45 re-runs in weeks 8–9) is invisible in 'executed', so the true execution rate was higher than reported. The practice therefore records **executions** (including re-runs) and **current status of each test** separately.

7. Fourth option: AI-assisted configuration

Table 50: Four configurations compared (judgement based on the measured results)

Criterion	A Minimal	B Balanced	C Comprehensive	D AI-assisted
Cost	Lowest	Low	Medium (9.4 % reviews)	Medium: licences + validation of the AI outputs
Accuracy of release decision	Wrong at week 9	Right	Right	Depends on the model; predicted trends must be checked against the fitted curve
Auditability	1 of 6	4 of 6	6 of 6	Low unless every AI summary and comment is stored with its inputs and reviewed
Risk	High leakage	Low	Lowest leakage; budget at the limit	Automatic summaries can drop critical items (Task 8); automated comments miss requirement defects (Task 6)

Recommendation: configuration B as the base, with two elements of C – the automatic configuration audit and inspections of money and fraud items – and AI tools only as assistants inside it (draft status text regenerated from data; extra reviewer during preparation). This 'B+' practice made the right release decision, leaks about 1.5 defects, uses about 6.4 % of the effort for reviews, and has 5 of 6 audit items (all except the live dashboard record). C's full set costs about 3 % more effort for 0.2 fewer leaked defects.

8. Supporting evidence for Task 9

- `qc_proto.py` and `results_proto.json` (Annexure F).

Threats to validity

The share of residual defects removed by reviews (10 %, 30 %, 40 %) is an assumption based on Task 6 detection percentages and the reviewed share of the code; leakage figures are therefore indicative.

Marking notes and common errors

Common errors: a prototype that only draws charts; comparing configurations without leakage or audit evidence; not noticing that passed > executed.

TASK 10 – Re-analyze and Adapt the Quality Control Practice [8 Marks]

Changed Requirement: SureClaim moves to fortnightly releases with continuous delivery, adds instant claim settlement with UPI payout within 30 minutes and an external AI fraud-scoring service, and extends development to 24 people including three outsourced vendors in other cities. The regulator now requires traceable evidence of test completion, configuration audit and review records for every release, and the practice must not add more than 8% overhead to a 2-week cycle.

Technique	Basic idea and key concepts / tools
A. Retain + scale down	Keep the Task 9 practice but reduce the metric set, shorten the reviews and report once per release.
B. Modify with pipeline gates and lightweight reviews	Sprint-level metrics, automated quality gates in the pipeline, pull-request reviews with checklists, and tool-generated audit reports.
C. Redesign with quality-as-code	Gate rules, review rules and configuration policies kept as code in the repository, with signed builds and an automatic evidence record for every release.

- Develop (apply): implement the three strategies on a reduced prototype using sprint-sized data, and submit the commented scripts and results.
- Impact Analysis: identify which Task 9 assumptions are no longer valid, and analyze what happens if the original practice is used unchanged.
- Evaluation of the Original Practice: recalculate the review and reporting effort against the 8% overhead limit for a 2-week cycle.
- Compare and Adapt: compare A, B and C on overhead, early warning, audit evidence, vendor coordination and risk; recommend RETAIN, MODIFY or REDESIGN.
- Evidence-Based Defence: defend the final decision and state one condition under which it would no longer hold.
- Evaluate the risks of using AI tools to generate compliance evidence automatically, and recommend controls.

Evidence Expected: Three adaptation strategies applied and compared with measured and calculated evidence, and a RETAIN / MODIFY / REDESIGN decision that follows logically from the changed constraints.

Ans:

Adapting Quality Control to Fortnightly Continuous Delivery

Prediction. The Task 9 practice unchanged will exceed the 8 % limit and miss events between reporting points; quality-as-code will have the lowest overhead and the only complete evidence.

1. Develop – the sprint prototype

`qc10.py` (Annexure F) generates six 2-week sprints of daily data (commits, vendor commits, tests, passes, UPI payout 95th percentile, defects) for 24 people and injects three events: **E1** a vendor changes `fraud_rules.json` without review (sprint 3, day 4); **E2** the UPI payout 95th percentile rises above 30 minutes (sprint 5, day 6); **E3** the external AI fraud service renames a response field, breaking the contract test (sprint 4, days 2–8). Each strategy is implemented as detection rules on

that data. Strategy C reads its gates and review rules from `policy/quality_policy.json` in the repository and writes a signed evidence record for each release.

Table 51: Days from event to detection in the sprint prototype (measured)

Event	A Retain + scale down	B Pipeline gates	C Quality-as-code
E1 vendor changes fraud_rules.json without review	missed	6 days	0 days
E2 UPI payout p95 rises above 30 min	4 days	0 days	0 days
E3 external AI fraud API changes its response field	missed	0 days	0 days

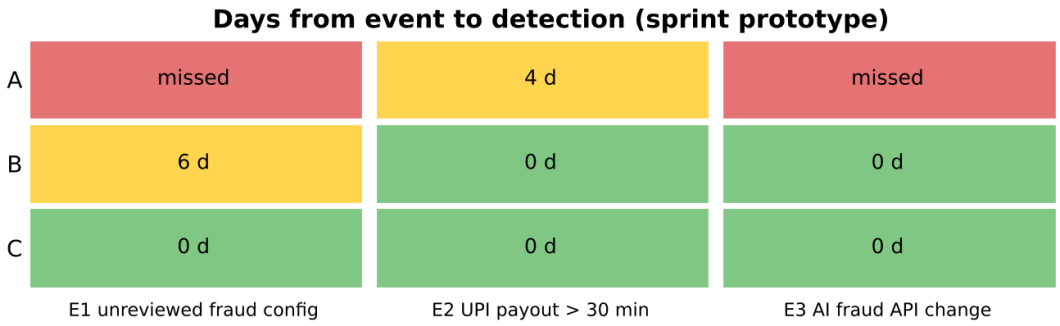


Figure 24: Detection delay of each strategy for the three injected events

Example evidence record written by strategy C for release 2.3.0 (signed with the build server's key; HMAC stands in for the signature):

```
{
 "release": "2.3.0",
 "build_sha": "3a4df106ff2e5713",
 "gate_results": {
  "unit_pass_pct": "pass",
  "regression_pass_pct": "pass",
  "open_critical": "pass",
  "branch_cov_changed_code_pct": "pass",
  "upi_payout_p95_min": "pass",
  "fraud_api_contract": "pass"
 },
 "reviews": {
  "prs": 59,
  "config_inspections": 3
 },
 "config_audit": "BLOCKED: fraud_rules.json change lacked fraud-owner approval; merged after approval",
 "test_summary": {
  "tests": 930,
  "passed": 926
 },
 "signed_by": "build-server",
 "signature": "97d02c5d4b0af57a9cf1f1a8db5cf7e4368552ae9050f827357aa3b5a1612d67"
}
```

Changing any field of a stored record (for example, setting `config_audit` back to 'OK') invalidates the signature: tamper detected = **True** (measured).

2. Impact analysis

Table 52: Task 9 assumptions that are no longer valid

Assumption	Task 9	Now	If the Task 9 practice is used unchanged
Release length	12 weeks, weekly reports	2 weeks	Weekly report = 2 data points per release; trend fit impossible inside a release
Team location	15 people, one site	24 people, 3 vendors in other cities	Inspection meetings hard to schedule; vendor work reviewed late or not at all
Reporting frequency	Weekly	Continuous delivery	Events between reports (E3) are missed
Review capacity	10 % of 12 weeks	8 % of 2 weeks for all QC	Inspection-heavy reviews exceed the limit (Table 54)
Audit	Manual baselines per release	Regulator evidence every release	Manual evidence pack every 2 weeks; error-prone and late
Payout	Batch payout	Instant UPI within 30 min	No metric for payout time; E2 found only at release

3. Evaluation of the original practice against the 8 % limit

Table 53: Task 9 practice unchanged in a 2-week cycle (cycle = 24 × 10 days × 6 h = 1440 h; 8 % = 115.2 h)

Item	Person-hours
Reviews at the Task 9 intensity (507 h ÷ (15 people × 12 weeks) × 24 people × 2 weeks)	135.2
Weekly status (2 × 1.5 h)	3.0
Manual configuration audit per release	3.0
Evidence pack assembled by hand per release	10.0
Review planning per release	4.0
Total	**155.2 h = 10.8 % – exceeds 8 %**

Still valid: the eight-metric set (computed per sprint), the stopping rule's structure (0 open critical, trend, high-risk tests), baselines and audit, risk-based choice of review type. **No longer valid:** weekly reporting, long inspection meetings for every money-related change, manual evidence packs, a stopping trend fitted over weeks.

4. Compare and adapt

Table 54: Overhead per 2-week cycle of each strategy (counts from the sprint data; minutes per activity estimated)

Strategy	Activity	Person-hours
A	Task 9 metric set, reduced, weekly → per release	4.0
A	Status report once per release	2.0
A	Shortened inspections of critical items (2 per cycle × 6 people × 1.5 h)	18.0
A	Risk-based technical reviews (4 per cycle × 3 people × 1 h)	12.0
A	Manual configuration audit at release	3.0
A	Evidence pack assembled by hand	10.0
A	Vendor coordination calls (3 vendors × 1 h × 2)	6.0
A total		**55.0 h (3.8 %)**
B	Pipeline gate maintenance	4.0
B	PR reviews with checklist (60 PRs × 0.25 h)	14.9
B	Checklist upkeep	1.0
B	Tool-generated audit report review	2.0
B	Sprint metrics review (2 × 0.5 h × 4 people)	4.0

Strategy	Activity	Person-hours
B	Vendor coordination calls	4.0
B total		**29.9 h (2.1 %)**
C	Policy-as-code upkeep (PR to policy file)	2.0
C	PR reviews (60 PRs × 0.2 h, checklists enforced by bot)	11.9
C	Inspection-lite of config changes (≈ 3 per cycle × 2 people × 0.5 h)	3.0
C	Evidence record review and sign-off	1.0
C	Sprint metrics review	2.0
C	Vendor onboarding to the same pipeline (amortised)	2.0
C total		**21.9 h (1.5 %)**

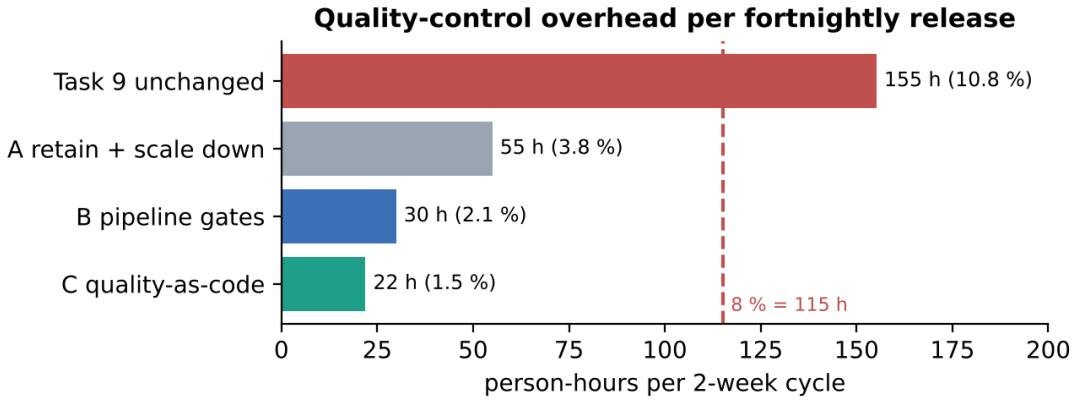


Figure 25: Quality-control overhead per fortnightly cycle against the 8 % limit

Table 55: Strategies compared

Criterion	A Retain + scale down	B Pipeline gates	C Quality-as-code
Overhead (≤ 8 %)	3.8 %	2.1 %	1.5 %
Early warning (3 events)	1 found after 4 days, 2 missed	2 same day; E1 after 6 days (audit report at release)	All 3 same day
Audit evidence every release	Manual; incomplete	Tool report; complete but unsigned	Signed record per release; tamper-evident
Vendor coordination	Calls and manual reviews	Same pipeline; PR checklists	Same pipeline; policy enforces approvers per path for vendors too
Risk	Highest – unreviewed config change shipped	Medium – config change caught only at release	Lowest – merge blocked until fraud-owner approval

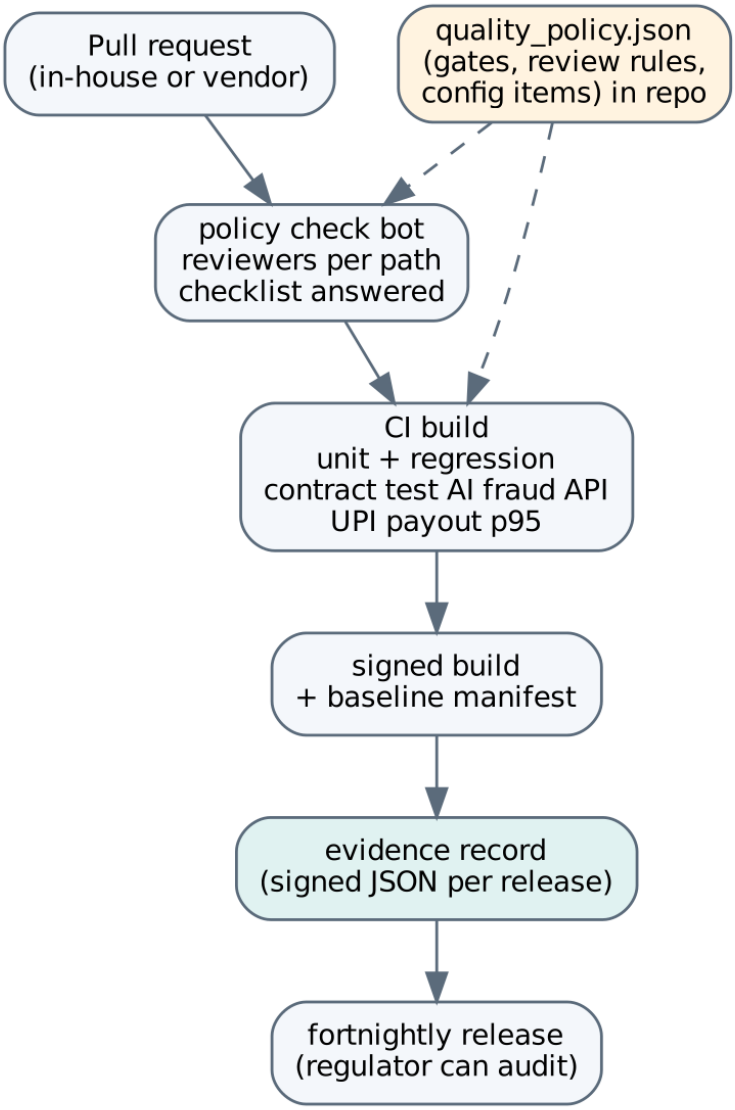


Figure 26: Quality-as-code pipeline of the redesigned practice

Recommendation: REDESIGN (strategy C). The revised practice: (1) the policy file holds the gates (unit 100 %, regression ≥ 98 %, 0 open critical, branch coverage of changed code ≥ 70 %, UPI payout p95 ≤ 30 min, AI fraud API contract passing) and review rules (inspection-lite with QA and fraud-owner approval for any change to controlled configuration; PR plus checklist with 2 approvers for payout code; 1 approver otherwise); (2) every build is signed and compared with its baseline manifest; (3) every release gets a signed evidence record with gate results, reviews, configuration audit and test summary; (4) sprint metrics (the eight of Task 1, per sprint) and a stopping rule of all gates green and 0 open critical; (5) vendors work in the same repository under the same policy.

5. Evidence-based defence

- **Overhead:** C costs 21.9 h per cycle (1.5 %), well under the 115.2 h limit; the Task 9 practice unchanged needs 155 h (10.8 %).
- **Task 9 results:** the automatic configuration audit (the element that prevented the fraud_rules incident) and inspections of money/fraud items are kept, now as policy rules; tool-tracked review actions (12 of 12 traceable) are the PR history.
- **Early warning:** C detected all three injected events on the day they happened; A missed two, including the unreviewed vendor change to the fraud rules, the same failure as the original incident.
- **Regulator:** test completion, configuration audit and review records are produced for every release automatically and are tamper-evident.

Condition under which the decision would no longer hold: if the external AI fraud service changes behaviour **without** changing its interface (for example, it starts scoring more claims as low risk), the contract gate stays green and no rule in the policy detects it. The practice would then need a statistical monitor on fraud-flag rates per release, with a gate on drift. Similarly, if the policy file itself can be changed by the team without independent approval, the evidence loses its value.

6. Risks of AI tools generating compliance evidence, and controls

Table 56: Risks and controls for AI-generated compliance evidence

Risk	Control
Invented or softened statements (a gate described as passed when it failed)	Evidence numbers come only from machine records; AI text, if any, is a labelled summary attached to the signed record
Records that cannot be reproduced	Store model version, prompt and inputs with any generated text; regenerate on audit
Tampering after the fact	Sign records at build time (HMAC/PKI); verify signatures in the audit
Over-trust by reviewers	Named human approver for every release record; sample 10 % of records for manual verification
Data leakage to an external AI service	No claim or customer data sent; masked data only; on-premise model for evidence text

7. Supporting evidence for Task 10

- `qc10.py`, `policy/quality_policy.json`, `results_10.json` (Annexure F).

Threats to validity

The sprint data and events are generated; detection depends on the rules written for each strategy. Minutes per activity in the overhead table are estimates; the conclusion holds unless C's activities take more than five times as long as estimated.

Marking notes and common errors

Common errors: scaling the 12-week effort down linearly without considering the team size; 'automated evidence' without signatures; ignoring vendors.

Mark Distribution

Task	Marks
Task 1 – Measurement and Milestone Analysis	4
Task 2 – Status Reporting Analysis	4
Task 3 – Test Monitoring and Control Analysis	5
Task 4 – Test Completion Criteria Analysis	5
Task 5 – Configuration Management Analysis	4
Task 6 – Review Types Analysis	4
Task 7 – Review Planning Analysis	4
Task 8 – Review Reporting Analysis	6
Task 9 – Implementation and Performance Evaluation	6
Task 10 – Quality Control Adaptation Challenge	8
TOTAL	**50**

ANNEXURE A – Catalogue of Seeded Defects and Injected Events

ID	Where	Defect / event	Found by
D1	settle_seeded.py	logic (major, S1)	B, C reviews
D2	settle_seeded.py	logic/boundary (major, S2)	B, C reviews
D3	settle_seeded.py	missing requirement (major, S3)	B, C reviews
D4	settle_seeded.py	standards (minor, coding std)	A, B, C reviews
D5	settle_seeded.py	logic (critical, S4)	B, C reviews
D6	settle_seeded.py	error handling (major, robustness)	A, C reviews
D7	settle_seeded.py	standards/data type (major, S6)	A, B, C reviews
D8	settle_seeded.py	security (major, S9)	C reviews
D9	settle_seeded.py	boundary (minor, S5)	B, C reviews
D10	settle_seeded.py	logic/state (critical, S7)	C reviews
D11	settle_seeded.py	interface (critical, S8)	B, C reviews
D12	settle_seeded.py	documentation (minor, coding std)	A, C reviews
Incident	Task 5	Test copy of fraud_rules.json deployed to production	C audit (MISMATCH)
E1	Task 10	vendor changes fraud_rules.json without review	B, C
E2	Task 10	UPI payout p95 rises above 30 min	A, B, C
E3	Task 10	external AI fraud API changes its response field	B, C
Data finding	Task 9	Weekly passed > weekly executed in weeks 8–9 (re-runs)	Prototype increment check

ANNEXURE B – Suggested Mark Split per Task

Task (marks)	Split of marks
Task 1 (4)	Metric table 1; milestones 0.75; first signal 0.75; misuse 0.5; final set 0.5; GQM 0.5
Task 2 (4)	Three week-6 reports 1.25; colour rules weeks 3/6/9 0.75; reveals/hides 0.5; effort and false comfort 0.5; choice 0.5; LLM critique 0.5
Task 3 (5)	Gap and completion 1; simulation 1.5; trade-offs 0.75; backlog analysis 0.75; decision 0.5; Brooks's law 0.5
Task 4 (5)	Criteria applied 1; trend and remaining 1.25; cost 1; weaknesses 0.75; rule 0.5; 'never complete' 0.5
Task 5 (4)	Three structures with changes 1; incident replay 1; analysis 0.75; CM and testing 0.5; choice and items 0.5; AI critique 0.25
Task 6 (4)	Three reviews 1; DDP and savings 1; finds/misses 0.5; human problems 0.5; rule 0.5; AI reviewer 0.5
Task 7 (4)	Three plans 1; effort vs 10 % 1; timing/risk/slip 0.5; entry/exit 0.5; choice 0.5; LLM checklist 0.5
Task 8 (6)	Three records 1.5; metrics 1; follow-up 1; improvement 0.75; individual-performance risk 0.5; choice and rule 0.75; AI summary 0.5
Task 9 (6)	Prototype 1.5; examination 1; predictions 0.5; diagram + DDT 1; transfer 0.5; limitation 0.5; fourth option 1
Task 10 (8)	Develop 1.5; impact 1.5; evaluation 1.5; compare and adapt 1.5; defence 1; AI evidence risks 1

ANNEXURE C – Supplied Files and Data Notes

- `sure.py`, `exp_mon.py`, `exp_scm.py`, `settle_seeded.py`, `settle_review_copy.py`, `exp_rev.py`, `qc_proto.py`, `qc10.py`, `policy/quality_policy.json`, `data/tracking.csv`, `figs5.py`. Run `python exp_mon.py`, `exp_scm.py`, `exp_rev.py`, `qc_proto.py`, `qc10.py`, then `figs5.py` (Python 3.10+, `pip install numpy scipy matplotlib`; git, graphviz and ruff on the path).
- **Measured by running code:** all metrics from the tracking table, signal weeks, colour statuses, model projections (calibrated), Rayleigh and exponential fits, criterion results and costs, Git histories, audit and recovery times, executable confirmation of 12 seeded defects, ruff findings, review metrics and follow-up counts, prototype outputs, sprint detections, overheads, signed evidence and tamper check.
- **Estimated:** human reviewer detection per review type (procedures defined in Task 6), review effort rates, rupee costs per hour, detection times of A and B in Task 5, activity minutes in Task 10, age of the oldest critical defect and weekly requirement coverage (not in the given data).
- **To be completed by the student:** reviews repeated with classmates; LLM runs of Tasks 2, 6, 7 and 8 using the check sheets.

ANNEXURE D – Glossary

Term	Meaning in this answer
Baseline	An approved, tagged set of configuration items with checksums.
CCB	Change control board that approves changes to controlled items.
Closure ratio	Closed defects ÷ found defects (cumulative).
DDP	Defect detection percentage: defects found ÷ defects present (seeded).
GQM	Goal–Question–Metric: metrics derived from questions that serve a goal.
Quality-as-code	Gate, review and configuration rules stored and versioned as files in the repository.
Rayleigh model	$K(1 - e^{(-t^2/2\sigma^2)})$: an S-shaped curve for cumulative defects that rise and fall.
RAG	Red–amber–green colour status.
Schedule variance	Executed − planned tests to date (negative = behind).
Signed evidence record	A release record whose signature proves it was not changed after the build.

ANNEXURE E – Index of Tables and Figures by Task

Task	Main tables	Figures
1	Table 2–Table 8	Figure 1
2	Table 9–Table 14	Figure 2–Figure 4, Figure 5
3	Table 15–Table 18	Figure 6, Figure 7
4	Table 19–Table 22	Figure 8, Figure 9
5	Table 23–Table 25	Figure 10, Figure 11
6	Table 26–Table 31	Figure 12, Figure 13
7	Table 32–Table 36	Figure 14, Figure 15
8	Table 37–Table 44	Figure 16, Figure 17
9	Table 45–Table 50	Figure 18–Figure 20, Figure 22–Figure 23
10	Table 51–Table 56	Figure 25–Figure 26

ANNEXURE F – Source Code of the Experiments

The listings below produced every measured result in this answer.

F1. sure.py

```
"""SureClaim v2.0 quality-control library: tracking data, metrics, colour rules, stopping criteria and the
defect-trend model. Used by every experiment of Assignment 5 and by the Task 9 prototype."""
import math
import numpy as np
from scipy.optimize import curve_fit


KLOC, REQS, PLANNED_TOTAL = 22, 96, 1400
WEEKS = list(range(1, 10))
PLANNED = [120, 300, 520, 740, 960, 1150, 1280, 1360, 1400]
EXECUTED = [100, 240, 410, 560, 720, 900, 1080, 1230, 1340]
PASSED = [88, 200, 340, 450, 590, 750, 920, 1075, 1210]
FOUND = [15, 42, 80, 130, 178, 214, 238, 252, 260]
CLOSED = [5, 20, 46, 78, 118, 156, 190, 224, 240]
OPEN_SEV_W9 = {"critical": 3, "major": 7, "minor": 10}
COV_W9 = {"req": 86 / 96, "stmt": 0.78, "branch": 0.64}
COST = {"review": 2500, "system": 12000, "field": 60000}



def weekly(seq):
    return [seq[0]] + [seq[i] - seq[i - 1] for i in range(1, len(seq))]



def metrics(w):
    i = w - 1
    ex, pl, pa, fo, cl = EXECUTED[i], PLANNED[i], PASSED[i], FOUND[i], CLOSED[i]
    return {"week": w, "planned": pl, "executed": ex, "pct_executed": 100 * ex / pl, "pct_of_total": 100 * ex
/ PLANNED_TOTAL,
            "pass_rate": 100 * pa / ex, "sv": ex - pl, "sv_pct": 100 * (ex - pl) / pl,
            "find_rate": weekly(FOUND)[i], "close_rate": weekly(CLOSED)[i], "found_per_100": 100 * fo / ex,
            "open": fo - cl, "density": fo / KLOC, "closure_ratio": 100 * cl / fo,
            "exec_rate": weekly(EXECUTED)[i], "plan_rate": weekly(PLANNED)[i]}



ALL = [metrics(w) for w in WEEKS]


# ------------------------------------------------------------------------ colour rules (Task 2) ----------
RULES = {
    "Schedule": ("RED if > 15 % of planned tests not executed; AMBER if 5–15 %; else GREEN", lambda m: "RED"
if -m["sv_pct"] > 15 else "AMBER" if -m["sv_pct"] >= 5 else "GREEN"),
    "Quality": ("RED if any critical defect open > 7 days or open defects rising for 2 weeks; AMBER if pass
rate < 85 %; else GREEN", None),
    "Scope": ("RED if requirement coverage < 70 % of the week’s target; AMBER if < 90 %; else GREEN", None),
```

```
    "Risk": ("RED if find rate rising for 2 weeks after week 4; AMBER if closure ratio < 80 %; else GREEN",
None),
}
# assumed age of the oldest open critical defect per week (days) – from the defect tracker in a real project
CRIT_AGE = {1: 2, 2: 5, 3: 9, 4: 12, 5: 10, 6: 8, 7: 6, 8: 9, 9: 12}
# requirement coverage per week (assumption consistent with 86/96 at week 9): roughly executed share of the
plan
REQ_COV = {w: min(86, round(96 * EXECUTED[w - 1] / PLANNED_TOTAL * 0.96)) for w in WEEKS}
REQ_TARGET = {w: round(96 * PLANNED[w - 1] / PLANNED_TOTAL) for w in WEEKS}



def rag(w):
    m = ALL[w - 1]; prev = ALL[w - 2] if w > 1 else None; prev2 = ALL[w - 3] if w > 2 else None
    sched = RULES["Schedule"][1](m)
    rising_open = prev is not None and prev2 is not None and m["open"] > prev["open"] > prev2["open"]
    qual = "RED" if CRIT_AGE[w] > 7 or rising_open else "AMBER" if m["pass_rate"] < 85 else "GREEN"
    rc = REQ_COV[w] / max(1, REQ_TARGET[w])
    scope = "RED" if rc < 0.70 else "AMBER" if rc < 0.90 else "GREEN"
    rising_find = w > 4 and prev and prev2 and m["find_rate"] > prev["find_rate"] > prev2["find_rate"]
    risk = "RED" if rising_find else "AMBER" if m["closure_ratio"] < 80 else "GREEN"
    return {"Schedule": sched, "Quality": qual, "Scope": scope, "Risk": risk}



# ---------------------------------------------------------------------- defect trend (Task 4) ---------
def rayleigh_cum(t, K, s):
    return K * (1 - np.exp(-(t ** 2) / (2 * s ** 2)))



def fit_rayleigh(weeks=WEEKS, found=FOUND):
    p, cov = curve_fit(rayleigh_cum, np.array(weeks, float), np.array(found, float), p0=[300, 4],
maxfev=20000)
    pred = rayleigh_cum(np.array(weeks, float), *p)
    sse = float(((pred - np.array(found)) ** 2).sum())
    return {"K": float(p[0]), "sigma": float(p[1]), "sse": sse, "se_K": float(math.sqrt(cov[0, 0]))}



def goel_okumoto(t, a, b):
    return a * (1 - np.exp(-b * t))



def fit_go(weeks=WEEKS, found=FOUND):
    p, cov = curve_fit(goel_okumoto, np.array(weeks, float), np.array(found, float), p0=[300, 0.1],
maxfev=20000)
    pred = goel_okumoto(np.array(weeks, float), *p)
    return {"a": float(p[0]), "b": float(p[1]), "sse": float(((pred - np.array(found)) ** 2).sum())}



# ---------------------------------------------------------------------- stopping criteria ------------
def crit_A(w, extra_exec=0, extra_pass=0):
    i = w - 1; ex = EXECUTED[i] + extra_exec; pa = PASSED[i] + extra_pass
    return ex >= PLANNED_TOTAL and pa / ex >= 0.95, {"executed_pct": 100 * ex / PLANNED_TOTAL, "pass_pct": 100
* pa / ex}



def crit_B(w, threshold=10, open_critical=None):
    wk = weekly(FOUND); oc = OPEN_SEV_W9["critical"] if open_critical is None else open_critical
    two = w >= 2 and wk[w - 1] < threshold and wk[w - 2] < threshold
    return two and oc == 0, {"last_two": wk[max(0, w - 2):w], "open_critical": oc}



def crit_C(remaining, crit_open, p_find_2wk, test_cost_2wk, field_cost=COST["field"], crit_field_cost=None):
    crit_field_cost = crit_field_cost or 5 * field_cost
    release_now = remaining * field_cost + crit_open * crit_field_cost
    found2 = remaining * p_find_2wk
    after = (remaining - found2) * field_cost + found2 * COST["system"] + crit_open * COST["system"] +
test_cost_2wk
```

```
    return {"release_now": release_now, "test_2_more": after, "stop": release_now < after}


# ------------------------------------------------------------------------ control responses (Task 3) ----
DECAY, CLOSE_SCALE = 0.70, 0.85         # calibrated so that 'no change' reproduces weeks 6–9 (see exp_mon.py)


def project(response, start=5, end=9, base_rate=None, extra_testers=2, ramp=0.6, overhead=0.10, descope=0.20,
fix_gain=0.35, decay=None, close_scale=None, close_boost=1.4):
    """Project executed tests, defects found and open defects from week `start` to `end` for a control
response.
    Model: the team executes `rate` tests per week. Defects found are proportional to tests executed with the
    observed find ratio, which decays as the product stabilises. Closing capacity per week is the mean of
weeks 3–5."""
    rate = base_rate or (EXECUTED[start - 1] - EXECUTED[0]) / (start - 1)      # average weekly rate weeks 2–5
    per_tester = rate / 5; plan_total = PLANNED_TOTAL
    ex, fo, cl = EXECUTED[start - 1], FOUND[start - 1], CLOSED[start - 1]
    close_cap = (CLOSED[start - 1] - CLOSED[2]) / (start - 3) * (CLOSE_SCALE if close_scale is None else
close_scale)
    ratio = (FOUND[start - 1] - FOUND[start - 2]) / (EXECUTED[start - 1] - EXECUTED[start - 2])
    rows, cost = [], 0
    if response == "B": plan_total = PLANNED_TOTAL - descope * (PLANNED_TOTAL - ex)
    for w in range(start + 1, end + 1):
        r = rate
        if response == "A":
            add = 0 if w == start + 1 else extra_testers * per_tester * ramp
            drag = per_tester * extra_testers * 0.5 if w == start + 1 else 0      # training takes senior time
in week 1
            r = rate * 1.10 + add - drag                                          # 10 % overtime
            cost += (extra_testers * 30 * 900 if w <= start + 4 else 0) + 5 * 6 * 600   # temp testers +
overtime (assumed ₹/h)
        if response == "C":
            r = rate * (1 + fix_gain * min(1, (w - start - 1) / 1.0)) if w > start + 1 else rate * 1.05
            cl_boost = close_boost
        else:
            cl_boost = 1.0
        r = min(r, plan_total - ex)
        ex += r
        ratio_w = ratio * ((DECAY if decay is None else decay) ** (w - start))
        if response == "B": ratio_w *= 0.85                                       # dropped tests were low-risk
but not defect-free
        f = r * ratio_w; fo += f
        c = min(fo - cl, close_cap * cl_boost * (1.1 if response == "A" else 1.0)); cl += c
        rows.append({"week": w, "executed": ex, "found": fo, "open": fo - cl, "rate": r})
    return {"rows": rows, "plan_total": plan_total, "cost": cost, "rate0": rate, "ratio0": ratio}
```

F2. exp_mon.py

```
"""Tasks 1–4: metrics, milestones, early signal, colour rules, control projections, stopping criteria and
cost.
Writes results_mon.json."""
import json, os, sys
import numpy as np
HERE = os.path.dirname(os.path.abspath(__file__)); sys.path.insert(0, HERE)
from sure import *


R = {"metrics": ALL}
# ---------------- Task 1 milestones (definitions and check against the data)
MS = [("M1", "Test design complete; ≥ 20 % of planned tests executed by end of week 2", 2, lambda: EXECUTED[1]
>= 0.2 * PLANNED_TOTAL),
      ("M2", "50 % of planned tests executed by end of week 5", 5, lambda: EXECUTED[4] >= 0.5 *
PLANNED_TOTAL),
      ("M3", "90 % of planned tests executed by end of week 7", 7, lambda: EXECUTED[6] >= 0.9 *
PLANNED_TOTAL),
      ("M4", "Defect density ≤ 10/KLOC and open defects ≤ 30 at end of week 8", 8, lambda: FOUND[7] / KLOC <=
10 and FOUND[7] - CLOSED[7] <= 30),
      ("M5", "100 % executed, pass ≥ 95 %, 0 open critical, requirement coverage 100 % by end of week 9", 9,
lambda: EXECUTED[8] >= 1400 and PASSED[8] / EXECUTED[8] >= 0.95 and OPEN_SEV_W9["critical"] == 0)]
R["milestones"] = [{"id": m, "cond": c, "week": w, "met": bool(f())} for m, c, w, f in MS]
R["ms_values"] = {"M1": EXECUTED[1] / PLANNED_TOTAL, "M2": EXECUTED[4] / PLANNED_TOTAL, "M3": EXECUTED[6] /
PLANNED_TOTAL, "M4": (FOUND[7] / KLOC, FOUND[7] - CLOSED[7])}
```

```
# ---------------- early signals: first week each measure crossed its threshold (two consecutive weeks for
noise)
def first(cond, consec=2):
    run = 0
    for m in ALL:
        run = run + 1 if cond(m) else 0
        if run >= consec: return m["week"] - consec + 1
    return None



SIG = {"Schedule variance > 15 % behind": first(lambda m: -m["sv_pct"] > 15),
       "Open defects rising week on week": first(lambda m: m["week"] > 1 and m["open"] > ALL[m["week"] -
2]["open"]),
       "Closure ratio < 80 %": first(lambda m: m["closure_ratio"] < 80),
       "Pass rate < 85 %": first(lambda m: m["pass_rate"] < 85),
       "Defect density > 10/KLOC": first(lambda m: m["density"] > 10, consec=1),
       "Weekly execution below weekly plan": first(lambda m: m["exec_rate"] < m["plan_rate"])}
R["signals"] = SIG


# ---------------- Task 2 colour status
R["rag"] = {w: rag(w) for w in WEEKS}


# ---------------- Task 3 gap, rates, completion week
R["gap"] = [{"week": w, "gap": PLANNED[w - 1] - EXECUTED[w - 1], "rate": weekly(EXECUTED)[w - 1], "avg_rate":
EXECUTED[w - 1] / w} for w in WEEKS]
r5 = weekly(EXECUTED)[4]; avg5 = EXECUTED[4] / 5
R["completion"] = {"week5_rate": r5, "avg_rate_w1_5": avg5, "weeks_needed_w5rate": (PLANNED_TOTAL -
EXECUTED[4]) / r5, "weeks_needed_avg": (PLANNED_TOTAL - EXECUTED[4]) / avg5}
R["projections"] = {k: project(k) for k in "ABC"}
R["proj_base"] = project("0")
R["sens"] = {"A ramp 0.3": project("A", ramp=0.3)["rows"][-1], "A ramp 0.8": project("A", ramp=0.8)["rows"][-
1],
             "C fix gain 0.15": project("C", fix_gain=0.15)["rows"][-1], "C fix gain 0.0": project("C",
fix_gain=0.0)["rows"][-1],
             "B descope 10 %": project("B", descope=0.10)["rows"][-1], "C no gains (0, 1.0)": project("C",
fix_gain=0.0, close_boost=1.0)["rows"][-1], "C half gains (0.15, 1.2)": project("C", fix_gain=0.15,
close_boost=1.2)["rows"][-1]}
R["backlog"] = [{"week": w, "open": FOUND[w - 1] - CLOSED[w - 1], "in": weekly(FOUND)[w - 1], "out":
weekly(CLOSED)[w - 1]} for w in WEEKS]


# ---------------- Task 4 trend and stopping
ry, go = fit_rayleigh(), fit_go()
R["rayleigh"], R["go"] = ry, go
remaining = ry["K"] - FOUND[-1]
F = lambda t: rayleigh_cum(np.array([t], float), ry["K"], ry["sigma"])[0]
p2 = (F(11) - F(9)) / max(1e-9, ry["K"] - F(9))
R["remaining"] = remaining; R["remaining_ci"] = (remaining - 2 * ry["se_K"], remaining + 2 * ry["se_K"]);
R["p_find_2wk"] = p2
R["pred_weekly"] = [float(F(t) - F(t - 1)) for t in range(1, 13)]
# fit using only weeks 1–6 (what the model would have said earlier)
ry6 = fit_rayleigh(WEEKS[:6], FOUND[:6]); R["rayleigh_w6"] = ry6
TEST_COST_2WK = (5 * 60 * 700) + (6 * 60 * 800)            # 5 testers and 6 developers for 2 weeks (assumed
₹/h)
R["test_cost_2wk"] = TEST_COST_2WK
R["costC"] = {"crit_x1": crit_C(remaining, 3, p2, TEST_COST_2WK, crit_field_cost=COST["field"]),
              "crit_x5": crit_C(remaining, 3, p2, TEST_COST_2WK)}
R["critA"] = {w: crit_A(w) for w in (9,)}
R["critA_more"] = {"w11": crit_A(9, 60, 60 + int(0.05 * 0))}
R["critB"] = {thr: {w: crit_B(w, thr, open_critical=OPEN_SEV_W9["critical"] if w == 9 else None) for w in (8,
9)} for thr in (10, 15)}
# when could A pass? remaining 60 tests + re-run of the 130 failed tests after fixes (assumption: 90 % of re-
runs pass)
failed = EXECUTED[-1] - PASSED[-1]
R["critA_detail"] = {"failed_w9": failed, "pass_after_rerun": (PASSED[-1] + 60 * 0.95 + failed * 0.9) /
PLANNED_TOTAL}
json.dump(R, open(os.path.join(HERE, "results_mon.json"), "w"), indent=1, default=lambda o: o.tolist() if
hasattr(o, "tolist") else bool(o) if isinstance(o, np.bool_) else str(o))
print(SIG); print(R["milestones"]); print(R["completion"]); print(ry, remaining, R["remaining_ci"], p2);
print(R["costC"]); print(ry6); print(R["critA_detail"], R["critB"])
print({k: v["rows"][-1] for k, v in R["projections"].items()}, R["proj_base"]["rows"][-1])
```

F3. exp_scm.py

```
"""Task 5: three configuration-management set-ups, three changes in each, and a replay of the incident in
which
the test copy of fraud_rules.json reaches production. Writes results_scm.json."""
import json, os, sys, shutil, subprocess, hashlib, time, glob
HERE = os.path.dirname(os.path.abspath(__file__))
W = os.path.join(HERE, "_scm"); shutil.rmtree(W, ignore_errors=True); os.makedirs(W)
sha = lambda p: hashlib.sha256(open(p, "rb").read()).hexdigest()
R = {}
PROD_RULES = {"version": "FR-2.0", "photo_mismatch_score": 0.80, "claim_amount_flag": 250000,
"repeat_claims_12m": 3}
TEST_RULES = {"version": "FR-2.0", "photo_mismatch_score": 0.20, "claim_amount_flag": 5000,
"repeat_claims_12m": 1}    # testers lowered thresholds to force flags
CODE = "def settle(claim):\n    return min(claim['estimate'], claim['limit']) - claim['excess']\n"
FIX = "def settle(claim):\n    return max(0, min(claim['estimate'], claim['limit']) - claim['excess'])\n"
NOTE = "SureClaim v2.0 release note\n"
dump = lambda d: json.dumps(d, indent=1, sort_keys=True)



def w(p, s):
    os.makedirs(os.path.dirname(p), exist_ok=True); open(p, "w").write(s)



def deploy_flag(prod_dir):
    return json.load(open(os.path.join(prod_dir, "fraud_rules.json")))



# ---------------------------------------------------------------------- A: shared folder, versions by file name
A = os.path.join(W, "A_shared"); P_A = os.path.join(W, "A_prod")
w(f"{A}/settle.py", CODE); w(f"{A}/fraud_rules_final.json", dump(PROD_RULES)); w(f"{A}/test_settle.py", "#
tests\n"); w(f"{A}/release_note.txt", NOTE)
# change 1: threshold change; change 2: bug fix; change 3: rollback of change 1
w(f"{A}/fraud_rules_final_v2.json", dump({**PROD_RULES, "claim_amount_flag": 200000}))
w(f"{A}/settle_fixed.py", FIX)
w(f"{A}/fraud_rules_final_v3_ROLLBACK.json", dump(PROD_RULES))
w(f"{A}/fraud_rules_final_v3_test_copy.json", dump(TEST_RULES))            # tester's copy left in the folder
hist_A = sorted(os.listdir(A))
# incident replay: the deployer picks the "latest-looking" fraud rules file (sorted by name, last one)
cand = sorted(glob.glob(f"{A}/fraud_rules_final*.json")); picked = cand[-1]
os.makedirs(P_A, exist_ok=True); shutil.copy(picked, f"{P_A}/fraud_rules.json")
R["A"] = {"files": hist_A, "candidates": [os.path.basename(c) for c in cand], "picked":
os.path.basename(picked),
          "wrong_deployed": deploy_flag(P_A)["claim_amount_flag"] != PROD_RULES["claim_amount_flag"],
          "detected": "no automatic check; noticed when flag rate jumps in production (assumed 1–2 days)",
          "recovery_steps": ["find which of the candidate files is correct (diff by eye)", "copy it to
production", "no record of who changed what"],
          "can_reproduce_past_release": False, "change_log": "file names only; no author, date or reason"}


# ---------------------------------------------------------------------- B: Git with branches, tags and PRs
(simulated by merges)
B = os.path.join(W, "B_git"); P_B = os.path.join(W, "B_prod")
g = lambda *a, cwd=B: subprocess.run(["git", *a], cwd=cwd, capture_output=True, text=True)
os.makedirs(B); g("init", "-q", "-b", "main"); g("config", "user.email", "qa@harbour"); g("config",
"user.name", "release")
w(f"{B}/settle.py", CODE); w(f"{B}/config/fraud_rules.json", dump(PROD_RULES)); w(f"{B}/tests/test_settle.py",
"# tests\n"); w(f"{B}/release_note.txt", NOTE)
g("add", "."); g("commit", "-qm", "SureClaim v2.0 baseline"); g("tag", "v2.0.0")
g("checkout", "-qb", "feature/threshold"); w(f"{B}/config/fraud_rules.json", dump({**PROD_RULES,
"claim_amount_flag": 200000})); g("commit", "-qam", "CR-101 lower amount flag to 2 lakh")
g("checkout", "-q", "main"); g("merge", "-q", "--no-ff", "feature/threshold", "-m", "Merge PR #12 CR-101
(reviewed by QA)"); g("tag", "v2.0.1")
g("checkout", "-qb", "fix/negative"); w(f"{B}/settle.py", FIX); g("commit", "-qam", "BUG-77 settlement never
negative"); g("checkout", "-q", "main"); g("merge", "-q", "--no-ff", "fix/negative", "-m", "Merge PR #13 BUG-
77"); g("tag", "v2.0.2")
g("revert", "--no-edit", g("log", "--format=%H", "-1", "--grep", "CR-101 lower").stdout.strip()); g("tag",
"v2.0.3")
```

```
g("checkout", "-qb", "test/fraud-tuning"); w(f"{B}/config/fraud_rules.json", dump(TEST_RULES)); g("commit", "-
qam", "testers: force fraud flags"); g("checkout", "-q", "main")
R["B_log"] = g("log", "--oneline", "--graph", "--all", "--decorate").stdout.splitlines()
# deployment is from a tag; the incident: an engineer copies the file from his working copy of the test branch
def deploy_tag(tag, dest):
    shutil.rmtree(dest, ignore_errors=True); os.makedirs(dest)
    t0 = time.perf_counter(); subprocess.run(f"git archive {tag} | tar -x -C {dest}", shell=True, cwd=B,
check=True); return time.perf_counter() - t0
t_b = deploy_tag("v2.0.3", P_B)
os.makedirs(f"{P_B}/config", exist_ok=True)
g("checkout", "-q", "test/fraud-tuning"); shutil.copy(f"{B}/config/fraud_rules.json",
f"{P_B}/config/fraud_rules.json"); g("checkout", "-q", "main")
wrong = json.load(open(f"{P_B}/config/fraud_rules.json"))["claim_amount_flag"] !=
PROD_RULES["claim_amount_flag"]
t0 = time.perf_counter(); t_redeploy = deploy_tag("v2.0.3", P_B); t_rec = time.perf_counter() - t0
R["B"] = {"wrong_deployed": wrong, "detected": "not automatically (production is not compared with the tag);
found when someone runs git diff or flag rate jumps",
          "recovery_s": t_rec, "recovery": "redeploy tag v2.0.3 (one command)", "can_reproduce_past_release":
True,
          "find_change": g("log", "--format=%h %s", "--", "config/fraud_rules.json").stdout.splitlines()}


# ------------------------------------------------------------------ C: Git + baseline manifest + CCB record +
automatic audit
C = os.path.join(W, "C_scm"); P_C = os.path.join(W, "C_prod")
shutil.copytree(B, C)
gc = lambda *a: subprocess.run(["git", *a], cwd=C, capture_output=True, text=True)
CI = ["settle.py", "config/fraud_rules.json", "tests/test_settle.py", "release_note.txt"]
manifest = {"baseline": "BL-2.0.3", "tag": "v2.0.3", "ccb": "CCB-2026-14 approved fraud_rules.json FR-2.0 (QA,
fraud head, release manager)",
            "items": {p: sha(os.path.join(C, p)) for p in CI}}
w(f"{C}/baseline/BL-2.0.3.json", dump(manifest)); gc("add", "."); gc("commit", "-qm", "Baseline BL-2.0.3
manifest (CCB-2026-14)"); gc("tag", "BL-2.0.3")



def audit(prod):
    res = {}
    for p, h_ in manifest["items"].items():
        f = os.path.join(prod, p); res[p] = "MISSING" if not os.path.exists(f) else ("OK" if sha(f) == h_ else
"MISMATCH")
    return res



shutil.rmtree(P_C, ignore_errors=True); os.makedirs(P_C); subprocess.run(f"git archive v2.0.3 | tar -x -C
{P_C}", shell=True, cwd=C, check=True)
audit_before = audit(P_C)
shutil.copy(f"{B}/config/fraud_rules.json", f"{P_C}/config/fraud_rules.json")   # same wrong copy as in B
(main checkout = prod file)
w(f"{P_C}/config/fraud_rules.json", dump(TEST_RULES))
t0 = time.perf_counter(); audit_after = audit(P_C); t_aud = time.perf_counter() - t0
t0 = time.perf_counter()
for p, st in audit_after.items():
    if st != "OK": subprocess.run(f"git show v2.0.3:{p} > {os.path.join(P_C, p)}", shell=True, cwd=C,
check=True)
t_fix = time.perf_counter() - t0
R["C"] = {"audit_before": audit_before, "audit_after": audit_after, "audit_s": t_aud, "restore_s": t_fix,
"audit_restored": audit(P_C), "manifest": manifest,
          "detected": "yes – the scheduled audit (every 15 min and on every deploy) reports MISMATCH for
config/fraud_rules.json"}
# what a test result is worth if the item is unknown: the same fraud test gives opposite results on the two
files
claim = {"amount": 60000, "photo_mismatch": 0.35, "claims_12m": 1}
flag = lambda rules: claim["amount"] >= rules["claim_amount_flag"] or claim["photo_mismatch"] >=
rules["photo_mismatch_score"] or claim["claims_12m"] >= rules["repeat_claims_12m"]
R["same_test"] = {"prod_rules": flag(PROD_RULES), "test_rules": flag(TEST_RULES)}
json.dump(R, open(os.path.join(HERE, "results_scm.json"), "w"), indent=1, default=str)
print(R["A"]["picked"], R["A"]["wrong_deployed"]); print(R["B"]["wrong_deployed"], R["B"]["recovery_s"]);
print(R["C"]["audit_after"], R["C"]["audit_s"], R["C"]["audit_restored"]); print(R["same_test"]);
print("\n".join(R["B_log"]))
```

F4. settle_seeded.py

```
"""SureClaim claim-settlement function as submitted for review (contains 12 seeded defects D1-D12).


Requirements it must meet (excerpt of the settlement user stories):
  S1  settlement = min(approved estimate after depreciation, claim limit) - excess; never below 0
  S2  depreciation on rubber/plastic parts 50 %; on metal parts 0 % if the car is < 5 years old, else 20 %
(age >= 5)
  S3  the claim date must lie within the policy period, otherwise reject
  S4  claims flagged by the fraud rules are never auto-approved, whatever the amount
  S5  auto-approval only for settlements strictly below Rs 50,000; others go to a claims officer
  S6  money is held in integer paise; amounts are rounded half-up to the paisa
  S7  a claim is settled at most once (status must be APPROVED, not SETTLED)
  S8  the UPI payout API takes the amount in paise
  S9  logs must not contain the full bank account / UPI id
"""
import logging
from datetime import date


log = logging.getLogger("settle")
AUTO_LIMIT = 50000
LABOUR_RATE = 1.0



def settle_claim(claim, policy, parts, fraud_flag, upi):
    # compute the settlement for an approved claim and pay it
    if claim is None or policy is None:
        raise ValueError("claim and policy are required")
    # D10: SETTLED claims are not stopped (S7)
    if claim["status"] == "REJECTED":
        return {"result": "REJECTED", "reason": "claim rejected earlier"}
    breakdown = []
    # D7: float rupees instead of integer paise (S6)
    total = 0.0
    for p in parts:
        cost = p["cost"]
        material = p["material"]
        if material in ("rubber", "plastic"):
            # D4: magic number instead of a named constant
            allowed = cost * 0.5
        elif material == "metal":
            # D2: age 5 should already be depreciated (S2)
            if policy["car_age"] > 5:
                allowed = cost * 0.8
            else:
                allowed = cost
        elif material == "labour":
            allowed = cost * LABOUR_RATE
        else:
            allowed = cost
        breakdown.append({"part": p.get("name", material), "claimed": cost, "allowed": allowed})
        total += allowed
    # D6: ZeroDivisionError when parts is empty
    avg_part = total / len(parts)
    log.debug("average part cost %s", avg_part)
    # D3: no check that claim["date"] lies inside policy["start"]..policy["end"] (S3)
    # D1: excess taken before the limit cap (S1)
    amount = total - policy["excess"]
    if amount > policy["limit"]:
        amount = policy["limit"]
    if amount < 0:
        amount = 0
    amount = round(amount, 2)
    # D9: must be strictly below 50,000 (S5)
    if amount <= AUTO_LIMIT:
        # D5: fraud_flag ignored on this path (S4)
        decision = "AUTO_APPROVED"
    elif fraud_flag:
        decision = "REFER_FRAUD"
    else:
        decision = "REFER_OFFICER"
    result = {"result": decision, "amount": amount, "breakdown": breakdown}
    if decision == "AUTO_APPROVED":
```

```
        # D8: full UPI id written to the log (S9)
        log.info("paying %s to %s", amount, upi["vpa"])
        # D11: rupees passed where paise are expected (S8)
        ref = upi["api"].pay(upi["vpa"], amount)
        claim["status"] = "SETTLED"
        claim["settled_on"] = date.today().isoformat()
        result["ref"] = ref
        return result
    if decision == "REFER_FRAUD":
        claim["status"] = "FRAUD_REVIEW"
    else:
        claim["status"] = "OFFICER_REVIEW"
    # D12: misleading comment below - amount is in rupees
    # return the amount in paise
    return result
```

F5. exp_rev.py

```
"""Tasks 6–8: confirm the 12 seeded defects by executable checks, run the static-analysis tool, apply the
three review
procedures (each defined by what it examines), compute detection %, effort, defects/hour and cost saving,
build the review
records in three formats and the follow-up traceability, and calculate the review plans. Writes
results_rev.json."""
import json, os, sys, subprocess, logging, io
HERE = os.path.dirname(os.path.abspath(__file__)); sys.path.insert(0, HERE)
import settle_seeded as S
from sure import COST


R = {}
LOC = sum(1 for l in open(os.path.join(HERE, "settle_seeded.py")) if l.strip() and not
l.strip().startswith("#") and '"""' not in l)
code_lines = [l for l in open(os.path.join(HERE, "settle_seeded.py")).read().split('"""', 2)[2].splitlines()]
R["loc_function"] = sum(1 for l in code_lines if l.strip())



class Api:
    def __init__(self): self.calls = []
    def pay(self, vpa, amt): self.calls.append((vpa, amt)); return "UPI123"



def run(claim=None, policy=None, parts=None, flag=False):
    claim = claim or {"status": "APPROVED", "date": "2026-10-01"}
    policy = policy or {"car_age": 3, "excess": 1000, "limit": 40000, "start": "2026-01-01", "end": "2026-12-
31"}
    parts = [{"material": "metal", "cost": 10000}] if parts is None else parts
    api = Api(); buf = io.StringIO(); h = logging.StreamHandler(buf); S.log.addHandler(h);
S.log.setLevel(logging.DEBUG)
    try: out = S.settle_claim(claim, policy, parts, flag, {"vpa": "ravi.k@okhdfc", "api": api})
    except Exception as e: out = {"error": type(e).__name__}
    S.log.removeHandler(h); return out, api, buf.getvalue()



# executable confirmation of each seeded defect (expected per the requirements S1–S9)
chk = {}
o, *_ = run(policy={"car_age": 3, "excess": 5000, "limit": 40000}, parts=[{"material": "metal", "cost":
60000}]); chk["D1"] = o["amount"] != 35000      # min(60000,40000)-5000
o, *_ = run(policy={"car_age": 5, "excess": 0, "limit": 99999}, parts=[{"material": "metal", "cost": 10000}]);
chk["D2"] = o["amount"] != 8000
o, *_ = run(claim={"status": "APPROVED", "date": "2027-03-01"}); chk["D3"] = o.get("result") != "REJECTED"
chk["D4"] = "* 0.5" in open(os.path.join(HERE, "settle_seeded.py")).read()
o, *_ = run(parts=[], policy={"car_age": 3, "excess": 0, "limit": 1}); chk["D6"] = o.get("error") ==
"ZeroDivisionError"
o, *_ = run(flag=True); chk["D5"] = o.get("result") == "AUTO_APPROVED"
o, *_ = run(parts=[{"material": "rubber", "cost": 101}], policy={"car_age": 1, "excess": 0, "limit": 99999});
chk["D7"] = isinstance(o["amount"], float)
o, api, logtxt = run(); chk["D8"] = "ravi.k@okhdfc" in logtxt
o, *_ = run(policy={"car_age": 1, "excess": 0, "limit": 99999}, parts=[{"material": "metal", "cost": 50000}]);
chk["D9"] = o["result"] == "AUTO_APPROVED"
```

```
c = {"status": "SETTLED", "date": "2026-10-01"}; o, api, _ = run(claim=c); chk["D10"] = len(api.calls) == 1
o, api, _ = run(); chk["D11"] = api.calls and api.calls[0][1] == 9000
chk["D12"] = "return the amount in paise" in open(os.path.join(HERE, "settle_seeded.py")).read()
R["confirmed"] = chk


DEF = {  # id: (type, severity, requirement)
    "D1": ("logic", "major", "S1"), "D2": ("logic/boundary", "major", "S2"), "D3": ("missing requirement",
"major", "S3"), "D4": ("standards", "minor", "coding std"),
    "D5": ("logic", "critical", "S4"), "D6": ("error handling", "major", "robustness"), "D7": ("standards/data
type", "major", "S6"), "D8": ("security", "major", "S9"),
    "D9": ("boundary", "minor", "S5"), "D10": ("logic/state", "critical", "S7"), "D11": ("interface",
"critical", "S8"), "D12": ("documentation", "minor", "coding std")}
R["defects"] = DEF


# static analysis with ruff (all default rules + bugbear + security), measured
r = subprocess.run(["ruff", "check", "--select", "E,F,W,B,S,PLR2004", "--line-length", "140", "--output-
format", "concise", "--no-cache", "settle_seeded.py"], cwd=HERE, capture_output=True, text=True)
R["ruff"] = [l for l in r.stdout.splitlines() if "settle_seeded.py" in l]


# ------------------------------------------------------------------ review procedures (Task 6)
# A desk-check: one reviewer reads the code alone, without the requirement text or a checklist.
#   Can see: crashes, data-type and style problems, misleading comments visible in the code itself.
# B technical review: three reviewers compare the code with the requirements S1-S9 and the coding standard.
#   Can see: everything that contradicts a stated requirement or standard; state/history problems if a
reviewer asks.
# C inspection: moderator, author, reader, recorder, 2 inspectors; entry criteria (code compiles, unit tests
pass);
#   checklist (each requirement traced to a line, every boundary, every error path, state, security,
interfaces).
FOUND = {
    "A": ["D4", "D6", "D7", "D12"],
    "B": ["D1", "D2", "D3", "D4", "D5", "D7", "D9", "D11"],
    "C": ["D1", "D2", "D3", "D4", "D5", "D6", "D7", "D8", "D9", "D10", "D11", "D12"],
}
WHY_MISSED = {
    "A": {"D1": "needs S1 (order of cap and excess)", "D2": "needs S2 (age 5 rule)", "D3": "requirement absent
from code; nothing to see", "D5": "needs S4", "D8": "log line looks normal",
          "D9": "needs S5 (strictly below)", "D10": "needs knowledge of claim states", "D11": "needs API
contract"},
    "B": {"D6": "reviewers focused on requirements, not error paths", "D8": "no security item on the agenda",
"D10": "state history not discussed", "D12": "comments not reviewed"},
    "C": {},
}
EFFORT = {  # person-hours (estimate from standard rates; LOC of the function)
    "A": {"prep": 0.5, "meeting": 0.0, "followup": 0.25, "people": 1},
    "B": {"prep": 3 * 0.75, "meeting": 3 * 1.0 + 1.0, "followup": 0.5, "people": 3},
    "C": {"prep": 4 * (R["loc_function"] / 150 + 0.25), "meeting": 6 * 1.5, "followup": 1.0, "people": 6},
}
RES = {}
for k in "ABC":
    e = EFFORT[k]; hrs = e["prep"] + e["meeting"] + e["followup"]; n = len(FOUND[k])
    save_sys = n * (COST["system"] - COST["review"]); save_field = n * (COST["field"] - COST["review"])
    review_cost = hrs * 800                                          # ₹800 per person-hour (assumption)
    RES[k] = {"found": n, "ddp": 100 * n / len(DEF), "hours": hrs, "per_hour": n / hrs, "save_vs_system":
save_sys, "save_vs_field": save_field,
              "review_cost": review_cost, "net_vs_system": save_sys - review_cost,
              "by_sev": {s: sum(1 for d in FOUND[k] if DEF[d][1] == s) for s in ("critical", "major",
"minor")}}
R["found"] = FOUND; R["why_missed"] = WHY_MISSED; R["effort"] = EFFORT; R["res"] = RES
R["ruff_maps"] = {}   # which seeded defects a static tool reported
src = open(os.path.join(HERE, "settle_seeded.py")).read().splitlines()
for l in R["ruff"]:
    ln = int(l.split(":")[1]); ctx = " ".join(src[max(0, ln - 3):ln])    # seeded-defect marker sits in the
comment just above
    hit = [d for d in DEF if f"# {d}:" in ctx]
    for d in hit[-1:]: R["ruff_maps"].setdefault(d, []).append(l.split(":", 3)[3].strip())


# ------------------------------------------------------------------ Task 7 review plans
PROJECT_H = 15 * 12 * 30; LIMIT = 0.10 * PROJECT_H
REQ_N, DESIGN_N, KLOC_ = 85, 12, 22
# effort assumptions: requirement review 6 min per requirement per reviewer; design review 2 h prep per
reviewer per document + 1 h meeting
```

```
def plan_effort(name):
    if name == "A":     # every second Friday, 2 h for 6 people on whatever is finished (6 sessions in 12
weeks)
        return {"Reviews (6 × 2 h × 6 people)": 6 * 2 * 6, "Preparation (none required)": 0}
    if name == "B":     # risk/size driven
        hi_code = 6.0; lo_code = KLOC_ - hi_code; hi_req = 30; lo_req = REQ_N - hi_req
        return {"Inspection of high-risk requirements (30 × 0.1 h × 4)": hi_req * 0.1 * 4, "Desk-check of
other requirements (55 × 0.1 h × 1)": lo_req * 0.1,
                "Inspection of 4 high-risk designs (4 × (2 h × 4 + 1.5 h × 5))": 4 * (2 * 4 + 1.5 * 5),
"Technical review of 8 other designs (8 × (1 h × 3 + 1 h × 3))": 8 * (3 + 3),
                "Code inspection of 6 KLOC high-risk (6000/150 × 3 inspectors + 6 × 1.5 h × 5)": 6000 / 150 *
3 + 6 * 1.5 * 5, "Desk-check of 16 KLOC (16000/300 × 1)": 16000 / 300}
    if name == "C":     # gates: all requirements, all designs, all code reviewed at its gate
        return {"Requirement review gate (85 × 0.1 h × 4)": REQ_N * 0.1 * 4, "Design review gate (12 × (2 h ×
3 + 1 h × 4))": 12 * (2 * 3 + 1 * 4),
                "Code review gate, all 22 KLOC (22000/150 × 2 reviewers)": 22000 / 150 * 2, "Gate meetings and
rework checks (3 gates × 4 h × 5)": 3 * 4 * 5}



R["plans"] = {k: plan_effort(k) for k in "ABC"}
R["plan_tot"] = {k: sum(v.values()) for k, v in R["plans"].items()}
R["project_h"], R["limit_h"] = PROJECT_H, LIMIT


# ---------------------------------------------------------------- Task 8 review records and follow-up
COMMENTS = [  # from the inspection of settle_seeded.py (C) – id, line marker, type, severity, text, action
owner
    ("RC-01", "D5", "logic", "critical", "Fraud flag is not checked before auto-approval; flagged claims below
₹50,000 are paid.", "author"),
    ("RC-02", "D10", "logic/state", "critical", "Only REJECTED is stopped; a SETTLED claim is paid again.",
"author"),
    ("RC-03", "D11", "interface", "critical", "UPI API expects paise; amount passed in rupees (₹90 paid
instead of ₹9,000).", "author"),
    ("RC-04", "D1", "logic", "major", "Excess subtracted before the limit cap; S1 requires cap first.",
"author"),
    ("RC-05", "D2", "boundary", "major", "car_age > 5 should be >= 5 (S2).", "author"),
    ("RC-06", "D3", "missing", "major", "No check that the claim date lies in the policy period (S3).",
"author + BA"),
    ("RC-07", "D6", "error handling", "major", "Division by len(parts) fails for an empty parts list; avg_part
is unused anyway.", "author"),
    ("RC-08", "D7", "standards", "major", "Money held as float rupees; standard requires integer paise and
half-up rounding.", "author"),
    ("RC-09", "D8", "security", "major", "Full UPI id written to the info log (S9).", "author"),
    ("RC-10", "D9", "boundary", "minor", "AUTO_LIMIT comparison must be < not <= (S5).", "author"),
    ("RC-11", "D4", "standards", "minor", "Magic number 0.5; use DEPRECIATION_RUBBER.", "author"),
    ("RC-12", "D12", "documentation", "minor", "Comment says paise; value is rupees.", "author"),
]
R["comments"] = COMMENTS
# formats: A = e-mail minutes written by the author from memory after the meeting (summary of the main points)
EMAIL = ("Subject: Review of settle_claim – summary\n\nThanks all. Main points: (1) fraud flag must be checked
before auto-approval; "
         "(2) UPI amount should be in paise; (3) excess/limit order to be fixed; (4) money as paise instead of
float; "
         "(5) some boundary issues with age and the 50k limit; (6) will tidy constants and comments.\nWill fix
this week. – Author")
covered_by_email = {"RC-01": "(1)", "RC-03": "(2)", "RC-04": "(3)", "RC-08": "(4)", "RC-05": "(5)", "RC-10":
"(5)", "RC-11": "(6)", "RC-12": "(6)"}
R["email"] = EMAIL
FOLLOW = {}
for cid, d, typ, sev, txt, own in COMMENTS:
    a = cid in covered_by_email
    FOLLOW[cid] = {"A_recorded": a, "A_traceable": False, "A_verified": False,       # no owner/status/link in
an e-mail
                   "B_recorded": True, "B_traceable": True, "B_verified": True, "B_linked": False,   #
verified at the follow-up date by the moderator; no link to the change
                   "C_recorded": True, "C_traceable": True, "C_verified": True, "C_linked": True}    # each
action linked to the fixing commit and closed by the reviewer
# after two weeks the author has fixed 9 actions (assumption: RC-06 waits for the BA, RC-07 and RC-12
deferred)
FIXED = {c[0] for c in COMMENTS} - {"RC-06", "RC-07", "RC-12"}
summ = {}
for f in "ABC":
    rec = sum(FOLLOW[c][f + "_recorded"] for c in FOLLOW); tr = sum(FOLLOW[c][f + "_traceable"] for c in
FOLLOW)
```

```
    ver = sum(1 for c in FOLLOW if FOLLOW[c][f + "_verified"] and c in FIXED)
    open_known = sum(1 for c in FOLLOW if FOLLOW[c][f + "_traceable"] and c not in FIXED)
    summ[f] = {"recorded": rec, "traceable": tr, "verified_fixed": ver, "open_known": open_known, "linked":
sum(FOLLOW[c].get(f + "_linked", False) for c in FOLLOW)}
R["follow"] = summ
hrs = RES["C"]["hours"]; maj = sum(1 for c in COMMENTS if c[3] in ("critical", "major"))
R["rmetrics"] = {"defects": len(COMMENTS), "hours": hrs, "per_hour": len(COMMENTS) / hrs, "per_kloc":
len(COMMENTS) / (R["loc_function"] / 1000),
                 "pct_major": 100 * maj / len(COMMENTS), "loc": R["loc_function"]}
from collections import Counter
R["by_type"] = Counter(c[2].split("/")[0] for c in COMMENTS).most_common()
json.dump(R, open(os.path.join(HERE, "results_rev.json"), "w"), indent=1, default=str)
print(chk); print(R["ruff"]); print(R["ruff_maps"]); print({k: (v["found"], round(v["hours"], 2),
round(v["per_hour"], 2)) for k, v in RES.items()})
print(R["plan_tot"], LIMIT); print(summ); print(R["rmetrics"]); print(R["loc_function"])
# review copy given to reviewers: the seeded-defect marker comments are removed
src_lines = open(os.path.join(HERE, "settle_seeded.py")).read().splitlines()
import re as _re
copy = [l for l in src_lines if not _re.match(r"\s*# D\d+:", l)]
copy = [l if "seeded defects" not in l else l.replace("(contains 12 seeded defects D1-D12)", "(review copy)")
for l in copy]
open(os.path.join(HERE, "settle_review_copy.py"), "w").write("\n".join(copy) + "\n")
print("review copy lines", len(copy))
```

F6. qc_proto.py

```
"""Task 9 prototype: reads the weekly tracking data, computes the metric set of each configuration for every
week,
applies the colour rules and the stopping criteria, draws the charts and reports the release decision.
Run:  python qc_proto.py  (writes data/tracking.csv if missing, figs/q_*.png and results_proto.json)"""
import csv, json, os, sys
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
HERE = os.path.dirname(os.path.abspath(__file__)); sys.path.insert(0, HERE)
import sure


DATA = os.path.join(HERE, "data", "tracking.csv"); os.makedirs(os.path.dirname(DATA), exist_ok=True)
if not os.path.exists(DATA):
    with open(DATA, "w", newline="") as f:
        wr = csv.writer(f); wr.writerow(["week", "planned", "executed", "passed", "found", "closed"])
        for i, w in enumerate(sure.WEEKS): wr.writerow([w, sure.PLANNED[i], sure.EXECUTED[i], sure.PASSED[i],
sure.FOUND[i], sure.CLOSED[i]])
rows = [{k: int(v) for k, v in r.items()} for r in csv.DictReader(open(DATA))]


CONFIG = {
    "A": {"name": "Minimal", "metrics": ["pct_executed", "sv"], "report": "narrative e-mail", "scm": "folder
versioning", "review": "informal",
          "stop": "planned tests executed ≥ 95 %", "review_h": 72, "report_h_week": 3.0, "audit_items": 1},
    "B": {"name": "Balanced", "metrics": ["pct_executed", "sv", "pass_rate", "open", "find_rate", "density",
"closure_ratio", "req_cov"], "report": "RAG one-pager",
          "scm": "Git + tags", "review": "risk-based (plan B of Task 7)", "stop": "defect trend < 10/week for
2 weeks and 0 open critical", "review_h": 346, "report_h_week": 1.5, "audit_items": 4},
    "C": {"name": "Comprehensive", "metrics": ["pct_executed", "sv", "pass_rate", "open", "find_rate",
"density", "closure_ratio", "req_cov", "stmt", "branch", "crit_age"],
          "report": "live dashboard", "scm": "baselines + automated audit", "review": "inspections of critical
items + tool tracking",
          "stop": "expected cost of release < cost of 2 more weeks", "review_h": 507, "report_h_week": 0.5,
"audit_items": 6},
}
AUDIT_REQ = ["test completion evidence", "configuration audit", "review records", "defect status at release",
"coverage", "decision sign-off"]



def week_metrics(r):
    m = {"week": r["week"], "pct_executed": 100 * r["executed"] / r["planned"], "sv": r["executed"] -
r["planned"], "pass_rate": 100 * r["passed"] / r["executed"],
         "open": r["found"] - r["closed"], "density": r["found"] / sure.KLOC, "closure_ratio": 100 *
r["closed"] / r["found"], "req_cov": sure.REQ_COV[r["week"]]}
    i = r["week"] - 1
```

```
    m["find_rate"] = r["found"] - (rows[i - 1]["found"] if i else 0)
    m["crit_age"] = sure.CRIT_AGE[r["week"]]
    return m



M = [week_metrics(r) for r in rows]
ry = sure.fit_rayleigh([r["week"] for r in rows], [r["found"] for r in rows]); remaining = ry["K"] - rows[-
1]["found"]
OUT = {}
for k, c in CONFIG.items():
    # early warning: first week a metric of this configuration crosses its threshold (two weeks running)
    tests = {"pct_executed": lambda m: m["pct_executed"] < 85, "sv": lambda m: m["sv"] < -0.15 *
sure.PLANNED[m["week"] - 1],
             "pass_rate": lambda m: m["pass_rate"] < 85, "open": lambda m: m["week"] > 1 and m["open"] >
M[m["week"] - 2]["open"], "closure_ratio": lambda m: m["closure_ratio"] < 80,
             "density": lambda m: m["density"] > 10, "find_rate": lambda m: False, "req_cov": lambda m: False,
"crit_age": lambda m: m["crit_age"] > 7}
    first = {}
    for met in c["metrics"]:
        f = tests.get(met)
        if not f: continue
        run = 0
        for m in M:
            run = run + 1 if f(m) else 0
            if run >= 2: first[met] = m["week"] - 1; break
    warn = min(first.values()) if first else None
    # release decision at week 9
    m9 = M[-1]
    if k == "A":
        release = m9["pct_executed"] >= 95; reason = f"{m9['pct_executed']:.1f} % executed"
    elif k == "B":
        ok, info = sure.crit_B(9, 10); release = ok; reason = f"last two weeks {info['last_two']}, open
critical {info['open_critical']}"
    else:
        F = lambda t: sure.rayleigh_cum(np.array([t], float), ry["K"], ry["sigma"])[0]; p2 = (F(11) - F(9)) /
(ry["K"] - F(9))
        cc = sure.crit_C(remaining, 3, p2, (5 * 60 * 700) + (6 * 60 * 800)); release = cc["stop"]; reason =
f"release now ₹{cc['release_now']:,.0f} vs 2 more weeks ₹{cc['test_2_more']:,.0f}"
    # expected leakage if released when the configuration says so
    F = lambda t: sure.rayleigh_cum(np.array([t], float), ry["K"], ry["sigma"])[0]
    rel_week = 9 if release else 11
    p2_ = (F(11) - F(9)) / (ry["K"] - F(9))
    resid = remaining if rel_week == 9 else remaining * (1 - p2_)
    review_catch = {"A": 0.10, "B": 0.30, "C": 0.40}[k]       # share of residual defects of the kinds reviews
remove before test (assumption from Task 6 DDP × reviewed share)
    crit_left = 3 if rel_week == 9 else 0
    leak = resid * (1 - review_catch) + crit_left
    op_h = c["report_h_week"] * 12 + {"A": 2, "B": 10, "C": 30}[k]      # reporting + set-up of SCM/audit
tooling
    OUT[k] = {"name": c["name"], "first_signal": first, "early_warning_week": warn, "release_at_w9":
bool(release), "reason": reason, "release_week": rel_week,
              "leak": leak, "review_h": c["review_h"], "review_pct": 100 * c["review_h"] / 5400, "op_h": op_h,
              "audit": f"{c['audit_items']} of {len(AUDIT_REQ)}", "audit_n": c["audit_items"], "leak_cost":
leak * sure.COST["field"]}
    # charts per configuration
    fig, ax = plt.subplots(figsize=(6.4, 2.8)); wk = [m["week"] for m in M]
    ax.plot(wk, [r["planned"] for r in rows], "--", color="#263238", label="planned"); ax.plot(wk,
[r["executed"] for r in rows], "-o", color="#3B6FB6", label="executed")
    if k != "A":
        ax.plot(wk, [r["passed"] for r in rows], "-s", color="#1F9E89", label="passed"); ax2 = ax.twinx();
ax2.bar(wk, [m["open"] for m in M], color="#C0504D", alpha=0.25, label="open defects"); ax2.set_ylabel("open
defects")
    if warn: ax.axvline(warn, color="#C0504D", ls=":"); ax.text(warn + 0.1, 1300, f"first warning W{warn}",
color="#C0504D", fontsize=8)
    ax.set_title(f"Configuration {k} ({c['name']}) – prototype output"); ax.legend(fontsize=7, loc="upper
left", frameon=False); ax.set_xlabel("week")
    fig.tight_layout(); fig.savefig(os.path.join(HERE, "figs", f"q_{k}.png"), dpi=200); plt.close(fig)
# limitation: cumulative passed grows faster than cumulative executed in weeks 8 and 9
lim = [{"week": r["week"], "exec_w": r["executed"] - (rows[i - 1]["executed"] if i else 0), "pass_w":
r["passed"] - (rows[i - 1]["passed"] if i else 0),
        "failed_cum": r["executed"] - r["passed"]} for i, r in enumerate(rows)]
OUT["limitation"] = lim; OUT["remaining"] = remaining; OUT["metrics"] = M
json.dump(OUT, open(os.path.join(HERE, "results_proto.json"), "w"), indent=1, default=str)
```

```
for k in "ABC": print(k, {x: OUT[k][x] for x in ("early_warning_week", "release_at_w9", "reason", "leak",
"review_pct", "op_h", "audit")})
print(lim)
```

F7. qc10.py

```
"""Task 10: fortnightly continuous-delivery releases. Generates six 2-week sprints of daily data (24 people, 3
vendors),
injects three events, and applies strategies A (retain + scale down), B (pipeline gates + PR checklists + tool
audit
report) and C (quality-as-code: gate/review/config policies in the repo, signed builds, automatic evidence
record).
Writes results_10.json and policy/quality_policy.json."""
import json, os, random, hmac, hashlib
HERE = os.path.dirname(os.path.abspath(__file__))
rng = random.Random(10)
os.makedirs(os.path.join(HERE, "policy"), exist_ok=True)
POLICY = {  # quality-as-code (strategy C) – kept in the repository, changed only by pull request with QA +
release-manager approval
    "gates": {"unit_pass_pct": 100, "regression_pass_pct": 98, "open_critical": 0,
"branch_cov_changed_code_pct": 70, "upi_payout_p95_min": 30, "fraud_api_contract": "pass"},
    "reviews": {"config/*.json": {"type": "inspection-lite", "approvers": ["QA", "fraud-owner"]}, "payout/**":
{"type": "PR + checklist", "approvers": 2}, "**": {"type": "PR", "approvers": 1}},
    "config_items": ["config/fraud_rules.json", "config/claim_limits.json", "config/premium_tables.json",
"pipeline.yml"],
    "evidence": ["build_sha", "signed_by", "gate_results", "reviews", "config_audit", "test_summary"],
}
json.dump(POLICY, open(os.path.join(HERE, "policy", "quality_policy.json"), "w"), indent=1)
KEY = b"release-signing-key (in HSM in production)"


SPRINTS, DAYS = 6, 10
data = []
for s in range(1, SPRINTS + 1):
    for d in range(1, DAYS + 1):
        commits = rng.randint(14, 26); vendor = rng.randint(3, 9)
        tests = 900 + 15 * (s - 1); passed = tests - rng.randint(0, 6)
        p95 = rng.uniform(6, 14)
        found = max(0, int(rng.gauss(3.0 if d < 8 else 1.5, 1.2))); closed = max(0, int(rng.gauss(2.8, 1.0)))
        data.append({"sprint": s, "day": d, "commits": commits, "vendor_commits": vendor, "tests": tests,
"passed": passed, "upi_p95": p95,
                     "found": found, "closed": closed, "critical_open": 0, "unreviewed_config": 0,
"fraud_contract": "pass"})
# injected events
EV = {"E1 vendor changes fraud_rules.json without review": (3, 4), "E2 UPI payout p95 rises above 30 min": (5,
6), "E3 external AI fraud API changes its response field": (4, 2)}
for x in data:
    if (x["sprint"], x["day"]) == EV["E1 vendor changes fraud_rules.json without review"]:
x["unreviewed_config"] = 1
    if x["sprint"] == 5 and x["day"] >= 6: x["upi_p95"] = 34 + 3 * (x["day"] - 6)
    if (x["sprint"], x["day"]) >= EV["E3 external AI fraud API changes its response field"]:
x["fraud_contract"] = "fail" if (x["sprint"], x["day"]) < (4, 9) else "pass"
    if (x["sprint"], x["day"]) in [(4, 3), (4, 4)]: x["critical_open"] = 1



def detect(strategy):
    """Return the day index at which each event is detected (None = not before the release decision)."""
    det = {}
    for name, (s, d) in EV.items():
        hit = None
        for x in data:
            if (x["sprint"], x["day"]) < (s, d): continue
            if strategy == "A":            # metrics reviewed once per release (day 10); no config audit;
manual load check per release
                if x["day"] == 10 and name.startswith("E2") and x["upi_p95"] > 30: hit = x
                if x["day"] == 10 and name.startswith("E3") and x["fraud_contract"] == "fail": hit = x
            elif strategy == "B":          # gates on every build; PR checklist; audit report generated at
release
                if name.startswith("E2") and x["upi_p95"] > 30: hit = x
                if name.startswith("E3") and x["fraud_contract"] == "fail": hit = x
                if name.startswith("E1") and x["day"] == 10: hit = x          # tool audit report at release
lists the unreviewed change
```

```
            else:                           # C: policy-as-code blocks merge of config without required
approvers; gates on every build
                if name.startswith("E1") and x["unreviewed_config"]: hit = x
                if name.startswith("E2") and x["upi_p95"] > POLICY["gates"]["upi_payout_p95_min"]: hit = x
                if name.startswith("E3") and x["fraud_contract"] != POLICY["gates"]["fraud_api_contract"]: hit
= x
            if hit: break
        det[name] = None if hit is None else (hit["sprint"] - s) * DAYS + hit["day"] - d
    return det



# overhead per 2-week cycle (person-hours): counts × minutes, from the sprint data
CYCLE_H = 24 * 10 * 6.0             # 24 people × 10 days × 6 productive hours
LIMIT = 0.08 * CYCLE_H
commits_per_cycle = sum(x["commits"] for x in data) / SPRINTS
OVER = {
    "A": {"Task 9 metric set, reduced, weekly → per release": 4, "Status report once per release": 2,
"Shortened inspections of critical items (2 per cycle × 6 people × 1.5 h)": 18,
         "Risk-based technical reviews (4 per cycle × 3 people × 1 h)": 12, "Manual configuration audit at
release": 3, "Evidence pack assembled by hand": 10, "Vendor coordination calls (3 vendors × 1 h × 2)": 6},
    "B": {"Pipeline gate maintenance": 4, f"PR reviews with checklist ({commits_per_cycle * 0.3:.0f} PRs ×
0.25 h)": commits_per_cycle * 0.3 * 0.25, "Checklist upkeep": 1,
         "Tool-generated audit report review": 2, "Sprint metrics review (2 × 0.5 h × 4 people)": 4, "Vendor
coordination calls": 4},
    "C": {"Policy-as-code upkeep (PR to policy file)": 2, f"PR reviews ({commits_per_cycle * 0.3:.0f} PRs ×
0.2 h, checklists enforced by bot)": commits_per_cycle * 0.3 * 0.2,
         "Inspection-lite of config changes (≈ 3 per cycle × 2 people × 0.5 h)": 3, "Evidence record review
and sign-off": 1, "Sprint metrics review": 2, "Vendor onboarding to the same pipeline (amortised)": 2},
}
OVT = {k: sum(v.values()) for k, v in OVER.items()}



# strategy C evidence record for each release, signed (HMAC stands in for a signature from the build server's
key)
def evidence(sprint):
    rec = {"release": f"2.{sprint}.0", "build_sha": hashlib.sha256(f"build-
{sprint}".encode()).hexdigest()[:16],
           "gate_results": {g: "pass" for g in POLICY["gates"]}, "reviews": {"prs": int(commits_per_cycle *
0.3), "config_inspections": 3},
           "config_audit": "OK", "test_summary": {"tests": data[sprint * DAYS - 1]["tests"], "passed":
data[sprint * DAYS - 1]["passed"]}}
    sx = [x for x in data if x["sprint"] == sprint]
    if any(x["upi_p95"] > 30 for x in sx): rec["gate_results"]["upi_payout_p95_min"] = "FAIL (blocked, fixed
day 9)"
    if any(x["fraud_contract"] == "fail" for x in sx): rec["gate_results"]["fraud_api_contract"] = "FAIL
(blocked until adapter fixed)"
    if any(x["unreviewed_config"] for x in sx): rec["config_audit"] = "BLOCKED: fraud_rules.json change lacked
fraud-owner approval; merged after approval"
    body = json.dumps(rec, sort_keys=True).encode()
    rec["signed_by"] = "build-server"; rec["signature"] = hmac.new(KEY, body, hashlib.sha256).hexdigest()
    return rec



EVID = [evidence(s) for s in range(1, SPRINTS + 1)]
# tamper check: changing any field invalidates the signature
t = dict(EVID[2]); sig = t.pop("signature"); t.pop("signed_by"); t["config_audit"] = "OK"
tamper_ok = hmac.new(KEY, json.dumps(t, sort_keys=True).encode(), hashlib.sha256).hexdigest() == sig
AUDIT = {"A": {"test completion": "manual summary", "config audit": "manual, at release", "review records":
"minutes", "complete for every release": False},
         "B": {"test completion": "tool report", "config audit": "tool report at release", "review records":
"PR history", "complete for every release": True},
         "C": {"test completion": "signed record", "config audit": "automatic, every build", "review records":
"PR + policy check", "complete for every release": True}}
R = {"policy": POLICY, "events": EV, "detect": {k: detect(k) for k in "ABC"}, "overhead": OVER,
"overhead_tot": OVT, "cycle_h": CYCLE_H, "limit_h": LIMIT,
     "evidence_example": EVID[2], "evidence_all": EVID, "tamper_detected": not tamper_ok, "audit": AUDIT,
"commits_per_cycle": commits_per_cycle,
     "daily": [{k: (round(v, 1) if isinstance(v, float) else v) for k, v in x.items()} for x in data]}
# original Task 9 practice unchanged in a 2-week cycle (scaled from 12 weeks): reviews 507 h/12 weeks → per 2
weeks; reporting; audits
```

```
R["t9_unchanged"] = {"Reviews at the Task 9 intensity (507 h ÷ (15 people × 12 weeks) × 24 people × 2 weeks)":
507 / (15 * 12) * 24 * 2,
                     "Weekly status (2 × 1.5 h)": 3, "Manual configuration audit per release": 3, "Evidence
pack assembled by hand per release": 10, "Review planning per release": 4}
R["t9_unchanged_tot"] = sum(R["t9_unchanged"].values())
json.dump(R, open(os.path.join(HERE, "results_10.json"), "w"), indent=1, default=str)
print(R["detect"]); print(OVT, LIMIT, CYCLE_H); print(R["tamper_detected"], R["t9_unchanged_tot"]);
print(json.dumps(EVID[2], indent=1)[:600])
```

F8. policy/quality_policy.json

```
{
 "gates": {
  "unit_pass_pct": 100,
  "regression_pass_pct": 98,
  "open_critical": 0,
  "branch_cov_changed_code_pct": 70,
  "upi_payout_p95_min": 30,
  "fraud_api_contract": "pass"
 },
 "reviews": {
  "config/*.json": {
   "type": "inspection-lite",
   "approvers": [
    "QA",
    "fraud-owner"
   ]
  },
  "payout/**": {
   "type": "PR + checklist",
   "approvers": 2
  },
  "**": {
   "type": "PR",
   "approvers": 1
  }
 },
 "config_items": [
  "config/fraud_rules.json",
  "config/claim_limits.json",
  "config/premium_tables.json",
  "pipeline.yml"
 ],
 "evidence": [
  "build_sha",
  "signed_by",
  "gate_results",
  "reviews",
  "config_audit",
  "test_summary"
 ]
}
```

ANNEXURE G – Viva Questions and Model Answers

Task 1

#	Question	Model answer
1	What was the schedule variance at week 5?	720 − 960 = −240 tests (−25 %).
2	Which measure warned first?	Schedule variance and closure ratio (weeks 1–2), 7–8 weeks before week 9; open-defect growth from week 2.
3	Why was M2 met?	It was defined on 50 % of all 1,400 tests (720 ≥ 700), not on the week-5 plan.
4	Defect density at week 9?	260 ÷ 22 = 11.8 per KLOC, above the target of 10.
5	Which GQM metric is manual?	Critical defects found after release (M8).

Task 2

#	Question	Model answer
1	Colour of week 9?	Schedule GREEN, Quality RED, Scope AMBER, Risk GREEN.
2	Which format shows the widening gap earliest?	The chart pack – the shaded gap grows weekly.
3	Why not only a dashboard?	No record, no trend, depends on attendance.
4	Week-6 pass rate?	750 ÷ 900 = 83.3 %.
5	Effort of A + B?	About 1.5 h per week with a script.

Task 3

#	Question	Model answer
1	When would the plan finish at the week-5 rate?	680 ÷ 160 = 4.25 weeks → week 10.
2	How was the model calibrated?	Decay and closure constants set so that ‘no change’ reproduces week 9 (1,340 / 260 / 20).
3	Cost of adding testers?	About ₹2.88 lakh (model assumptions).
4	What did the backlog show?	Fixing was the bottleneck; open grew 10 → 60 until closures exceeded arrivals in week 6.
5	Is Brooks’s law confirmed?	Only for week 6; later A is ahead, but the backlog stays because fixing is the bottleneck.

Task 4

#	Question	Model answer
1	How many defects remain?	Rayleigh K ≈ 271 → ≈ 11 remain (5–17).
2	Why not the exponential model?	The curve is S-shaped; the exponential fit is 40 times worse and gives an absurd total.
3	Cost: release now vs two more weeks?	₹8.65 lakh (critical ×1) or ₹15.85 lakh (×5) vs ₹7.74 lakh.
4	Your stopping rule?	< 10 new/week for 2 weeks, 0 critical, ≤ 5 major, all high-risk tests passed, cost check.
5	Why ‘never complete’?	≈ 2–11 defects remain whatever the criterion; the decision is about residual risk.

Task 5

#	Question	Model answer
1	Which approach detected the wrong file?	Only C – the audit reported MISMATCH in milliseconds.

#	Question	Model answer
2	How fast was recovery in B?	One command (redeploy tag) in milliseconds; but detection took ~36 h.
3	Why is CM part of testing?	The same fraud test passed with production rules and failed with the test copy.
4	Items under change control?	fraud_rules.json, premium and limit tables, build/deploy scripts, environment config, DB migrations, release note.
5	AI tuning of thresholds?	Only as a proposal in a PR with back-test evidence, approved by the CCB.

Task 6

#	Question	Model answer
1	DDP of each review?	A 33 %, B 67 %, C 100 % of 12 seeded defects.
2	Defects per hour?	A 5.3, B 1.2, C 0.9.
3	Why did the desk-check miss D5?	No requirement text – the fraud rule S4 was not in front of the reviewer.
4	What did ruff find?	Only a magic value at D2.
5	Your rule?	Inspection for money, fraud and controlled config; technical review for other designs; desk-check for the rest.

Task 7

#	Question	Model answer
1	10 % limit?	15 × 12 × 30 = 5,400 h; 540 h.
2	Effort of the plans?	A 72 h, B 346 h, C 507 h.
3	Which plan is earliest?	B (high-risk requirements in week 1).
4	Most affected by a slip?	C – gates block the next phase.
5	What if entry criteria are skipped?	Detection drops to desk-check level at inspection cost.

Task 8

#	Question	Model answer
1	Metrics of the review?	0.93 defects/h; 174/KLOC; 12.8 h; 75 % major.
2	Traceable after two weeks?	E-mail 0, report 12 (not linked), tool 12 linked.
3	Most frequent defect type?	Logic, then boundary and standards.
4	Why not rank reviewers?	It corrupts the data – trivial comments and severity bargaining.
5	Decision for this review?	Re-review (3 critical in money logic).

Task 9

#	Question	Model answer
1	What does the prototype do?	Reads tracking data, computes metrics, applies colour rules and stopping criteria, fits the trend, draws charts per configuration.
2	Which configuration releases wrongly?	A at 95.7 % executed with 3 critical open.
3	Which result disagreed with expectation?	Passed > executed in weeks 8–9 because re-runs count as passes but not as executions.
4	Final practice?	B + automatic audit + inspections of money/fraud items; AI only as assistant.
5	Leakage of B and C?	About 1.5 and 1.3 defects.

Task 10

#	Question	Model answer
1	8 % of a cycle?	24 × 10 × 6 = 1,440 h; 115.2 h.
2	Task 9 practice unchanged?	155 h – exceeds the limit.
3	Overhead of C?	21.9 h (1.5 %).
4	Which events did A miss?	The unreviewed vendor change to fraud_rules.json and the AI API contract break.
5	When does C fail?	If the AI fraud service drifts without an interface change; add a drift gate.

ANNEXURE H – Full Weekly Metric Table (weeks 1–9, from sure.metrics)

Week	% exec	Pass %	SV	SV %	Find/wk	Close/wk	Open	Dens.	Closure %	Def/100
1	83.3	88.0	-20	-16.7	15	5	10	0.7	33.3	15.0
2	80.0	83.3	-60	-20.0	27	15	22	1.9	47.6	17.5
3	78.8	82.9	-110	-21.2	38	26	34	3.6	57.5	19.5
4	75.7	80.4	-180	-24.3	50	32	52	5.9	60.0	23.2
5	75.0	81.9	-240	-25.0	48	40	60	8.1	66.3	24.7
6	78.3	83.3	-250	-21.7	36	38	58	9.7	72.9	23.8
7	84.4	85.2	-200	-15.6	24	34	48	10.8	79.8	22.0
8	90.4	87.4	-130	-9.6	14	34	28	11.5	88.9	20.5
9	95.7	90.3	-60	-4.3	8	16	20	11.8	92.3	19.4

Colour status for every week

Week	Schedule	Quality	Scope	Risk
1	RED	GREEN	AMBER	AMBER
2	RED	AMBER	AMBER	AMBER
3	RED	RED	AMBER	AMBER
4	RED	RED	AMBER	AMBER
5	RED	RED	AMBER	AMBER
6	RED	RED	AMBER	AMBER
7	RED	GREEN	AMBER	AMBER
8	AMBER	RED	AMBER	GREEN
9	GREEN	RED	AMBER	GREEN

Rayleigh forecast of weekly discovery

Week	1	2	3	4	5	6	7	8	9	10	11	12
Actual	15	27	38	50	48	36	24	14	8	–	–	–
Model	11.1	30.6	43.3	47.2	43.6	35.2	25.3	16.3	9.5	5.1	2.4	1.1

ANNEXURE I – Weekly RAG Status Pages, Weeks 1–9 (generated by the prototype rules)

Week 1	Status	Comment
Schedule	RED	100 of 120 executed (83.3 %); variance -20
Quality	GREEN	Pass 88.0 %; open 10; oldest critical 2 days (assumed)
Scope	AMBER	About 7 of 96 stories covered (target 8)
Risk	AMBER	Find 15/wk, close 5/wk; closure ratio 33.3 %

Week 2	Status	Comment
Schedule	RED	240 of 300 executed (80.0 %); variance -60
Quality	AMBER	Pass 83.3 %; open 22; oldest critical 5 days (assumed)
Scope	AMBER	About 16 of 96 stories covered (target 21)
Risk	AMBER	Find 27/wk, close 15/wk; closure ratio 47.6 %

Week 3	Status	Comment
Schedule	RED	410 of 520 executed (78.8 %); variance -110
Quality	RED	Pass 82.9 %; open 34; oldest critical 9 days (assumed)
Scope	AMBER	About 27 of 96 stories covered (target 36)
Risk	AMBER	Find 38/wk, close 26/wk; closure ratio 57.5 %

Week 4	Status	Comment
Schedule	RED	560 of 740 executed (75.7 %); variance -180
Quality	RED	Pass 80.4 %; open 52; oldest critical 12 days (assumed)
Scope	AMBER	About 37 of 96 stories covered (target 51)
Risk	AMBER	Find 50/wk, close 32/wk; closure ratio 60.0 %

Week 5	Status	Comment
Schedule	RED	720 of 960 executed (75.0 %); variance -240
Quality	RED	Pass 81.9 %; open 60; oldest critical 10 days (assumed)
Scope	AMBER	About 47 of 96 stories covered (target 66)
Risk	AMBER	Find 48/wk, close 40/wk; closure ratio 66.3 %

Week 6	Status	Comment
Schedule	RED	900 of 1150 executed (78.3 %); variance -250
Quality	RED	Pass 83.3 %; open 58; oldest critical 8 days (assumed)
Scope	AMBER	About 59 of 96 stories covered (target 79)
Risk	AMBER	Find 36/wk, close 38/wk; closure ratio 72.9 %

Week 7	Status	Comment
Schedule	RED	1080 of 1280 executed (84.4 %); variance -200
Quality	GREEN	Pass 85.2 %; open 48; oldest critical 6 days (assumed)
Scope	AMBER	About 71 of 96 stories covered (target 88)
Risk	AMBER	Find 24/wk, close 34/wk; closure ratio 79.8 %

Week 8	Status	Comment
Schedule	AMBER	1230 of 1360 executed (90.4 %); variance -130
Quality	RED	Pass 87.4 %; open 28; oldest critical 9 days (assumed)
Scope	AMBER	About 81 of 96 stories covered (target 93)
Risk	GREEN	Find 14/wk, close 34/wk; closure ratio 88.9 %

Week 9	Status	Comment
Schedule	GREEN	1340 of 1400 executed (95.7 %); variance -60
Quality	RED	Pass 90.3 %; open 20; oldest critical 12 days (assumed)
Scope	AMBER	About 86 of 96 stories covered (target 96)
Risk	GREEN	Find 8/wk, close 16/wk; closure ratio 92.3 %

ANNEXURE J – Review Checklist Used for the Inspection and the Review Copy

#	Checklist item	Seeded defect it targets
1	Every requirement S1–S9 is traced to a line of code	D3
2	Every comparison with a limit matches the wording (below / at / above)	D2, D9
3	Money is held in integer paise; rounding half-up	D7
4	Every external call uses the unit its contract requires	D11
5	Every state transition is allowed (APPROVED → SETTLED only)	D10
6	Fraud flag is checked on every approval path	D5
7	Division, indexing and empty collections are guarded	D6
8	No secrets or full account/UPI ids in logs	D8
9	No magic numbers – named constants	D4
10	Comments agree with the code	D12
11	Error paths return or raise a defined result	D6
12	Dates are checked against the policy period	D3

Review copy given to reviewers (markers removed)

```
"""SureClaim claim-settlement function as submitted for review (review copy).


Requirements it must meet (excerpt of the settlement user stories):
  S1  settlement = min(approved estimate after depreciation, claim limit) - excess; never below 0
  S2  depreciation on rubber/plastic parts 50 %; on metal parts 0 % if the car is < 5 years old, else 20 %
(age >= 5)
  S3  the claim date must lie within the policy period, otherwise reject
  S4  claims flagged by the fraud rules are never auto-approved, whatever the amount
  S5  auto-approval only for settlements strictly below Rs 50,000; others go to a claims officer
  S6  money is held in integer paise; amounts are rounded half-up to the paisa
  S7  a claim is settled at most once (status must be APPROVED, not SETTLED)
  S8  the UPI payout API takes the amount in paise
  S9  logs must not contain the full bank account / UPI id
"""
import logging
from datetime import date


log = logging.getLogger("settle")
AUTO_LIMIT = 50000
LABOUR_RATE = 1.0



def settle_claim(claim, policy, parts, fraud_flag, upi):
    # compute the settlement for an approved claim and pay it
    if claim is None or policy is None:
        raise ValueError("claim and policy are required")
    if claim["status"] == "REJECTED":
        return {"result": "REJECTED", "reason": "claim rejected earlier"}
    breakdown = []
    total = 0.0
    for p in parts:
        cost = p["cost"]
        material = p["material"]
        if material in ("rubber", "plastic"):
            allowed = cost * 0.5
        elif material == "metal":
            if policy["car_age"] > 5:
                allowed = cost * 0.8
            else:
                allowed = cost
        elif material == "labour":
            allowed = cost * LABOUR_RATE
        else:
            allowed = cost
        breakdown.append({"part": p.get("name", material), "claimed": cost, "allowed": allowed})
        total += allowed
    avg_part = total / len(parts)
```

```
log.debug("average part cost %s", avg_part)
amount = total - policy["excess"]
if amount > policy["limit"]:
    amount = policy["limit"]
if amount < 0:
    amount = 0
amount = round(amount, 2)
if amount <= AUTO_LIMIT:
    decision = "AUTO_APPROVED"
elif fraud_flag:
    decision = "REFER_FRAUD"
else:
    decision = "REFER_OFFICER"
result = {"result": decision, "amount": amount, "breakdown": breakdown}
if decision == "AUTO_APPROVED":
    log.info("paying %s to %s", amount, upi["vpa"])
    ref = upi["api"].pay(upi["vpa"], amount)
    claim["status"] = "SETTLED"
    claim["settled_on"] = date.today().isoformat()
    result["ref"] = ref
    return result
if decision == "REFER_FRAUD":
    claim["status"] = "FRAUD_REVIEW"
else:
    claim["status"] = "OFFICER_REVIEW"
# return the amount in paise
return result
```

ANNEXURE K – Configuration Records of Task 5

Git history of structure B (output of git log --graph)

```
* fb71f79 (test/fraud-tuning) testers: force fraud flags
* 5dd5c4e (HEAD -> main, tag: v2.0.3) Revert "CR-101 lower amount flag to 2 lakh"
*   2bc43aa (tag: v2.0.2) Merge PR #13 BUG-77
|\
| * 1921df9 (fix/negative) BUG-77 settlement never negative
|/
*   3827580 (tag: v2.0.1) Merge PR #12 CR-101 (reviewed by QA)
|\
| * 6c7c310 (feature/threshold) CR-101 lower amount flag to 2 lakh
|/
* c37172e (tag: v2.0.0) SureClaim v2.0 baseline
```

Baseline manifest BL-2.0.3 (structure C)

```
{
 "baseline": "BL-2.0.3",
 "tag": "v2.0.3",
 "ccb": "CCB-2026-14 approved fraud_rules.json FR-2.0 (QA, fraud head, release manager)",
 "items": {
  "settle.py": "6393c8a37a609c7a4888847b3afea1031d61b8904e3be7aa7f480c0ac59b9c72",
  "config/fraud_rules.json": "562e88df38432b9ece2bf3f39328a91dfc0b12fc6376c5aad9b6c63d1cb80a92",
  "tests/test_settle.py": "d9d7a49315104067ccd93c792222819c2a9cad762359466130def97ae7658bc3",
  "release_note.txt": "618c716d4cfd00152bb719826bcbd0c0b9c77aac05a06d49fecf9493cbeb7000"
 }
}
```

Audit results

Item	Before incident	After wrong deploy	After automatic restore
settle.py	OK	OK	OK
config/fraud_rules.json	OK	MISMATCH	OK
tests/test_settle.py	OK	OK	OK
release_note.txt	OK	OK	OK

Change request and CCB record template

Field	Example (CR-101)
Item	config/fraud_rules.json
Change	claim_amount_flag 2,50,000 → 2,00,000
Reason	Fraud team: rise in mid-value staged claims
Evidence	Back-test on Q3 claims: +14 flags/week, 2 confirmed frauds
Tests	Fraud-rule regression suite; flag-rate check in staging
CCB decision	Approved 14-10-2026 (QA, fraud head, release manager)
Baseline	BL-2.0.1
Rollback	Revert commit; new baseline BL-2.0.3

ANNEXURE L – Sprint Data of the Task 10 Prototype (daily)

Sprint	Day	Commits	Vendor	Tests	Passed	UPI p95 min	Found	Closed	Unreviewed config	AI API contract
1	1	23	3	900	897	9.9	4	2		pass
1	2	21	9	900	898	11.2	3	3		pass
1	3	19	3	900	899	13.6	2	1		pass
1	4	20	9	900	899	10.8	2	3		pass
1	5	24	5	900	897	7.4	2	3		pass
1	6	21	9	900	894	13.9	4	2		pass
1	7	14	7	900	900	7.9	3	4		pass
1	8	22	5	900	894	13.2	0	3		pass
1	9	20	6	900	900	11.2	0	3		pass
1	10	17	6	900	899	6.3	0	2		pass
2	1	24	3	915	911	13.4	4	3		pass
2	2	19	7	915	914	6.9	3	2		pass
2	3	16	9	915	914	13.6	1	3		pass
2	4	17	8	915	913	7.1	0	2		pass
2	5	18	6	915	913	7.3	1	1		pass
2	6	26	4	915	912	12.2	2	3		pass
2	7	22	8	915	915	8.7	2	3		pass
2	8	21	6	915	911	7.3	2	2		pass
2	9	17	8	915	911	13.9	1	3		pass
2	10	21	3	915	914	9.9	0	3		pass
3	1	20	7	930	926	8.8	1	3		pass
3	2	22	6	930	930	13.7	2	2		pass
3	3	14	6	930	926	6.5	2	3		pass
3	4	23	6	930	924	13.9	1	2	yes	pass
3	5	20	9	930	926	13.4	2	3		pass
3	6	26	8	930	927	9.4	1	1		pass
3	7	16	8	930	929	8.6	3	2		pass
3	8	23	7	930	924	11.0	2	2		pass
3	9	18	6	930	924	12.9	1	0		pass
3	10	14	7	930	926	12.1	4	2		pass
4	1	18	4	945	940	9.8	2	3		pass
4	2	26	4	945	943	7.5	1	2		fail
4	3	15	8	945	943	13.1	1	3		fail
4	4	25	9	945	941	11.8	1	3		fail
4	5	17	5	945	941	8.9	1	1		fail
4	6	17	6	945	941	9.3	1	4		fail
4	7	17	3	945	939	9.4	3	2		fail
4	8	18	5	945	942	9.4	2	4		fail

Sprint	Day	Commits	Vendor	Tests	Passed	UPI p95 min	Found	Closed	Unreviewed config	AI API contract
4	9	21	8	945	943	10.1	0	2		pass
4	10	21	8	945	943	8.8	1	4		pass
5	1	21	7	960	957	6.7	1	2		pass
5	2	25	6	960	954	7.3	2	2		pass
5	3	23	3	960	956	12.8	3	2		pass
5	4	19	8	960	958	6.6	2	3		pass
5	5	23	4	960	959	7.3	4	3		pass
5	6	14	4	960	956	34.0	3	2		pass
5	7	23	6	960	954	37.0	3	2		pass
5	8	23	4	960	956	40.0	0	2		pass
5	9	19	9	960	960	43.0	1	2		pass
5	10	19	9	960	957	46.0	0	1		pass
6	1	15	7	975	969	12.9	3	2		pass
6	2	22	7	975	969	8.8	1	3		pass
6	3	19	3	975	974	13.4	1	3		pass
6	4	21	8	975	972	13.9	4	4		pass
6	5	19	7	975	971	9.7	3	3		pass
6	6	16	3	975	969	7.5	1	1		pass
6	7	26	5	975	971	9.0	3	2		pass
6	8	18	4	975	975	10.6	2	3		pass
6	9	16	5	975	969	6.3	0	2		pass
6	10	19	9	975	974	9.9	2	4		pass

ANNEXURE M – Formula Sheet

Quantity	Formula	Worked value
% executed	executed ÷ planned to date	720 ÷ 960 = 75 %
Pass rate	passed ÷ executed	1,210 ÷ 1,340 = 90.3 %
Schedule variance	executed − planned	−240 (week 5)
Defect density	found ÷ KLOC	260 ÷ 22 = 11.8
Weeks to finish	remaining ÷ rate	680 ÷ 160 = 4.25
Rayleigh	K(1 − e^(−t²/2σ²))	K = 271, σ = 3.46
Expected cost of release	remaining × ₹60,000 + critical × field cost	11.4 × 60,000 + 3 × 60,000 = ₹8.65 lakh
Cost of 2 more weeks	test + dev hours × rates + remaining after testing	₹4.98 lakh + … = ₹7.74 lakh
DDP	found ÷ seeded	C: 12 ÷ 12 = 100 %
Review saving	n × (later cost − ₹2,500)	C vs system: 12 × 9,500 = ₹1,14,000
Review limit	0.10 × 15 × 12 × 30	540 h
Cycle overhead limit	0.08 × 24 × 10 × 6	115.2 h

ANNEXURE O – Approved Review Plan (plan B) in Detail

#	Work product	Risk	Review type	Reviewers	Week	Effort (h)
1	Fraud-flag requirements (12 stories)	High	Inspection	QA, BA, fraud owner, 2 testers	1	4.8
2	Payout and settlement stories (10)	High	Inspection	QA, BA, 2 developers	1	4.0
3	Damage estimate and photo upload stories (8)	High	Inspection	QA, BA, 2 testers	1	3.2
4	Other 55 stories	Low–medium	Desk-check	1 tester each	2	5.5
5	4 high-risk designs (fraud, payout, estimate, upload)	High	Inspection	Moderator + 4	3	62
6	8 other designs	Medium	Technical review	3 reviewers	3–4	48
7	6 KLOC high-risk code	High	Inspection (≤ 400 lines per meeting)	Moderator + 3	5–8	165
8	16 KLOC other code	Low–medium	Desk-check with checklist	1 reviewer	5–8	53
	Total					**346**

ANNEXURE N – Procedure to Reproduce the Results

- Install Python 3.10+, git, graphviz and ruff; `pip install numpy scipy matplotlib`.
- `python exp_mon.py` – Tasks 1–4 (metrics, signals, projections, trend, costs).
- `python exp_scm.py` – Task 5 (creates `_scm/` with three structures and replays the incident).
- `python exp_rev.py` – Tasks 6–8 (confirms seeded defects, runs ruff, review metrics, plans, follow-up).
- `python qc_proto.py` – Task 9 prototype (charts q_A–q_C).
- `python qc10.py` – Task 10 sprint prototype (policy file, evidence records).
- `python figs5.py` – all other figures.

ANNEXURE P – Self-Evaluation Against the Evidence Expected

Task	Evidence	Where	Self-check
1	Metric table, milestones, GQM, final set	Table 2, Table 3, Figure 1, Table 7	Complete
2	Three week-6 reports, colour rules, choice	Table 9, Figure 2–Figure 4, Table 10, Table 11	Complete
3	Gap, projections, decision	Table 15, Table 16	Complete
4	Criteria, trend, cost, rule	Table 19, Figure 8, Table 21	Complete
5	Structures, replay, history, choice	Figure 10, Table 24, Table 23	Complete
6	Review table, problems, rule	Table 27, Table 29, Table 30	Classmate repeat open
7	Plans, effort, criteria, choice	Table 32, Table 33, Table 35	Complete
8	Three records, metrics, follow-up	Table 37–Table 42	Complete
9	Prototype, diagram, DDT, transfer	Table 45, Figure 22, Table 47, Table 48	Complete
10	Three strategies applied, decision	Table 51, Table 54, Table 55	Complete

ANNEXURE Q – Re-review: Corrected settle_claim and Verification

After the re-review decision (Task 8), all twelve actions were applied. The executable checks that confirmed each seeded defect were rerun on the corrected function (`exp_fix.py`):

Check (defect and requirement)	Result on settle_fixed.py
D1 cap before excess (S1)	PASS
D2 age 5 depreciated (S2)	PASS
D3 policy period (S3)	PASS
D4 named constants	PASS
D5 fraud flag (S4)	PASS
D6 empty parts	PASS
D7 paise half-up (S6)	PASS
D8 no UPI id in log (S9)	PASS
D9 ₹50,000 not auto (S5)	PASS
D10 no second payment (S7)	PASS
D11 paise to UPI (S8)	PASS
D12 comment matches	PASS
Total	**12 of 12 pass**

settle_fixed.py

```
"""settle_claim after the re-review: all 12 review actions applied (requirements S1-S9 as in
settle_seeded.py)."""
import logging
from datetime import date
from decimal import Decimal, ROUND_HALF_UP


log = logging.getLogger("settle")
AUTO_LIMIT_PAISE = 50000 * 100          # S5: strictly below Rs 50,000
DEPRECIATION_RUBBER = Decimal("0.50")   # S2
DEPRECIATION_METAL_OLD = Decimal("0.20")
METAL_AGE_YEARS = 5
SETTLEABLE = {"APPROVED"}               # S7



def to_paise(rupees):
    return int((Decimal(str(rupees)) * 100).quantize(Decimal("1"), rounding=ROUND_HALF_UP))   # S6



def mask(vpa):
    name, _, bank = vpa.partition("@")
    return name[:2] + "***@" + bank                                  # S9



def settle_claim(claim, policy, parts, fraud_flag, upi):
    if claim is None or policy is None:
        raise ValueError("claim and policy are required")
    if claim["status"] not in SETTLEABLE:                            # RC-02 / S7
        return {"result": "NOT_SETTLEABLE", "status": claim["status"]}
    if not (policy["start"] <= claim["date"] <= policy["end"]):      # RC-06 / S3
        claim["status"] = "REJECTED"
        return {"result": "REJECTED", "reason": "claim date outside policy period"}
    if not parts:                                                    # RC-07
        return {"result": "REFER_OFFICER", "reason": "no parts in estimate"}
    breakdown, total = [], 0
    for p in parts:
        cost = to_paise(p["cost"])
        if p["material"] in ("rubber", "plastic"):
            allowed = cost * (1 - DEPRECIATION_RUBBER)               # RC-11
        elif p["material"] == "metal" and policy["car_age"] >= METAL_AGE_YEARS:   # RC-05
            allowed = cost * (1 - DEPRECIATION_METAL_OLD)
```

```
        else:
            allowed = Decimal(cost)
        allowed = int(Decimal(allowed).quantize(Decimal("1"), rounding=ROUND_HALF_UP))
        breakdown.append({"part": p.get("name", p["material"]), "claimed_paise": cost, "allowed_paise":
allowed})
        total += allowed
    capped = min(total, to_paise(policy["limit"]))                    # RC-04 / S1: cap first
    amount = max(0, capped - to_paise(policy["excess"]))
    if fraud_flag:                                                    # RC-01 / S4
        decision = "REFER_FRAUD"
    elif amount < AUTO_LIMIT_PAISE:                                   # RC-10 / S5
        decision = "AUTO_APPROVED"
    else:
        decision = "REFER_OFFICER"
    result = {"result": decision, "amount_paise": amount, "breakdown": breakdown}
    if decision == "AUTO_APPROVED":
        log.info("paying %d paise to %s", amount, mask(upi["vpa"]))    # RC-09
        result["ref"] = upi["api"].pay(upi["vpa"], amount)             # RC-03: paise
        claim["status"] = "SETTLED"
        claim["settled_on"] = date.today().isoformat()
        return result
    claim["status"] = "FRAUD_REVIEW" if decision == "REFER_FRAUD" else "OFFICER_REVIEW"
    return result                                                     # RC-12: amount is in paise
```

exp_fix.py

```
"""Re-review check: the same executable checks that confirmed D1-D12 are run on settle_fixed.py. Writes
results_fix.json."""
import json, os, sys, io, logging
HERE = os.path.dirname(os.path.abspath(__file__)); sys.path.insert(0, HERE)
import settle_fixed as S



class Api:
    def __init__(self): self.calls = []
    def pay(self, vpa, amt): self.calls.append((vpa, amt)); return "UPI123"



def run(claim=None, policy=None, parts=None, flag=False):
    claim = claim or {"status": "APPROVED", "date": "2026-10-01"}
    base = {"car_age": 3, "excess": 1000, "limit": 40000, "start": "2026-01-01", "end": "2026-12-31"};
base.update(policy or {})
    parts = [{"material": "metal", "cost": 10000}] if parts is None else parts
    api = Api(); buf = io.StringIO(); h = logging.StreamHandler(buf); S.log.addHandler(h);
S.log.setLevel(logging.DEBUG)
    try: out = S.settle_claim(claim, base, parts, flag, {"vpa": "ravi.k@okhdfc", "api": api})
    except Exception as e: out = {"error": type(e).__name__}
    S.log.removeHandler(h); return out, api, buf.getvalue()



ok = {}
o, *_ = run(policy={"excess": 5000, "limit": 40000}, parts=[{"material": "metal", "cost": 60000}]); ok["D1 cap
before excess (S1)"] = o["amount_paise"] == 3500000 and o["result"] == "AUTO_APPROVED"
o, *_ = run(policy={"car_age": 5, "excess": 0, "limit": 99999}, parts=[{"material": "metal", "cost": 10000}]);
ok["D2 age 5 depreciated (S2)"] = o["amount_paise"] == 800000
o, *_ = run(claim={"status": "APPROVED", "date": "2027-03-01"}); ok["D3 policy period (S3)"] = o["result"] ==
"REJECTED"
ok["D4 named constants"] = "* 0.5" not in open(os.path.join(HERE, "settle_fixed.py")).read()
o, *_ = run(flag=True); ok["D5 fraud flag (S4)"] = o["result"] == "REFER_FRAUD"
o, *_ = run(parts=[]); ok["D6 empty parts"] = o.get("result") == "REFER_OFFICER"
o, *_ = run(parts=[{"material": "rubber", "cost": 101.01}], policy={"car_age": 1, "excess": 0, "limit":
99999}); ok["D7 paise half-up (S6)"] = o["amount_paise"] == 5051
o, api, logtxt = run(); ok["D8 no UPI id in log (S9)"] = "ravi.k@okhdfc" not in logtxt and "ra***@okhdfc" in
logtxt
o, *_ = run(policy={"car_age": 1, "excess": 0, "limit": 99999}, parts=[{"material": "metal", "cost": 50000}]);
ok["D9 ₹50,000 not auto (S5)"] = o["result"] == "REFER_OFFICER"
c = {"status": "SETTLED", "date": "2026-10-01"}; o, api, _ = run(claim=c); ok["D10 no second payment (S7)"] =
len(api.calls) == 0
o, api, _ = run(); ok["D11 paise to UPI (S8)"] = api.calls[0][1] == 900000
ok["D12 comment matches"] = "amount is in paise" in open(os.path.join(HERE, "settle_fixed.py")).read()
```

```
json.dump(ok, open(os.path.join(HERE, "results_fix.json"), "w"), indent=1)
print(ok, sum(ok.values()))
```

ANNEXURE R – Signed Evidence Records of the Six Prototype Releases (strategy C)

Each record is produced by the pipeline at release time and signed; the regulator verifies the signature with the build server's public key (HMAC in the prototype).

Release 2.1.0

Field	Value
Build	e844adc60c72d254
Gates	unit_pass_pct: pass; regression_pass_pct: pass; open_critical: pass; branch_cov_changed_code_pct: pass; upi_payout_p95_min: pass; fraud_api_contract: pass
Reviews	59 PRs, 3 config inspections
Configuration audit	OK
Test summary	899 of 900 passed
Signature	4ba863c6b7bef39f0efb5e2a7429f785…

Release 2.2.0

Field	Value
Build	4f6caf3fbf934663
Gates	unit_pass_pct: pass; regression_pass_pct: pass; open_critical: pass; branch_cov_changed_code_pct: pass; upi_payout_p95_min: pass; fraud_api_contract: pass
Reviews	59 PRs, 3 config inspections
Configuration audit	OK
Test summary	914 of 915 passed
Signature	6b5f24e68ce6ddfd58f424e181fc48b7…

Release 2.3.0

Field	Value
Build	3a4df106ff2e5713
Gates	unit_pass_pct: pass; regression_pass_pct: pass; open_critical: pass; branch_cov_changed_code_pct: pass; upi_payout_p95_min: pass; fraud_api_contract: pass
Reviews	59 PRs, 3 config inspections
Configuration audit	BLOCKED: fraud_rules.json change lacked fraud-owner approval; merged after approval
Test summary	926 of 930 passed
Signature	97d02c5d4b0af57a9cf1f1a8db5cf7e4…

Release 2.4.0

Field	Value
Build	17fdd6d996f8a2f8
Gates	unit_pass_pct: pass; regression_pass_pct: pass; open_critical: pass; branch_cov_changed_code_pct: pass; upi_payout_p95_min: pass; fraud_api_contract: FAIL (blocked until adapter fixed)
Reviews	59 PRs, 3 config inspections
Configuration audit	OK
Test summary	943 of 945 passed
Signature	27a464a4dbb4fa0afd15d240d063d8ef…

Release 2.5.0

Field	Value
Build	2f32eef6136008ed

Field	Value
Gates	unit_pass_pct: pass; regression_pass_pct: pass; open_critical: pass; branch_cov_changed_code_pct: pass; upi_payout_p95_min: FAIL (blocked, fixed day 9); fraud_api_contract: pass
Reviews	59 PRs, 3 config inspections
Configuration audit	OK
Test summary	957 of 960 passed
Signature	65a0e50c1eb5cf0961ab269775fd3dc3…

Release 2.6.0

Field	Value
Build	faf5adb690bcd467
Gates	unit_pass_pct: pass; regression_pass_pct: pass; open_critical: pass; branch_cov_changed_code_pct: pass; upi_payout_p95_min: pass; fraud_api_contract: pass
Reviews	59 PRs, 3 config inspections
Configuration audit	OK
Test summary	974 of 975 passed
Signature	2b56e79018de0f7c8b106bd5c2fe7773…

ANNEXURE S – Projection Detail of the Control Responses (Task 3)

Response A – Add effort

Week	Tests executed that week	Cumulative executed	Cumulative found	Open
6	140	860	207	56
7	208	1067	238	53
8	208	1275	259	40
9	125	1400	268	16

Response B – Re-prioritise and descope

Week	Tests executed that week	Cumulative executed	Cumulative found	Open
6	155	875	206	57
7	155	1030	225	46
8	155	1185	239	29
9	79	1264	243	3

Response C – Remove the cause

Week	Tests executed that week	Cumulative executed	Cumulative found	Open
6	163	883	212	51
7	209	1092	243	39
8	209	1301	264	18
9	99	1400	272	0

Model with no change (calibration check)

Week	Model executed	Actual executed	Model found	Actual found	Model open	Actual open
6	875	900	211	214	62	58
7	1030	1080	233	238	54	48
8	1185	1230	249	252	39	28
9	1340	1340	260	260	20	20

The model reproduces week 9 exactly on executed and found tests; open defects are 4–12 higher than actual in weeks 6–8, so the model is slightly pessimistic about the backlog.

ANNEXURE T – Quality Monitoring and Control Procedure QMC-01 (the practice of Task 9)

Purpose

Measure, report, monitor and control the testing of each SureClaim release; decide release on defined criteria; keep configuration and review evidence.

Roles

QA analyst owns the procedure and the weekly report; release manager decides control actions and release; build and configuration engineer owns baselines and audits; test lead owns the daily dashboard; moderators own inspections.

Daily

Dashboard reviewed in a 5-minute call: blocked tests, oldest critical, builds deployed and audit status.

Weekly (Monday 10:00)

Prototype script run on the tracking export: 8 metrics, colour status, three charts, trend fit; one-page RAG with charts sent to the release manager.

Control trigger

Any RED for two consecutive weeks requires a cause analysis within 2 days and a control action recorded in the report (Task 3 order: remove cause, then re-prioritise, then add effort).

Reviews

Plan B (Annexure O); inspection entry/exit criteria (Task 7); actions tracked in the review tool; decision rule of Task 8.

Configuration

Baseline per release candidate; CCB for controlled items; automated audit on every deployment and every 15 minutes in staging and production.

Stopping and release

Stopping rule of Task 4; go/no-go meeting with the RAG page, the trend chart, the cost check and the audit report; decision and residual-risk statement signed by the release manager.

Records

Weekly reports, review records, audit logs and the release decision kept for 7 years (assumed regulatory period).

ANNEXURE U – Release Risk Register and Severity Definitions

ID	Risk	P	I	Response
Q1	Critical defect in payout or fraud logic reaches production	3	5	Inspection of money/fraud code; 0-critical release rule
Q2	Wrong configuration file deployed again	2	5	Baselines, CCB, automatic audit
Q3	Testing stops because testers are exhausted, not because quality is high	3	4	Trend + high-risk test completion + cost check, not trend alone
Q4	Defect fixing remains the bottleneck	4	4	2-day fix target for critical/major; closure ratio on the RAG page
Q5	Review budget exceeded by re-inspections	2	3	Plan B reserve of 194 h
Q6	Metrics misused for individual appraisal	2	4	Policy: product-level reporting only
Q7	Cumulative data hides re-runs	4	2	Record executions and current status separately (Task 9)

Severity	Definition for SureClaim	Release rule
Critical	Wrong payout amount, payment to wrong party, fraud flag bypassed, data breach, service down	0 open
Major	Wrong function with workaround; wrong status; wrong document	≤ 5 open with workaround
Minor	Cosmetic, wording, layout	Logged

ANNEXURE V – Week-9 Status Pack (format A + B, as sent to the release manager)

Area	Status	Comment
Schedule	GREEN	1,340 of 1,400 executed (95.7 %); 60 tests left
Quality	RED	3 critical open (oldest 12 days); pass 90.3 %; density 11.8/KLOC > 10
Scope	AMBER	86 of 96 stories covered; 10 without an executed test
Risk	GREEN	Discovery 8/week; Rayleigh: ≈ 11 defects remain
Recommendation	–	HOLD release; re-check end of week 10 against the stopping rule



Figure 27: Week-9 chart pack – planned, executed and passed tests, weeks 1–9

ANNEXURE W – Measurement Plan for the Eight Metrics

#	Metric	Formula	Source / query	Frequency	Owner	Threshold
1	Schedule variance	executed − planned; % of planned	TM tool export	Daily / weekly	Test lead	RED > 15 % behind
2	Pass rate	passed ÷ executed (first runs)	TM tool	Weekly	Test lead	AMBER < 85 %
3	Open defects by severity, oldest critical	count by severity; max age	JIRA filter status != Closed	Daily	QA analyst	Critical > 7 days RED
4	Find / close rates	weekly created / resolved	JIRA created-vs-resolved	Weekly	QA analyst	Close < find 2 weeks
5	Defect density	found ÷ KLOC changed	JIRA + git diff --stat	Weekly	QA analyst	> 10 / KLOC
6	Requirement coverage (high-risk first)	stories with ≥ 1 executed test ÷ stories	TM tool links	Weekly	BA	< 90 % of target AMBER
7	Branch coverage of claim/fraud/payout	coverage.py on the CI build	CI artefact	Every build	Build engineer	< 80 % for these packages
8	Estimated remaining defects	Rayleigh fit K − found	Script on JIRA export	Weekly from week 4	QA analyst	Input to stopping rule

ANNEXURE X – Record of the Week-5 Control Meeting (illustrative template; causes are assumptions)

Item	Record
Date	Monday of week 6, 10:00–10:40
Present	Release manager, QA analyst, test lead, development lead, build engineer
Data	Gap 240 (25 %); backlog 60 and rising 5 weeks; closure ratio 66 %; staging down 9 h in week 5; 2 builds rejected by smoke test
Causes found	1 Staging shared with a performance test (blocked tests). 2 Defects waited 3–5 days for triage. 3 Unstable builds: 2 of 5 failed smoke
Actions	1 Dedicated staging window 09:00–18:00 for testers (build engineer). 2 Daily triage 15 min, 2-day fix target for critical/major (dev lead). 3 Smoke test in CI before deployment (build engineer). 4 Risk-ranked list of lowest 10 % tests prepared as fallback (QA)
Measure of success	Executed ≥ 200/week from week 7; closures > arrivals from week 6
Review date	Week 7 control meeting

Outcome in the actual data: closures exceeded arrivals from week 6 (38 > 36) and the backlog fell from 60 to 20 by week 9; execution reached 180 per week, below the 200 target, so the remaining gap of 60 tests at week 9 is consistent with only part of the cause being removed.

ANNEXURE Y – Consolidated Check Sheets for the LLM Runs

Task	What is given to the LLM	Reference used for checking	Measures recorded
2	Tracking table weeks 1–6; ‘write the week-6 status report’	Table 14 fact sheet	Correct / wrong / unsupported / invented statements
6	Review copy + S1–S9; ‘review this code’	Table 26 seeded defects	True positives, false positives, critical found
7	Project description; ‘write a review checklist’	Table 36 criteria; Annexure J checklist	Specific items, reachable defects, item count
8	Defect log of 12 comments; ‘write the review report’	Table 44 check sheet	Comments kept, severities, decision, inventions

For each run record the tool name and version, the date, the exact prompt and the full output in an appendix, so that the check can be repeated.

ANNEXURE Z – Sprint-Level Summary of the Task 10 Prototype

Sprint	Commits	Vendor commits	Found	Closed	Max UPI p95 (min)	Unreviewed config	AI API contract	Config audit
1	201	62	20	26	13.9	no	pass	OK
2	201	62	16	25	13.9	no	pass	OK
3	196	70	19	20	13.9	yes	pass	BLOCKED
4	195	60	13	28	13.1	no	fail	OK
5	209	60	19	21	46.0	no	pass	OK
6	191	58	20	27	13.9	no	pass	OK

Sprint 3 contains the unreviewed vendor change (blocked by the policy check in C), sprint 4 the AI-API contract break (blocked by the gate until the adapter was fixed), and sprint 5 the UPI payout slowdown (gate failed from day 6).